

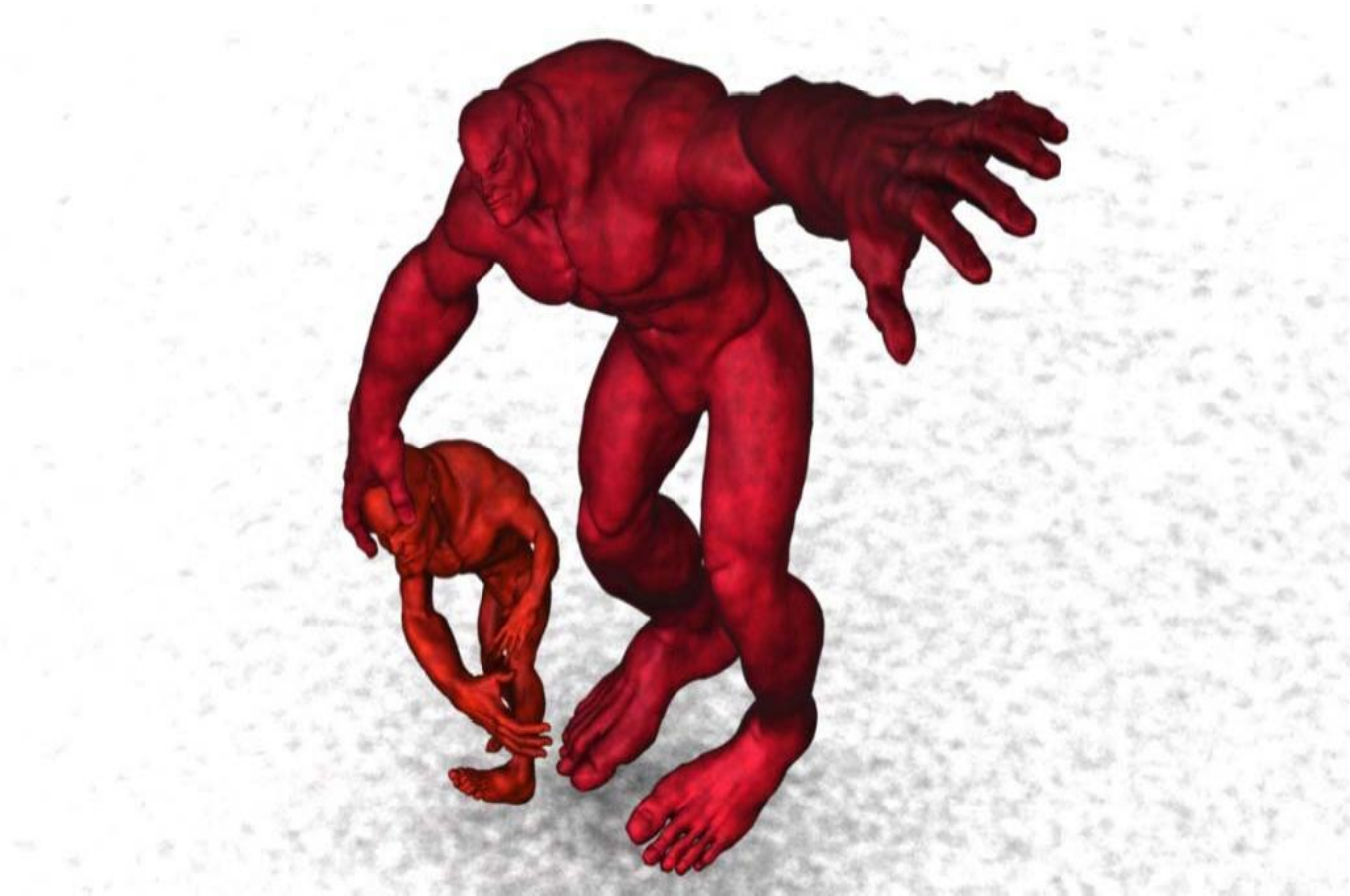
# 网格变形

高林

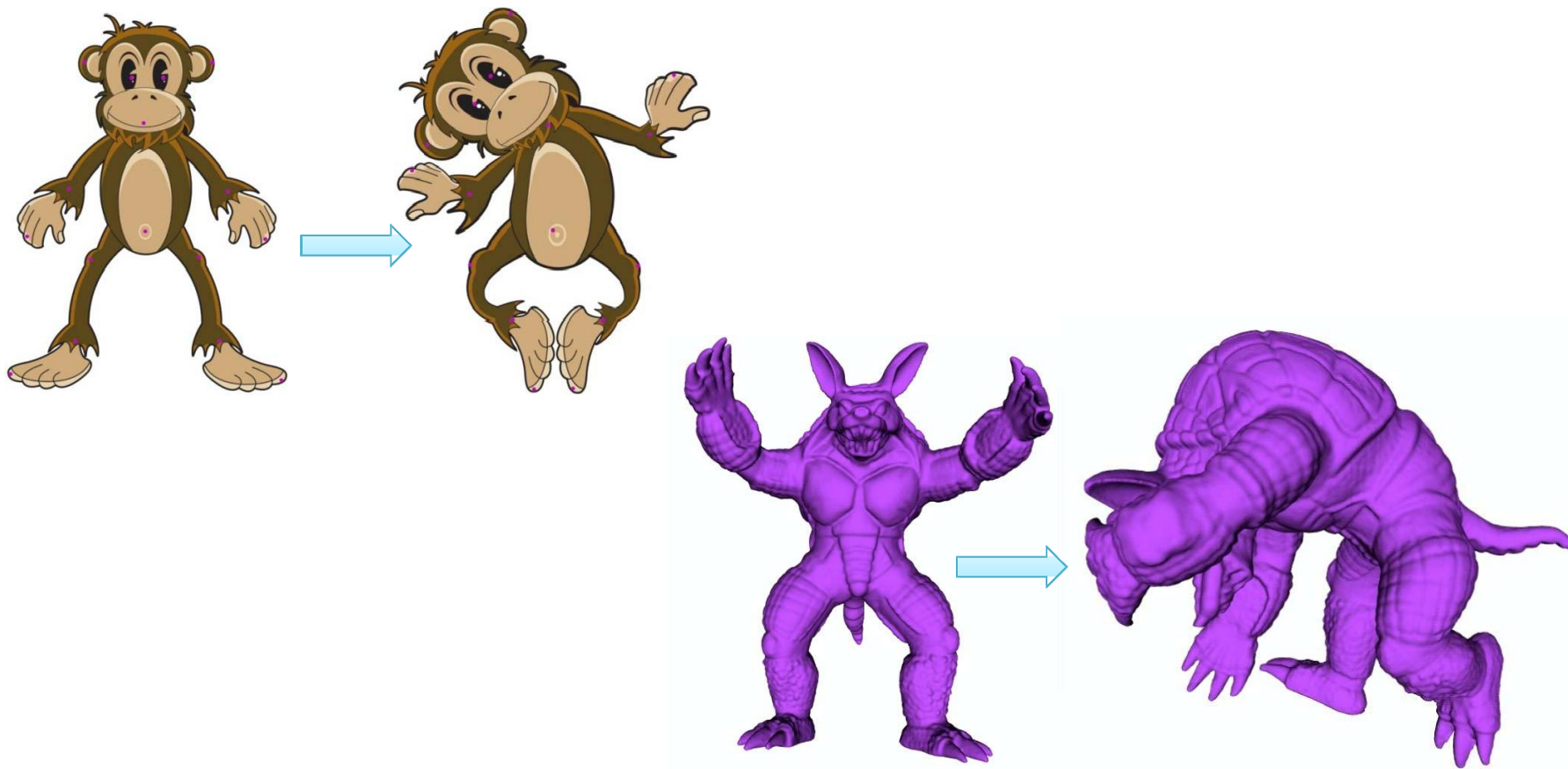
中国科学院计算技术研究所

# 网格变形

---

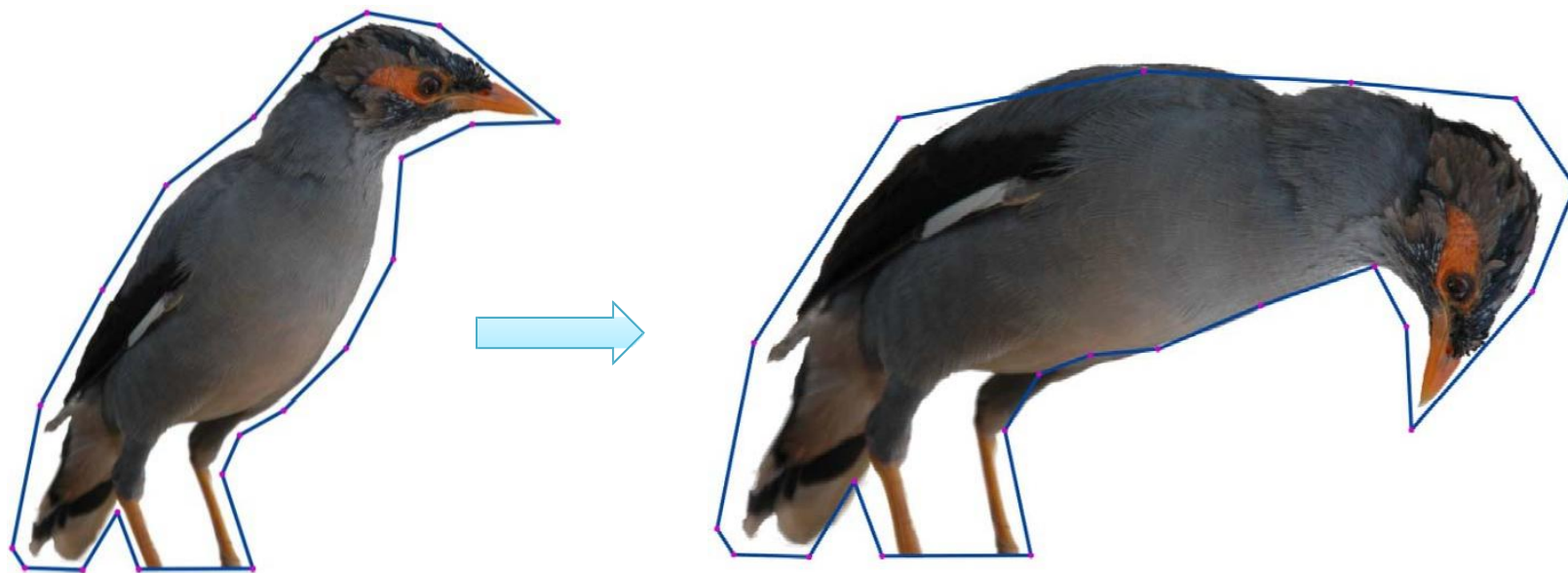


# 网格变形



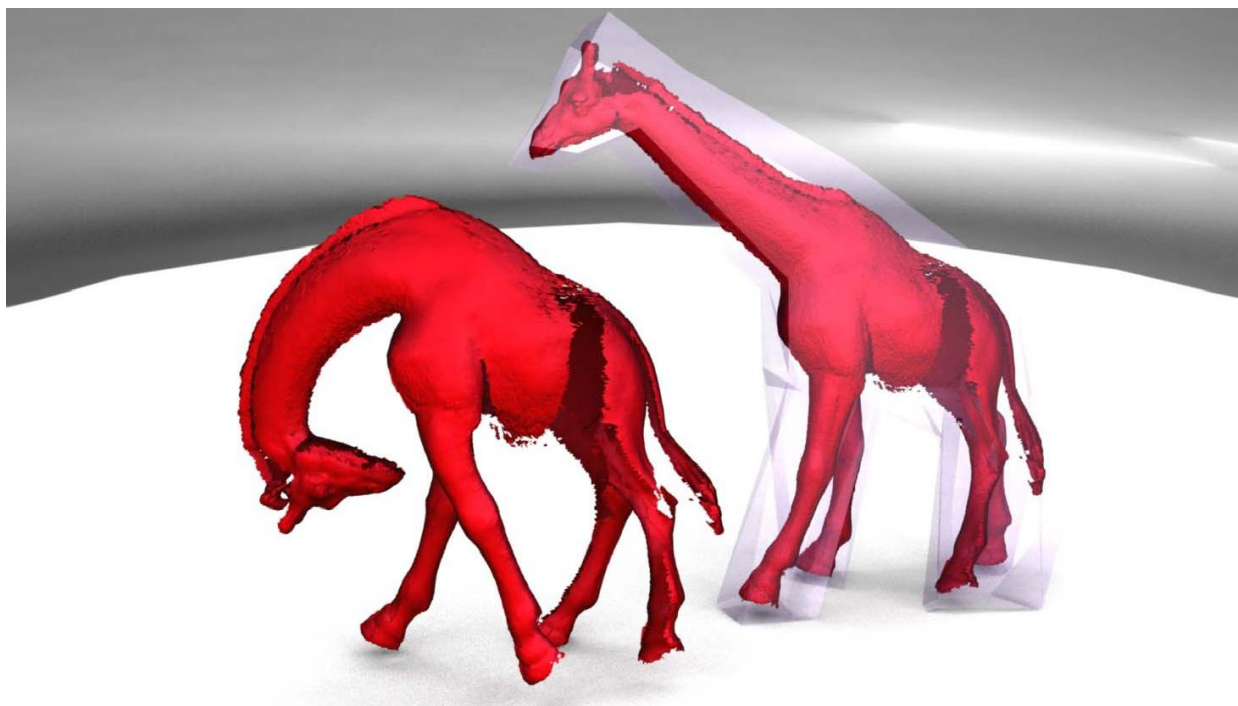
# 网格变形

- 更加容易的建模 – 通过对已有模型进行变形来生成新的模型



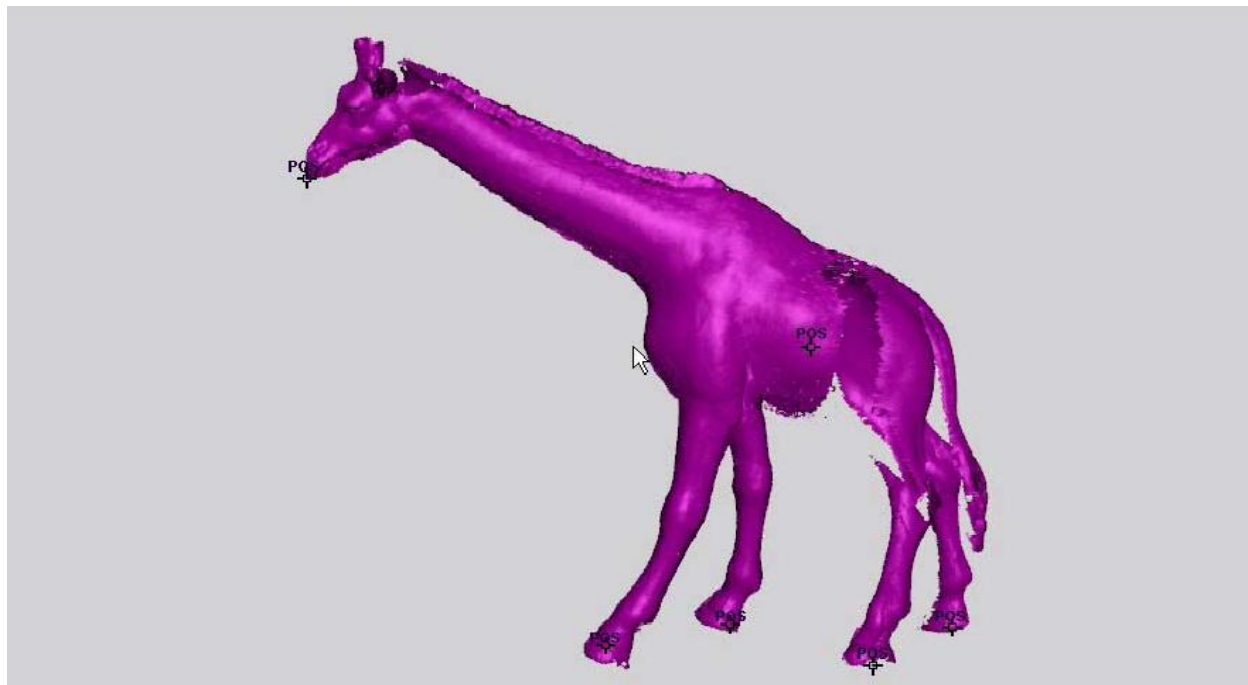
# 网格变形

- 更加容易的建模 – 通过对已有模型进行变形来生成新的模型



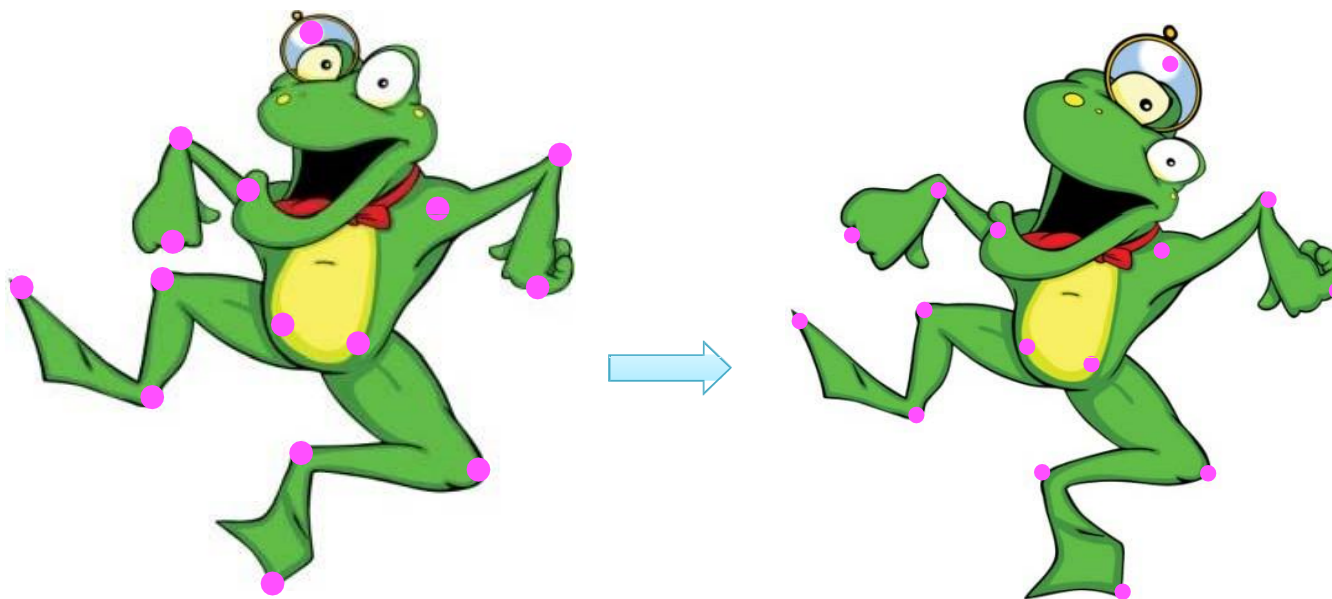
# 网格变形

- 摆正姿势用于生成动画



# 网格变形挑战

- 用户使用尽量少的交互...

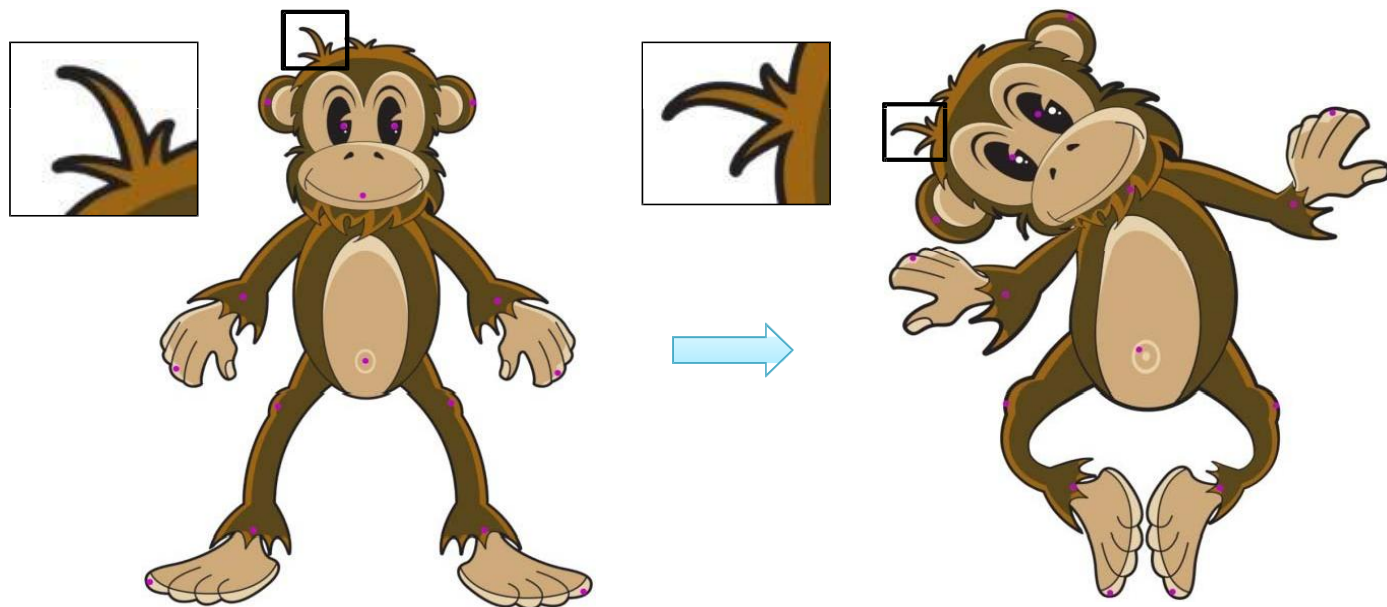


...其余的部分由算法自动生成



# 网格变形挑战

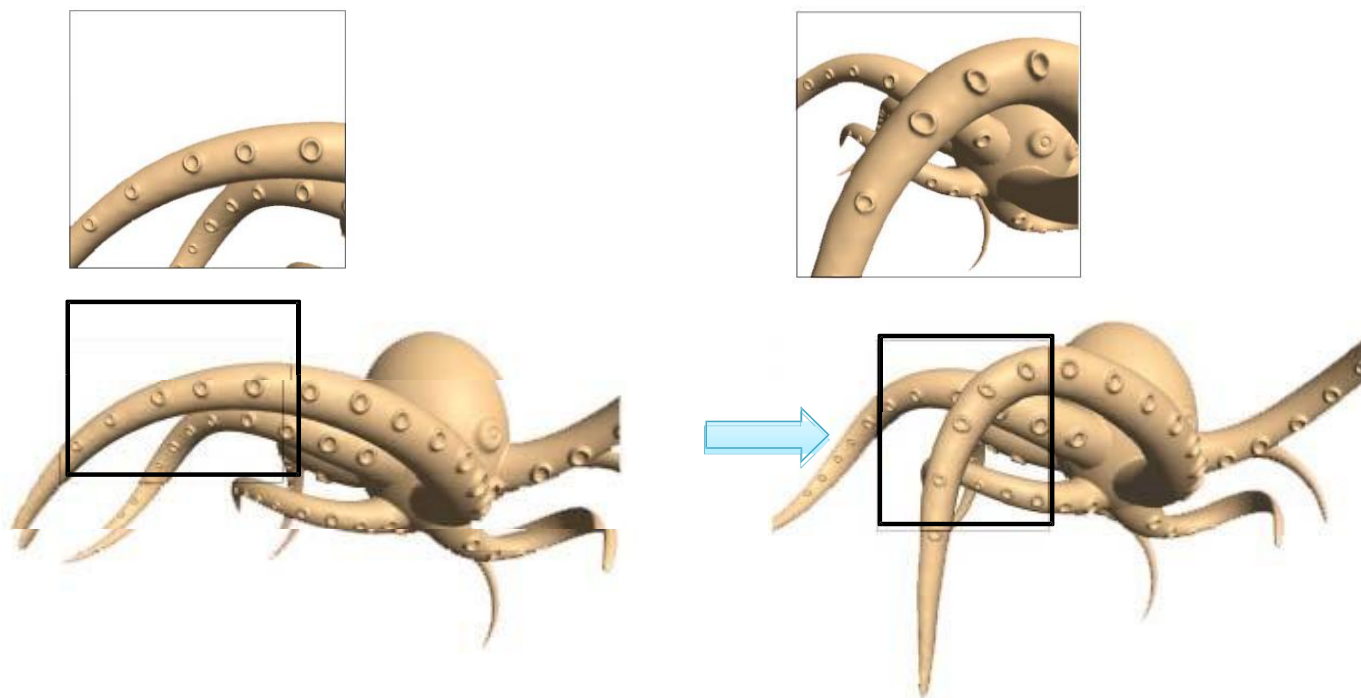
- “符合直觉的变形” 全局形变 + 局部细节保持





# 网格变形挑战

- “符合直觉的变形” 全局形变 + 局部细节保持

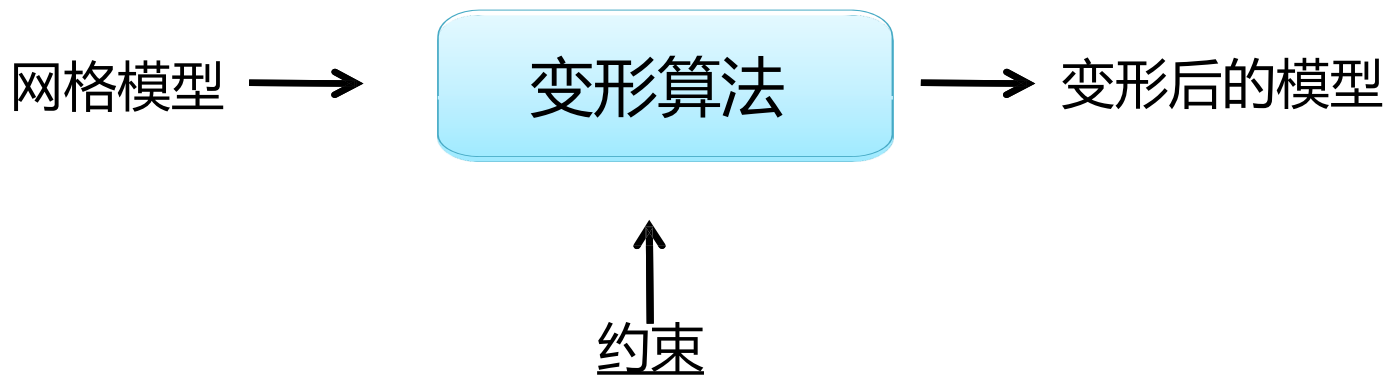


# 网格变形挑战

## ■ 高效



# 网格变形的基本算法框架



**顶点：**“控制顶点位置的移动”

**朝向/缩放：**“控制顶点的旋转和缩放的约束”

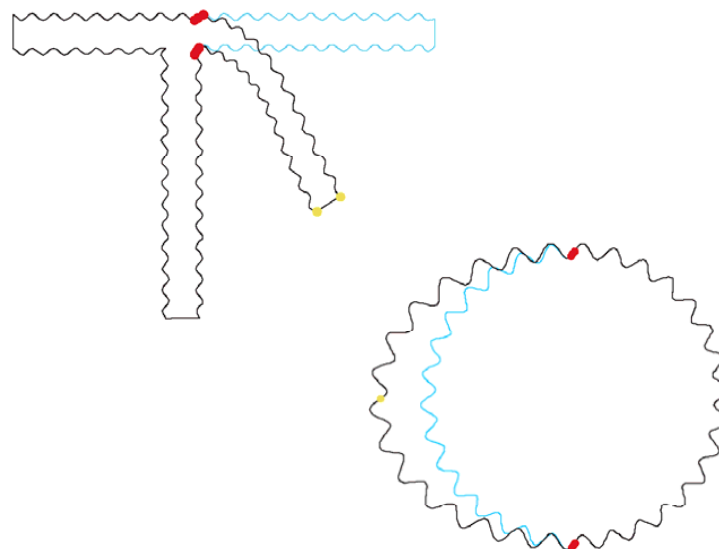
**其他的模型特征：**  
曲率，周长， ...

[ 参数化也是变形的一种形式: 约束 = 处处曲率为零]

# 方法

## ■ 曲面变形

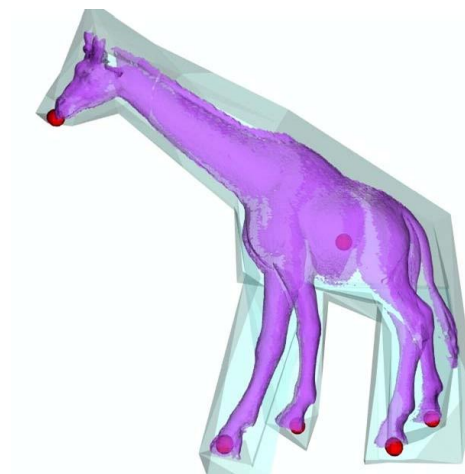
- ◆ 模型定义为空壳
- ◆ 2D变形使用外在的曲线
- ◆ 3D变形使用外在的曲面
- ◆ 变形只定义在曲线、曲面上
- ◆ 变形和网格表示是紧耦合的



# 方法

## ■ 空间变形

- ◆ 模型定义为体素
- ◆ 2D情况下平面区域
- ◆ 3D情况下多面体网格
- ◆ 变形定义在模型的邻域内
- ◆ 可以被应用到任何模型表示



# 方法

---

## ■ 曲面变形

- ◆ 寻找 “变形不变的” 模型表示

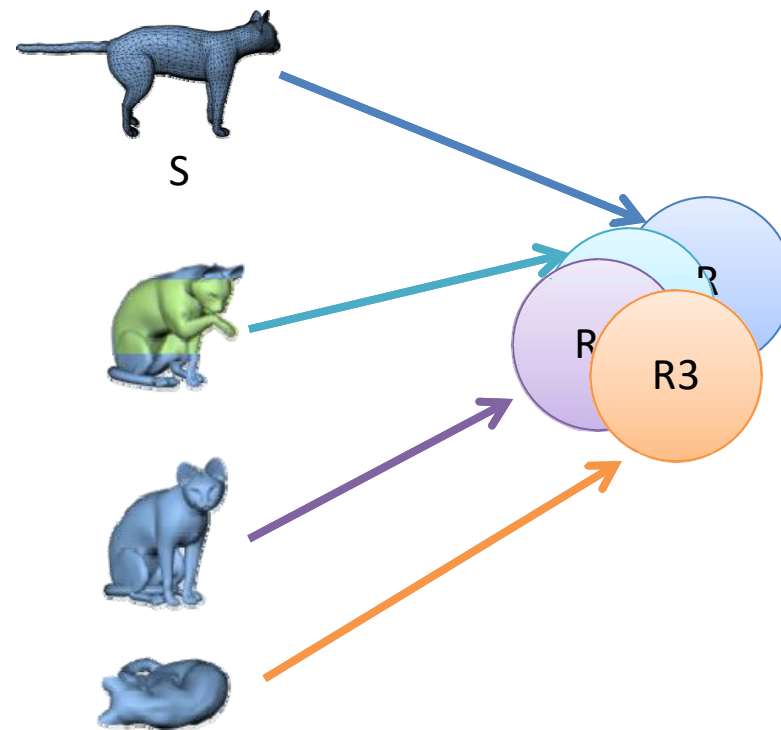
## ■ 空间变形

- ◆ 寻找具有 “优良属性” 的空间映射

# 曲面变形

## ■ 设置:

- ◆ 选择模型模型表示  $f(S)$
- ◆ 给定  $S$  寻找  $S'$  使其  $S'$  满足外在的约束  $f(S') = f(S)$  (或者尽量接近) 一个优化问题





# 模型表示

---

## ■ 什么是好的模型模型?

- ◆ 模型表示应对以下属性保持不变: 全局平移、全局旋转、全局缩放? 取决与具体的应用
- ◆ 变形后的模型应该具有接近的模型表示
- ◆ 接近等距变换的变形
- ◆ 局部平移 + 旋转
- ◆ 接近共形变换的变形
- ◆ 局部平移 + 旋转 + 缩放

# 模型表示

---

## ■ 鲁棒性

- ◆ 求解优化是否困难？
- ◆ 是否能找到全局最优解？
- ◆ 控制点的微小扰动是否会导致剧烈变化？

## ■ 高效性

- ◆ 是否可以以交互的速度进行求解？

# 模型表示

---

## ■ 优良的特征:

- ◆ 如果表示是坐标的线性函数, 那么变形是:
  - 鲁棒
  - 快速
- ◆ 但是表示不是旋转不变的! (对于大尺度的旋转)

# 模型表示

---

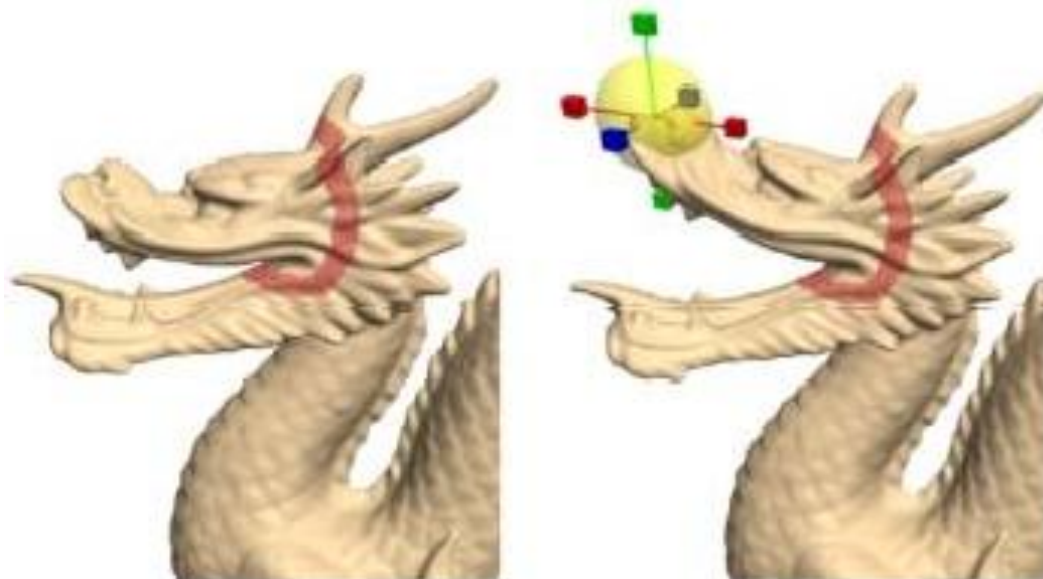
- Laplacian 坐标
- 边长 + 二面角
- 金字塔坐标
- 局部坐标架

....

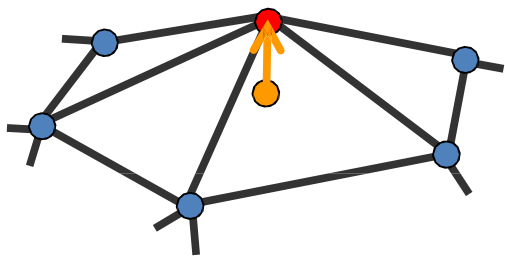
# Laplacian 坐标

## ■ 控制机制

- ◆ 用户拖动控制顶点
- ◆ 受影响的区域 (ROI)



# Laplacian 坐标



$$\delta = LV = (I - D^{-1}A)V$$

$I$  = 单位阵

$D$  = 对角阵 [ $d_{ii} = \deg(v_i)$ ]

$A$  = 邻接矩阵

$V$  = 网格上的顶点

$$\delta_i = \mathbf{v}_i - \sum_{j \in N(i)} \frac{1}{d_i} \mathbf{v}_j = \sum_{j \in N(i)} \frac{1}{d_i} (\mathbf{v}_i - \mathbf{v}_j)$$

法向的逼近 - 去掉平移后解是唯一的通过求解泊松方程  $LV = \delta$  来得到顶点坐标  $V$

# 变形

- 变形中的顶点  $C \subset V$

- $v'_i = u_i \quad i \in C$

- 最小化能量

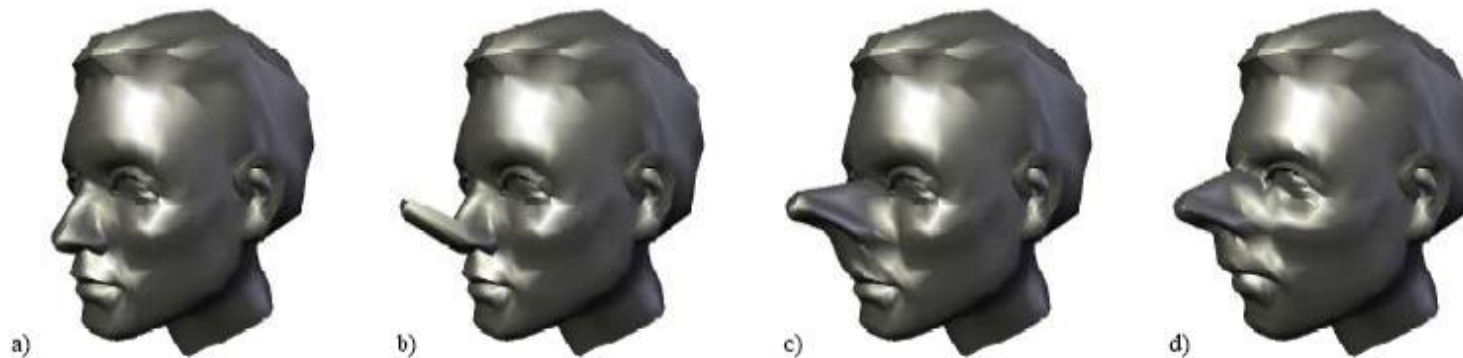
- $V' = \operatorname{argmin}_{V'} \sum_{i=1}^n \|\delta_i - L(v'_i)\|^2 + \sum_{i \in C} \|v'_i - u_i\|^2$

原有网格的  
Laplacian  
坐标

变形后网格  
的Laplacian  
坐标

用户限制

# Deformation



$$V' = \operatorname{argmin}_{V'} \sum_{i=1}^n \|\delta_i - L(v'_i)\|^2 + \sum_{i \in C} \|v'_i - u_i\|^2$$

原有网格的  
Laplacian  
坐标

变形后网格  
的Laplacian  
坐标

用户限制



# 最小二乘求解

---

变形中的顶点  $C \subset V$

$Ax = b$  具有  $m$  方程,  $n$  未知量

$$(A^T A)x = A^T b$$

通过伪逆求解:

$$x = A^+ b = \left[ (A^T A)^{-1} A^T \right] b$$

如果方程组是不定的:  $x = \operatorname{argmin}\{\|x\|: Ax = b\}$

如果方程组是超定的:  $x = \operatorname{argmin}\{\|Ax - b\|^2\}$

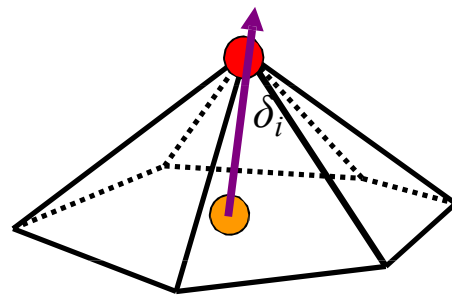
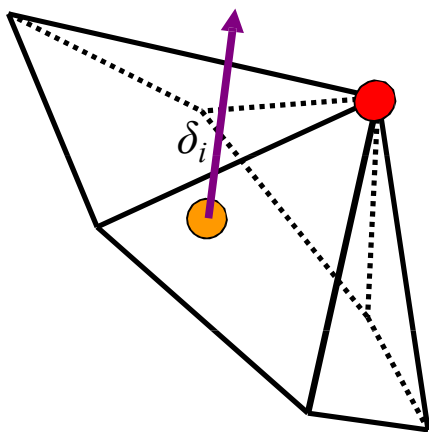
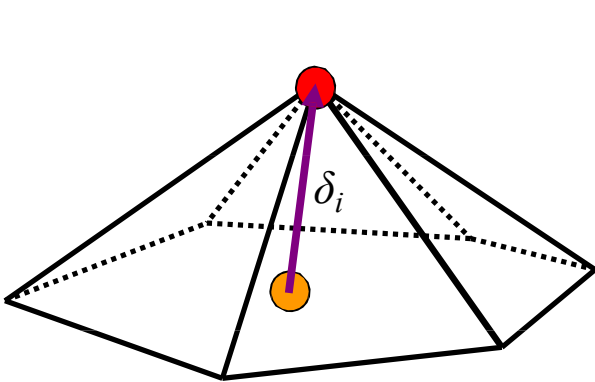
# Laplacian 坐标

平移不变?

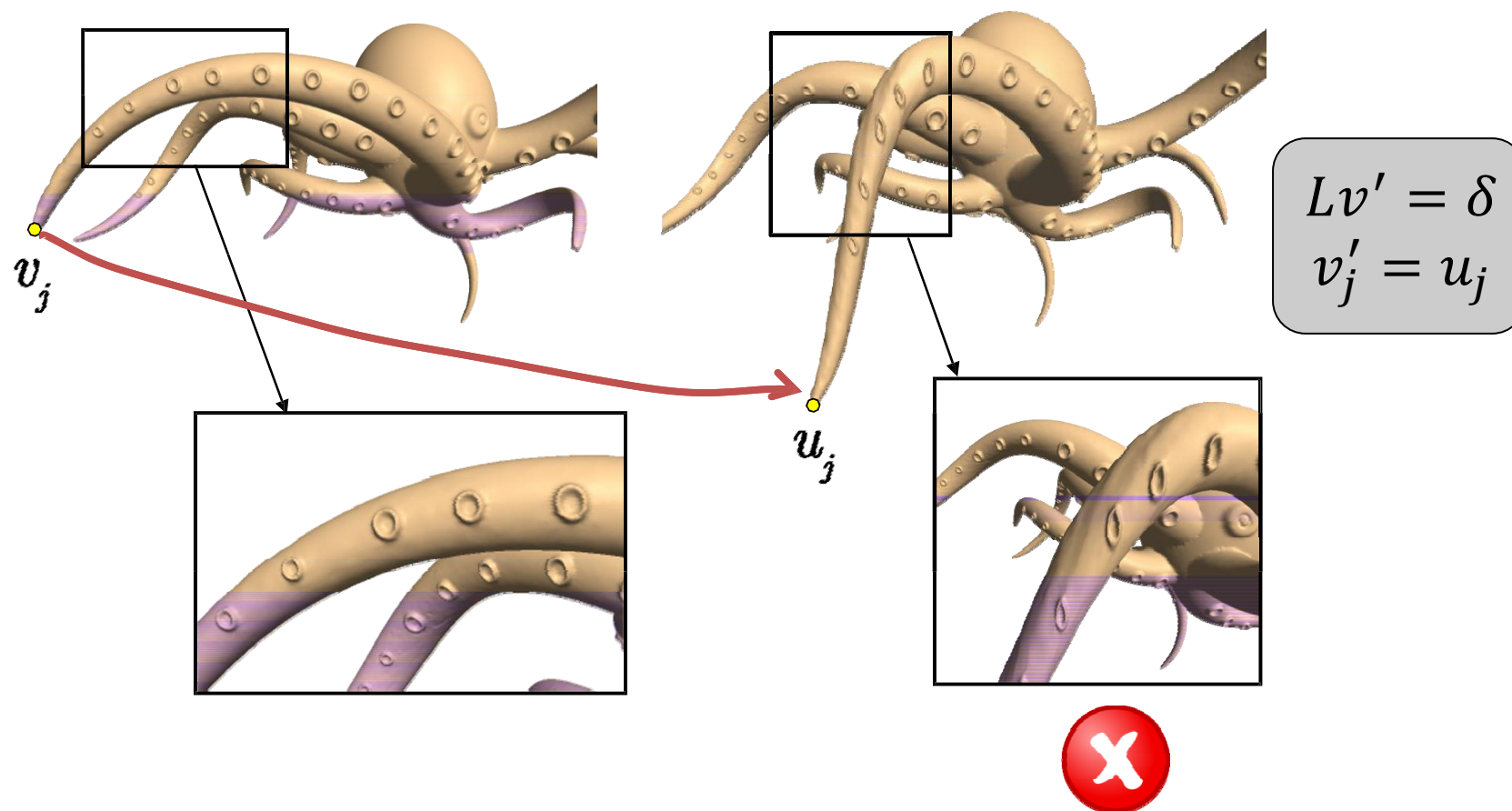


$$\delta_i = L(V_i') = L(V_i' + t) \quad \forall t \in R^3$$

旋转/缩放不变?



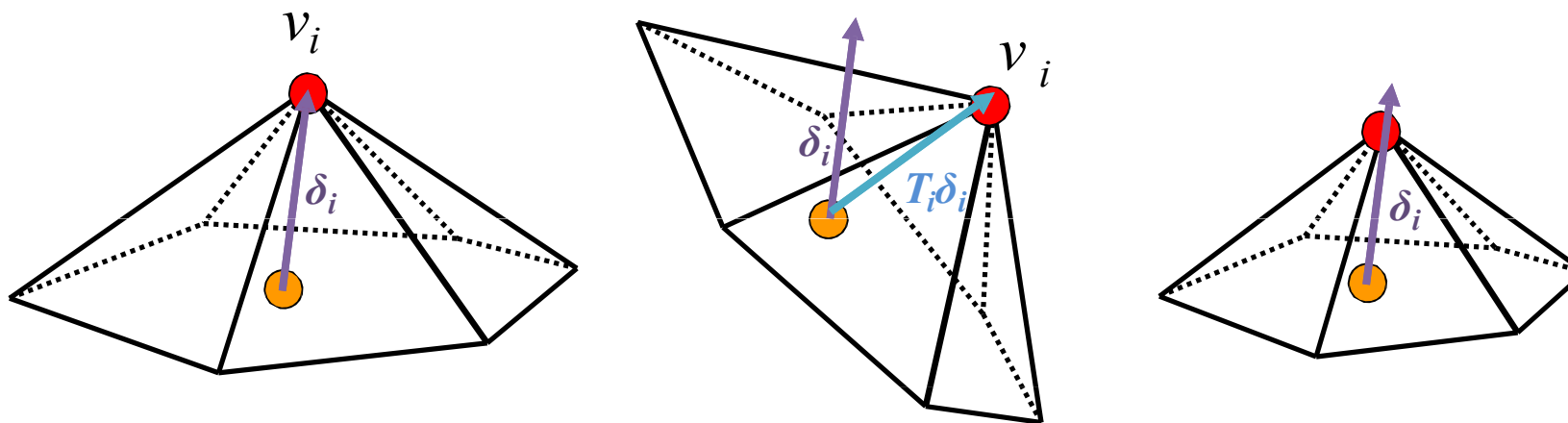
# 存在的问题



# 旋转不变坐标

- 模型表示应将旋转与缩放考虑进去

$$\delta_i = L(v_i) \quad T_i \delta_i = L(v'_i)$$



# 旋转不变坐标

---

- 模型表示应将旋转与缩放考虑进去

$$\delta_i = L(v_i) \quad T_i \delta_i = L(v'_i)$$

- 存在的问题:  $T_i$  依赖于变形后的模型顶点位置  $v'_i$

# 如何求解：隐式的表达变换

- 原理:同时求解局部变换和模型上的顶点坐标

$$V' = \arg \min_{V'} \left( \sum_{i=1}^n \|L(V'_i) - T_i(\delta_i)\|^2 + \sum_{j \in C} \|V'_j - u_j\|^2 \right)$$

相似变换

# 相似变换

■  $T_i$  成为相似变换矩阵的条件?

■ 2D:

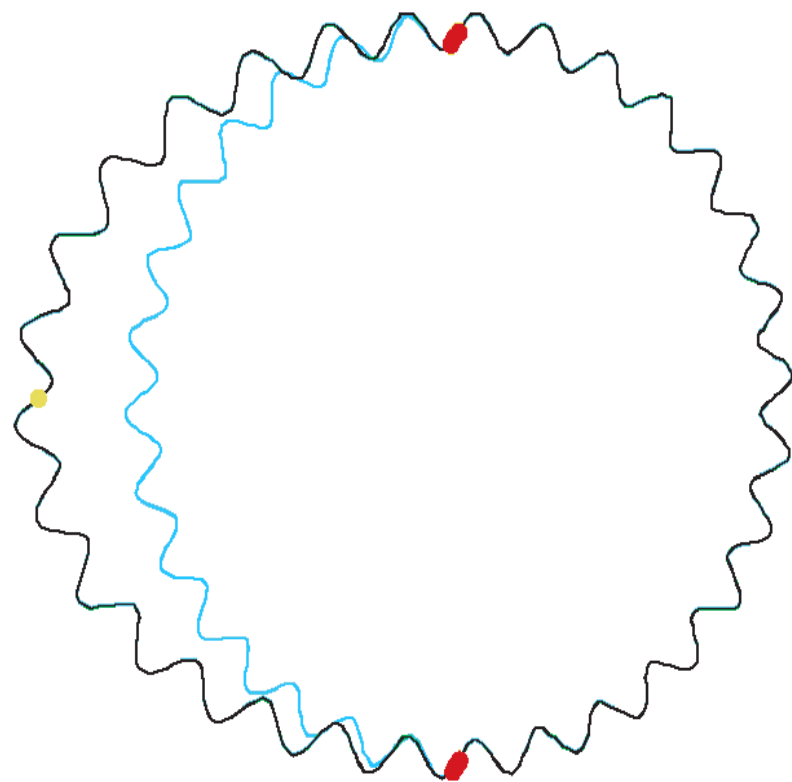
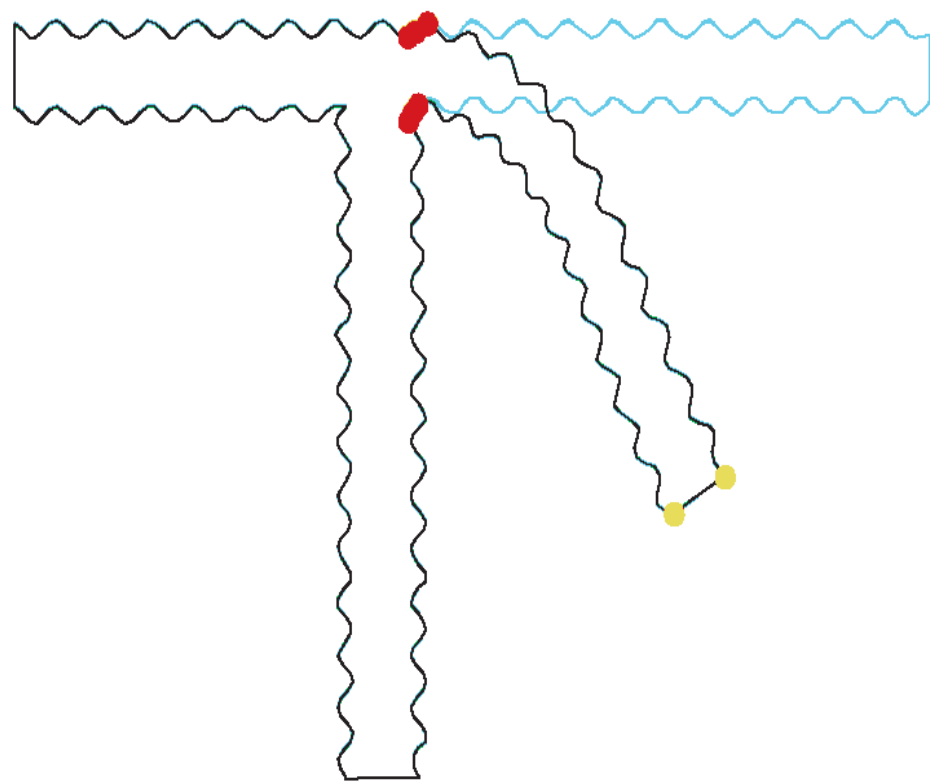
$$T_i = \begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos\theta & \sin\theta & d_x \\ -\sin\theta & \cos\theta & d_y \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} w & a & t_x \\ -a & w & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

引入的变量

缩放矩阵

旋转+平移

# 2D 下的相似变换





# 3D 下的相似变换

- 3D下是非线性的:

$$\begin{pmatrix} \text{rotation} + \\ \text{uniform scale} \end{pmatrix} = s \exp H = s(\alpha I + \beta H + h^T h)$$

- 线性化来降低二次能量

- ◆ 有效性: 小尺度的旋转

$H$  是  $3 \times 3$  的反对称矩阵,  $Hx = h \times x$

# Laplacian 坐标

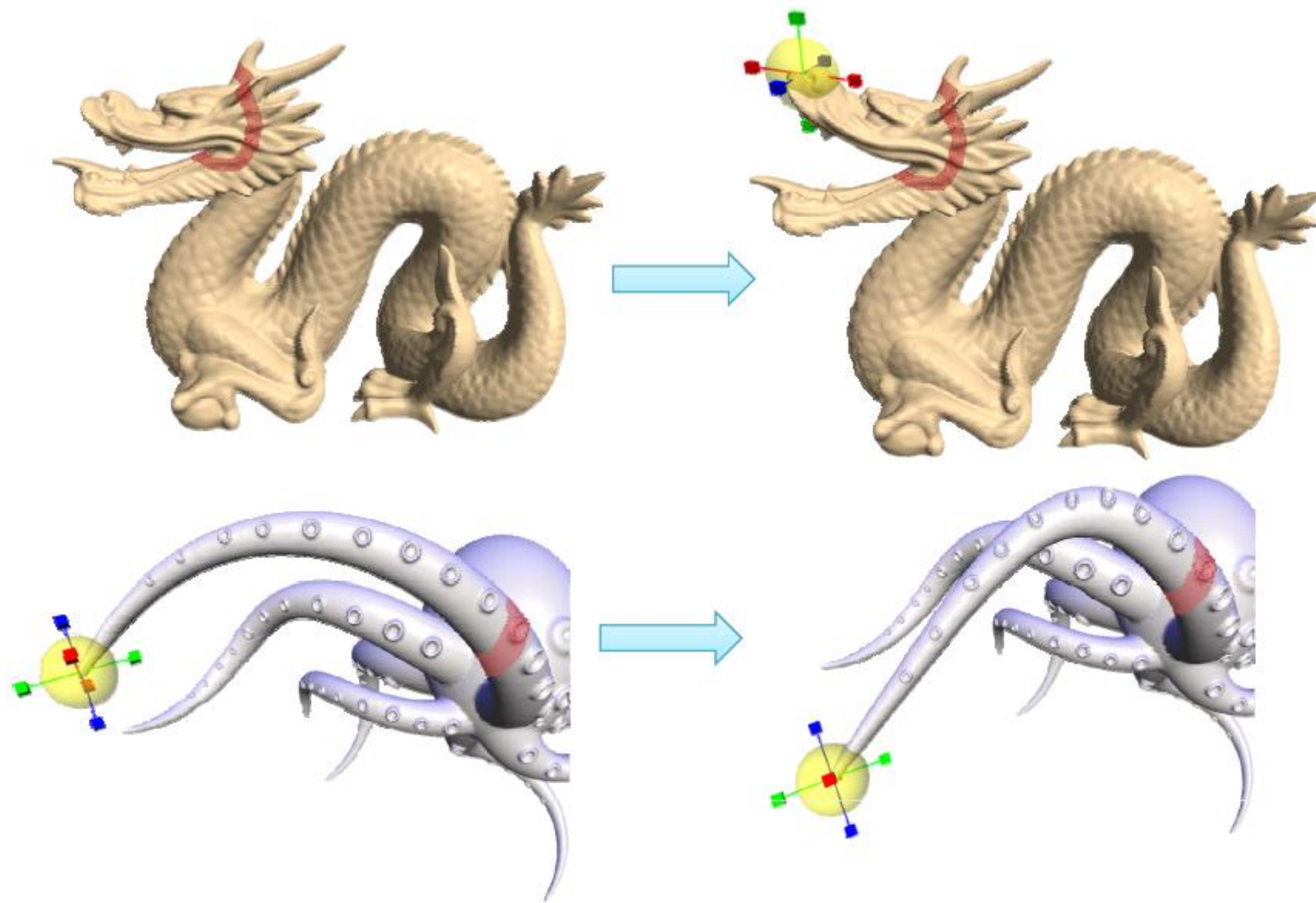
---

- 每一帧需要求解一个方程组

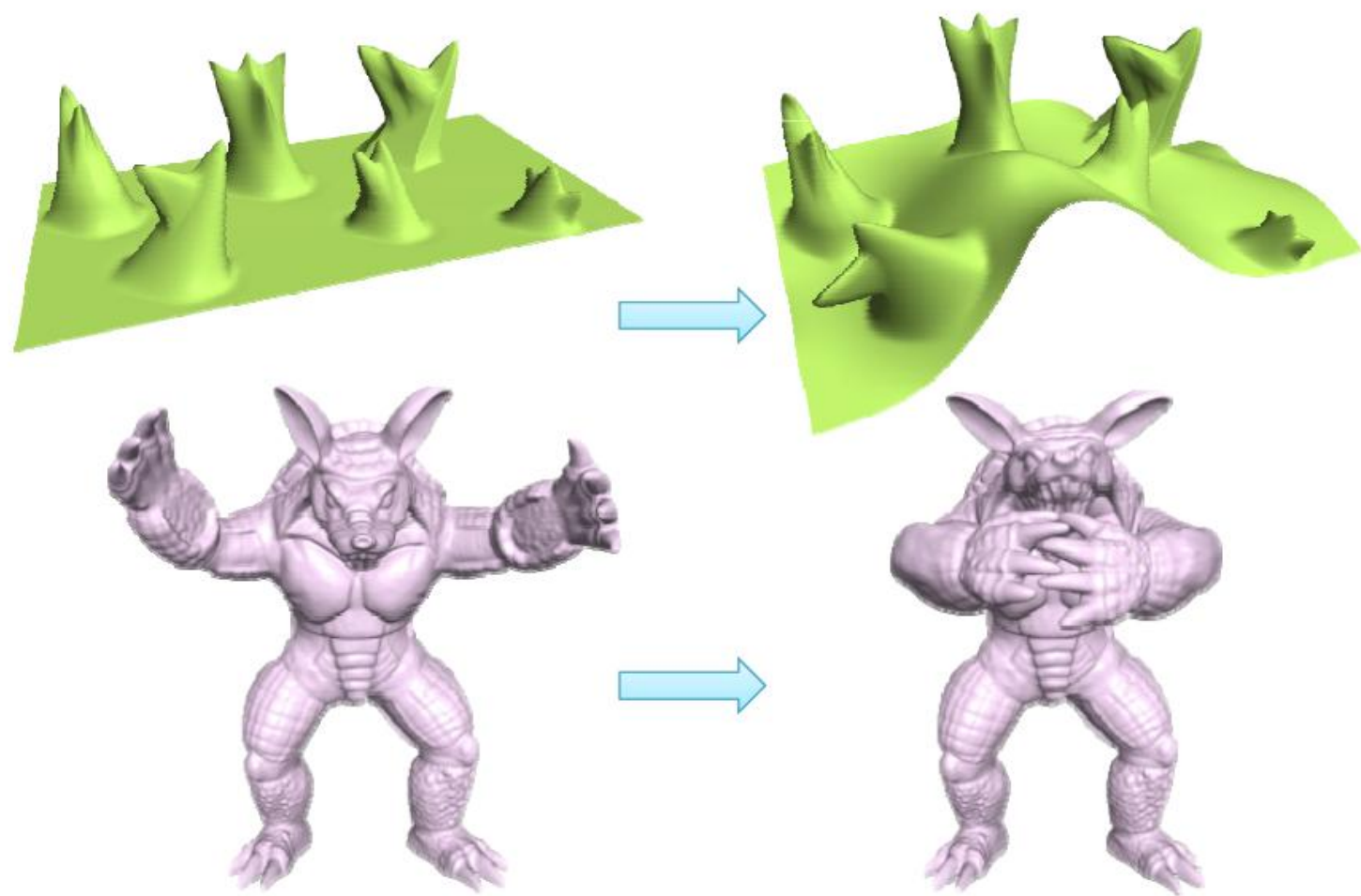
$$(A^T A)x = A^T b$$

- 预先进行Cholesky分解
- 每帧只需回代求解

# 结果



# 结果



# 局限: 大尺度旋转

| Approach                                                | Pure Translation                                                                    | 120° bend                                                                             | 135° twist                                                                           | 70° bend                                                                             |
|---------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Original model                                          |   |    |   |   |
| Laplacian-based editing with implicit optimization [60] |  |  |  |  |

# 如何得到旋转？

---

- Laplacian 坐标 – 通过优化求解得到
  - ◆ 问题: 非线性的
- 另一种方案: 通过控制顶点来传播

# 旋转传播

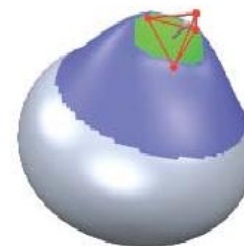
- 计算控制顶点处的 “deformation gradient”
- 提取出旋转与伸缩/错切分量
- 在ROI衰减的传播旋转



# Deformation Gradient

- 定义在控制顶点上的仿射变换

$$T(x) = Ax + t$$



- Deformation gradient 定义为:

$$\nabla T(x) = A$$

- 提取出旋转量  $R$  和伸缩/旋转量  $S$

$$A = U\Sigma V^T \Rightarrow R = UV^T \quad S = V\Sigma V^T$$



# 平滑的传播

## ■ 构建光滑的数值域 $[0,1]$

- ◆  $\alpha(x) = 1$       动的控制顶点
- ◆  $\alpha(x) = 0$       不动的控制顶点
- ◆  $\alpha(x) \in [0,1]$     中间的区域

## ■ 伸缩/错切分量:

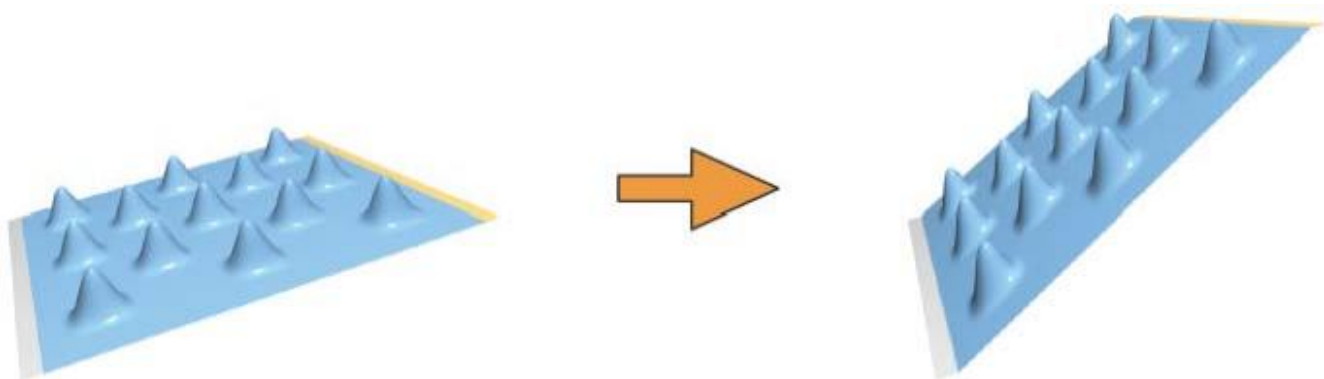
$$S(x) = \alpha(x)S(handle)$$

## ■ 旋转分量:

$$R(x) = \exp(\alpha(x) \log(R(handle)))$$

# 局限性

- 适用于旋转
- 平移不会改变 deformation gradient
  - ◆ “平移不敏感”



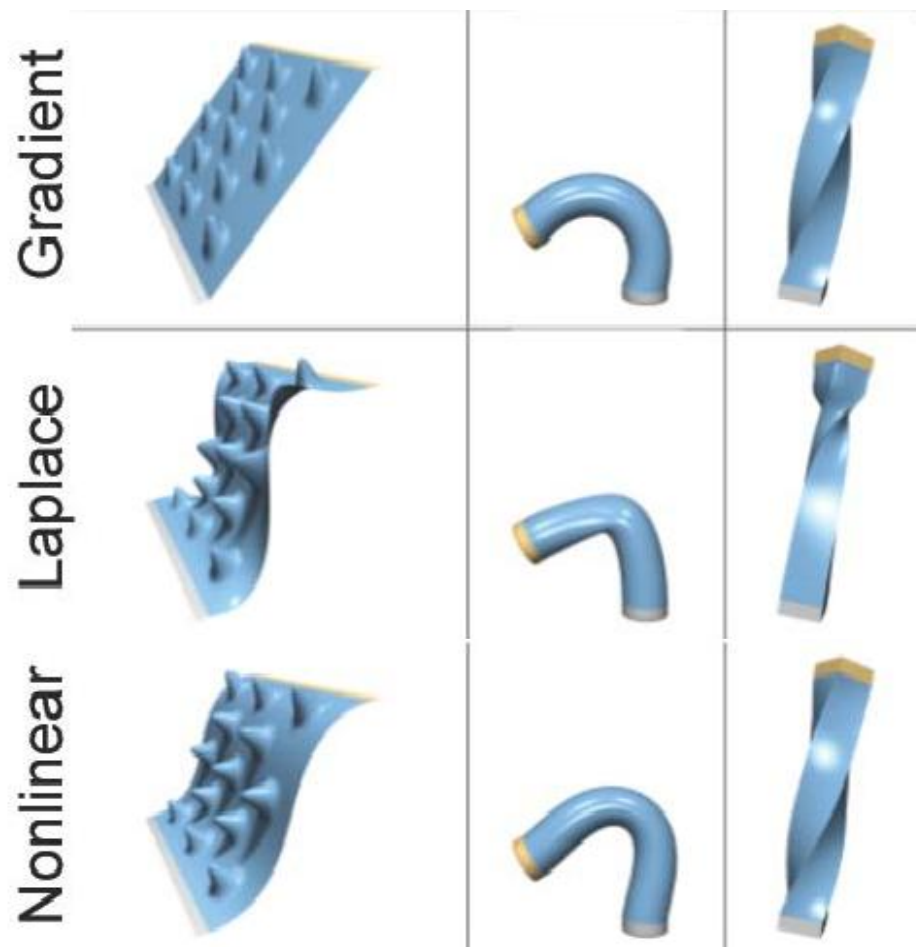
# 处理旋转所存在的问题

---

- 并不能用线性系统直接求解
- 如果控制顶点没有指定旋转内部区域也无法旋转，一些线性的方法适用于旋转，一些线性的方法适用于平移，没有方法同时适用于两样

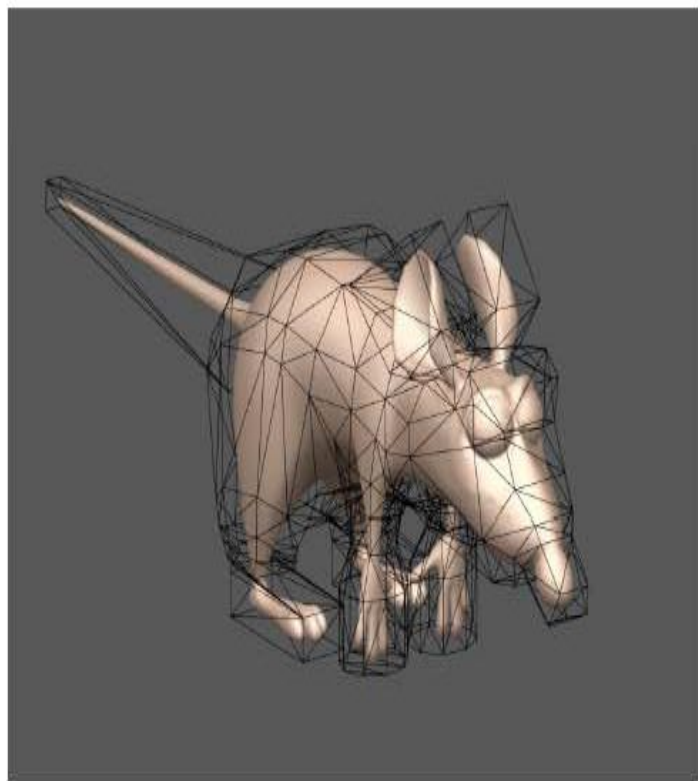
# 处理旋转所存在的问题

- 同时能够处理大尺度旋转和平移的方法是非线性的
- 计算也相对更加复杂



# 变形 II

---

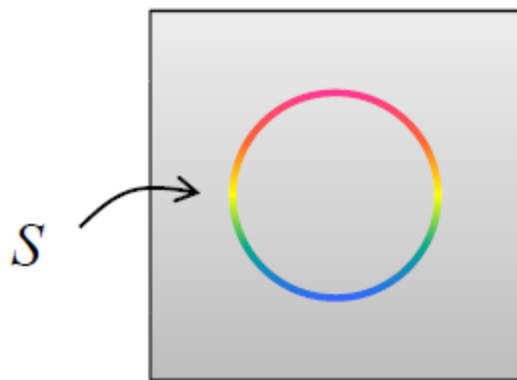


# 空间变形

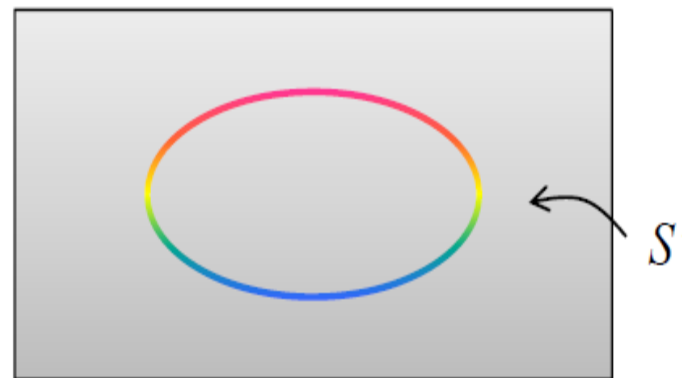
- 变形函数定义在空间上

$$f: R^n \rightarrow R^n$$

- 通过对空间  $S$  中的点应用函数  $f$  来进行变形



$$S' = f(S)$$
$$f(x, y) = (2x, y)$$



# 目标

---

- 可以适用于任何几何表示
  - ◆ 网格 (= 非流形, 多个组件)
  - ◆ 多边形集合
  - ◆ 点云
  - ◆ 体数据
- 复杂度与模型表示无关
  - ◆ 可以选择适于变形的模型复杂度

# 所需的属性

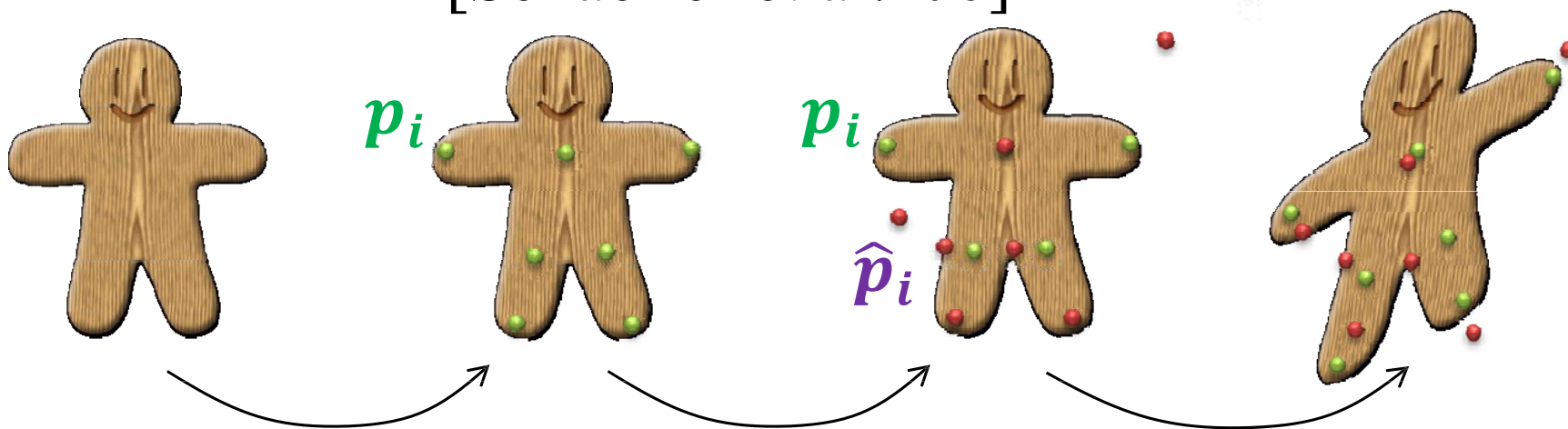
---

- 全局变换无关
  - ◆ 全局平移
  - ◆ 全局旋转
- 光滑
- 便于高效计算
  - ◆ “符合直觉的变形”？
  - ◆ 在变形的过程中添加约束



# MLS 变形

[Schaeffer et al. '06]



1. 控制顶点  $p_i$       2. 目标位置  $\hat{p}_i$

3. 寻找最佳的仿射

变换使得从  $p_i$  变为  $\hat{p}_i$

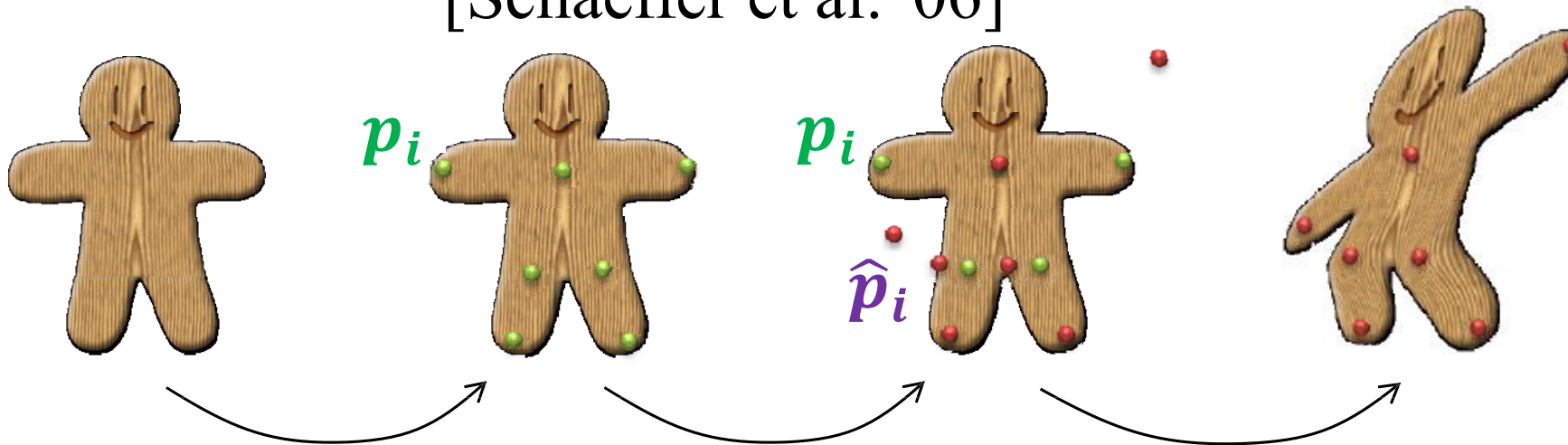
$$\min_{M,T} \sum_i |(M p_i + T) - \hat{p}_i|^2$$

4. 变形

$$f(v) = Mv + T$$

# MLS 变形

[Schaeffer et al. '06]



1. 控制顶点  $p_i$       2. 目标位置  $\hat{p}_i$

3. 寻找最佳的仿射

变换使得从  $p_i$  变为  $\hat{p}_i$

$$\min_{M, T} \sum_i \frac{1}{|p_i - v|} |(M p_i + T) - \hat{p}_i|^2$$

4. 变形

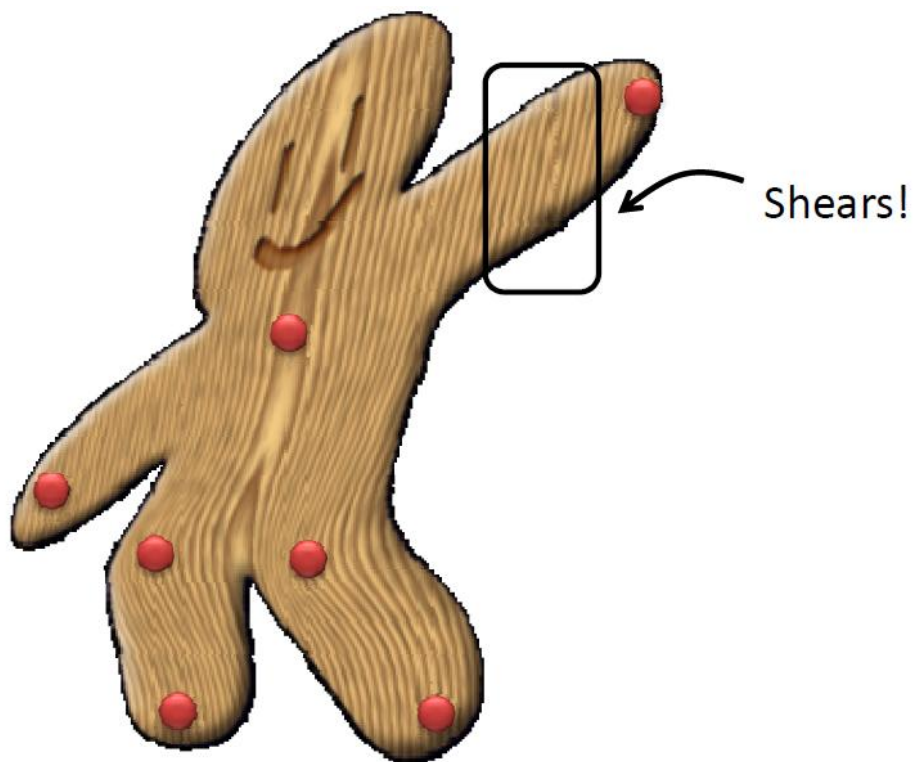
$$f(v) = M v + T$$

闭式解

离控制点越远权重越小，  
可以添加不同的范数

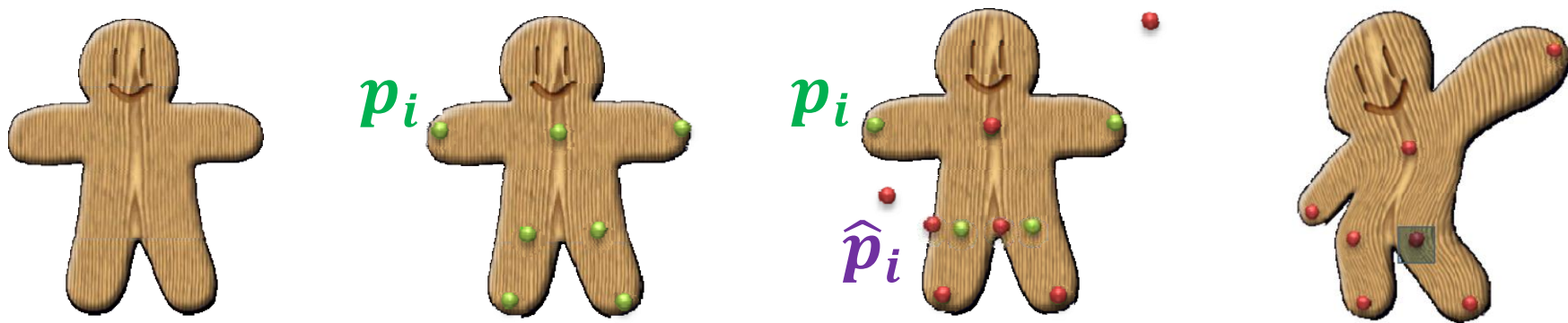
# 仿射变换

相似



# 相似变换

[Schaeffer et al. '06]



1. 控制顶点  $p_i$

2. 目标位置  $\hat{p}_i$

3. 寻找最佳的仿射

变换使得从  $p_i$  变为  $\hat{p}_i$

$$\min_{M, T} \sum_i \frac{1}{|p_i - v|} |(M p_i + T) - \hat{p}_i|^2$$

$$M = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

4. 变形

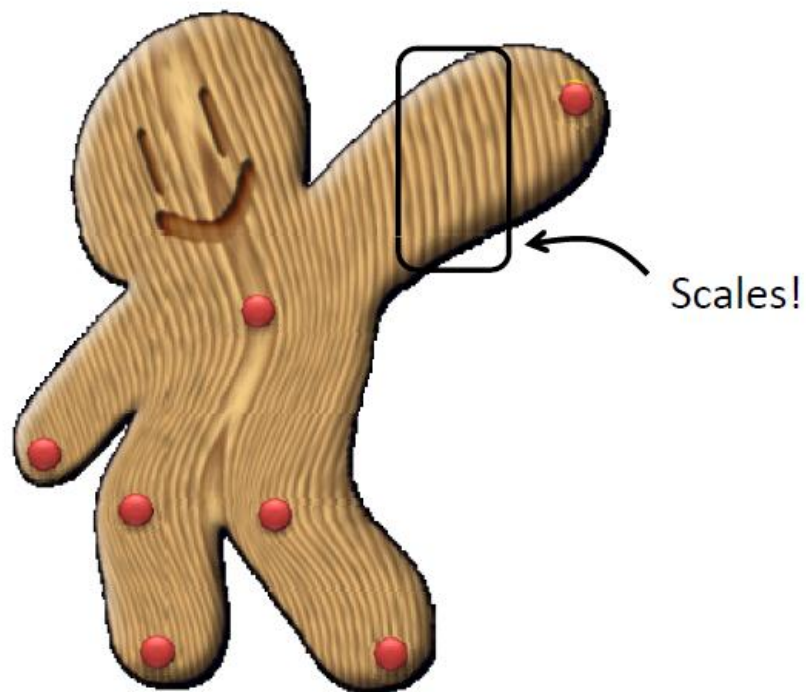
$$f(v) = Mv + T$$

闭式解

# 相似变换

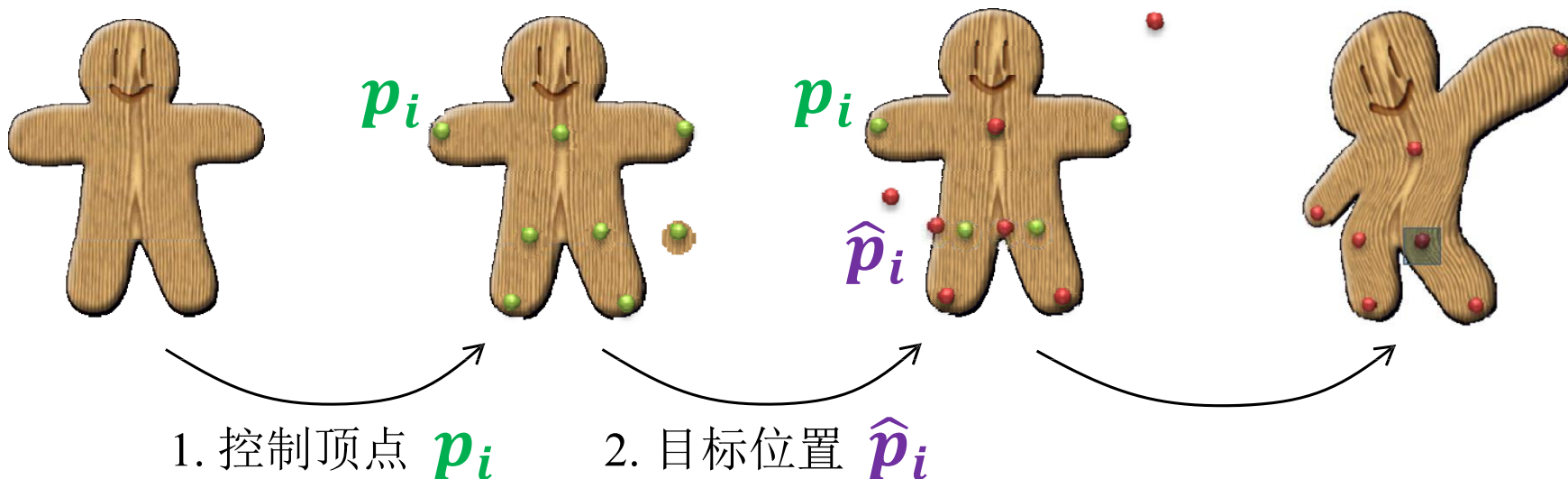
---

刚体



# 刚体变换

[Schaeffer et al. '06]



3. 寻找最佳的仿射

变换使得从  $p_i$  变为  $\hat{p}_i$   $\min_{M,T} \sum_i \frac{1}{|p_i - v|} |(Mp_i + T) - \hat{p}_i|^2$

4. 变形

$$f(v) = Mv + T$$

$$M = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

$$c^2 + s^2 = 1$$

# 比较



Thin-Plat  
[Bookstein '89]



仿射 MLS



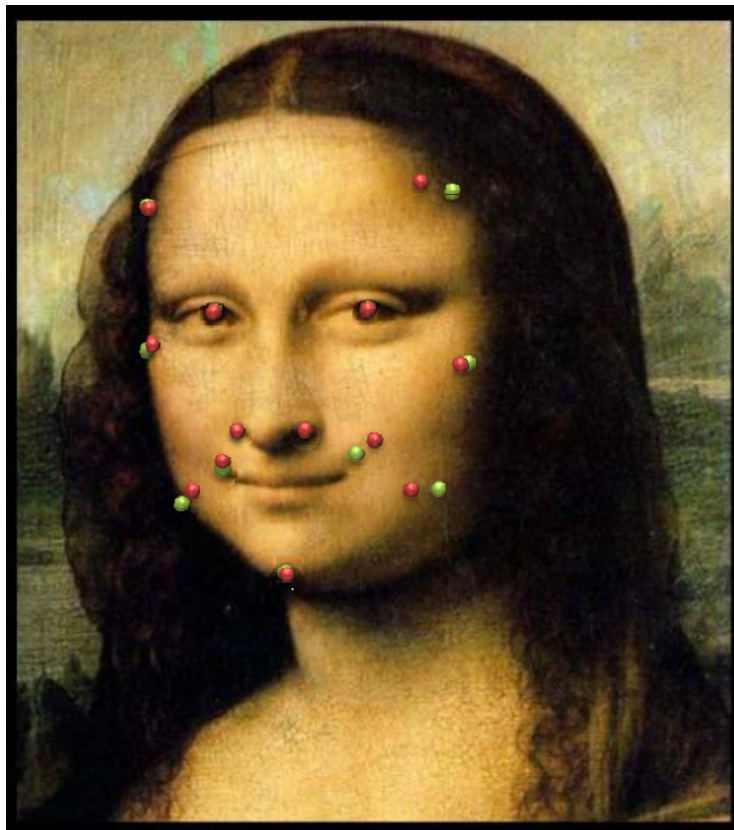
相似 MLS



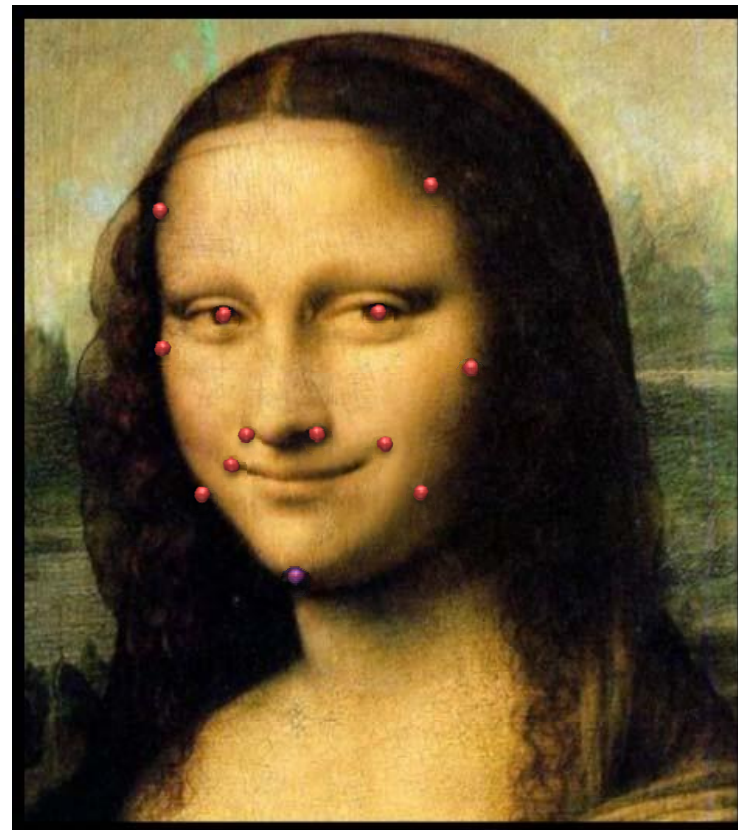
刚体 MLS



# 例子



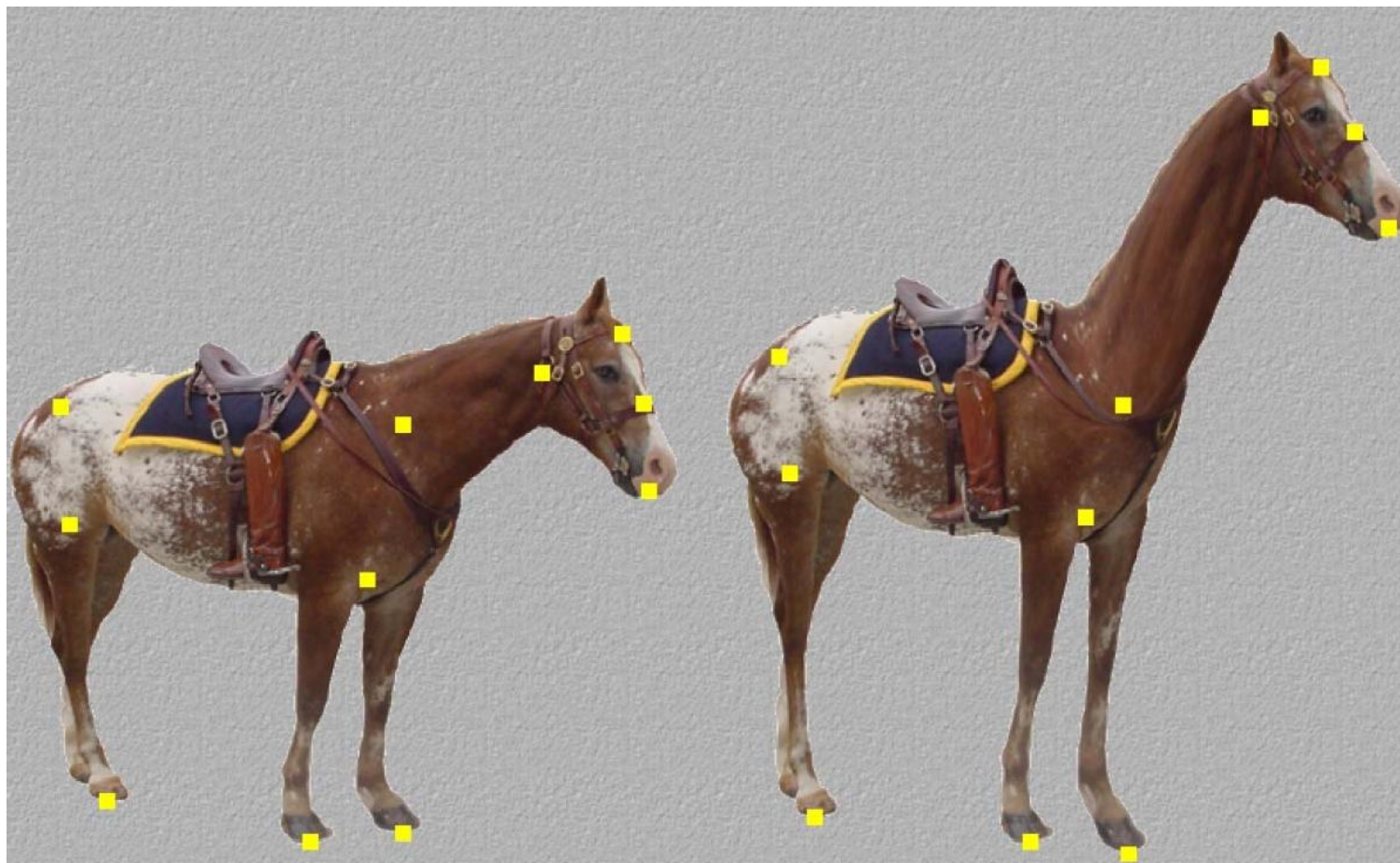
编辑前



编辑后



# 例子



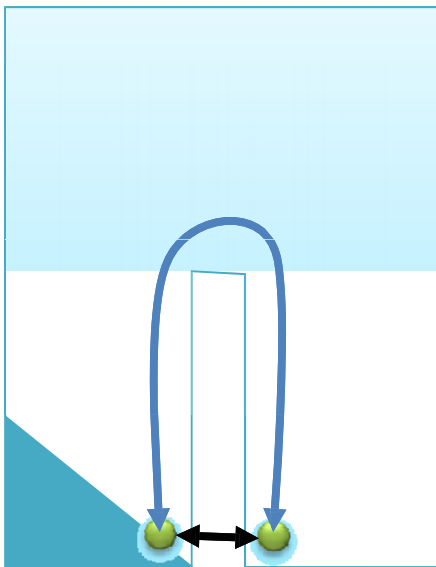
# 局限性

- 整个空间都会变形而不只是限制在模型上



# Pants问题

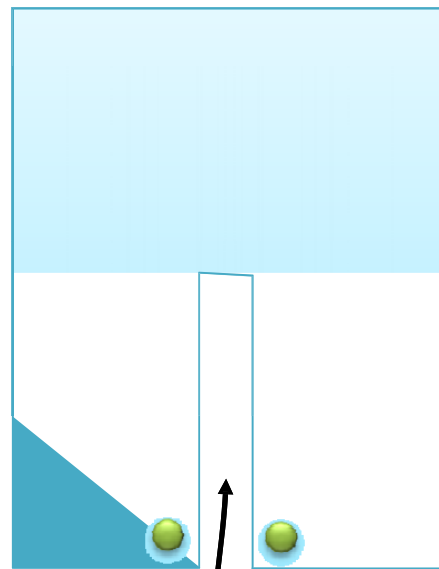
- 欧式距离小
- 测地距离大



# Pants问题

---

不需要考虑模型外的扭曲



# 求解方案: 使用包围盒

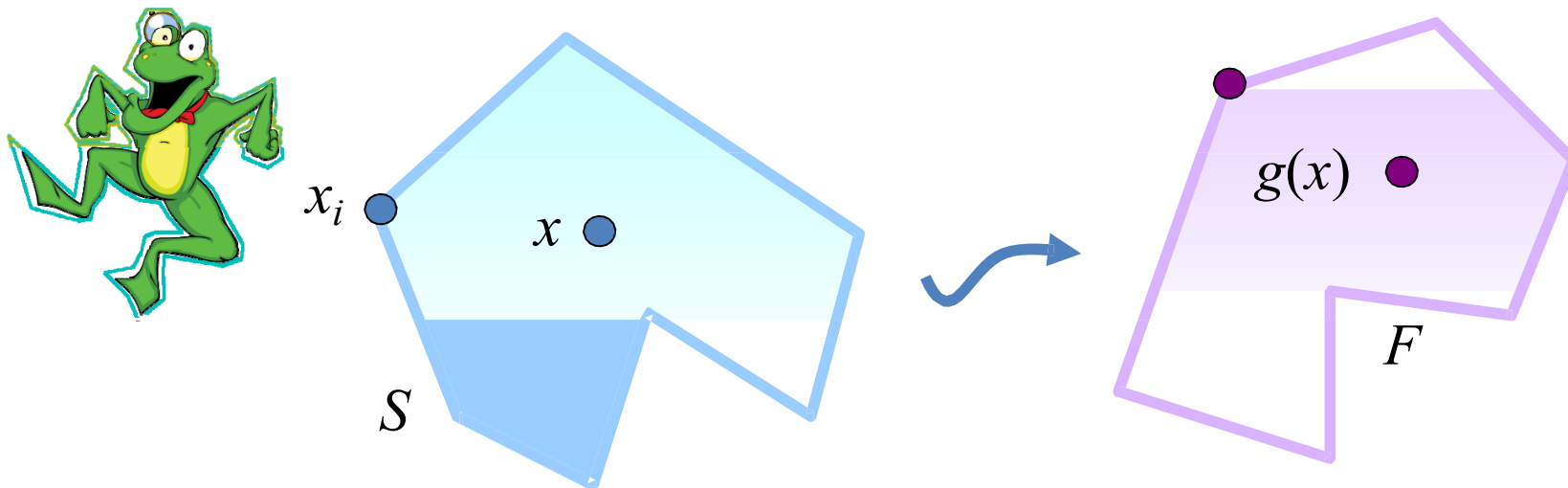
- 将整个模型放在一个包围盒里  $\Omega \subset R^n$
- 变形函数只定义在包围盒上



$$f: \Omega \rightarrow R^n$$

新的问题: 如何建立包围盒?

# 使用包围盒进行变形



$$S = \{x_1, x_2, \dots, x_n\}$$

源多边形

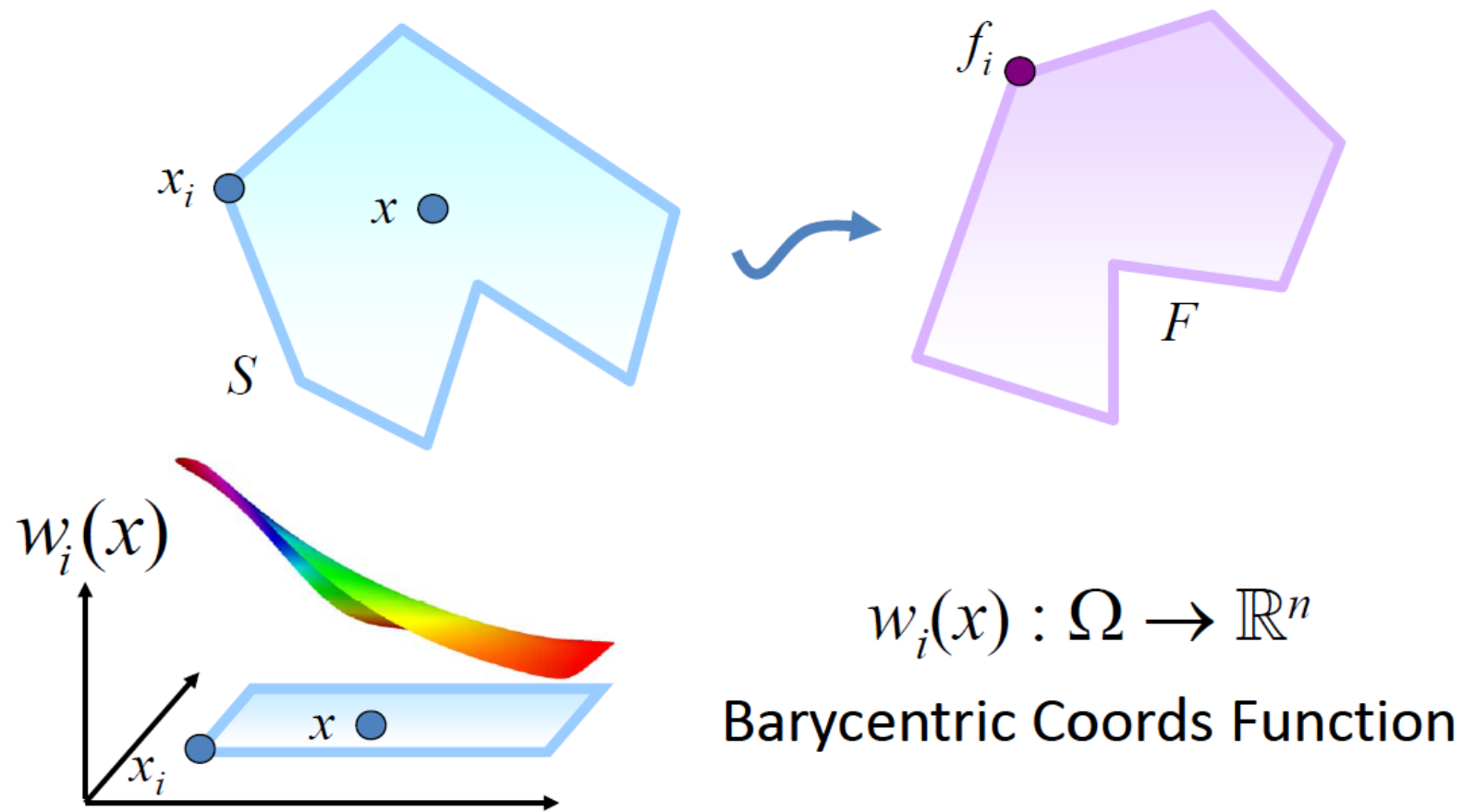
$$g(x) = ?$$

$$x_i \rightarrow f_i$$

目标多边形

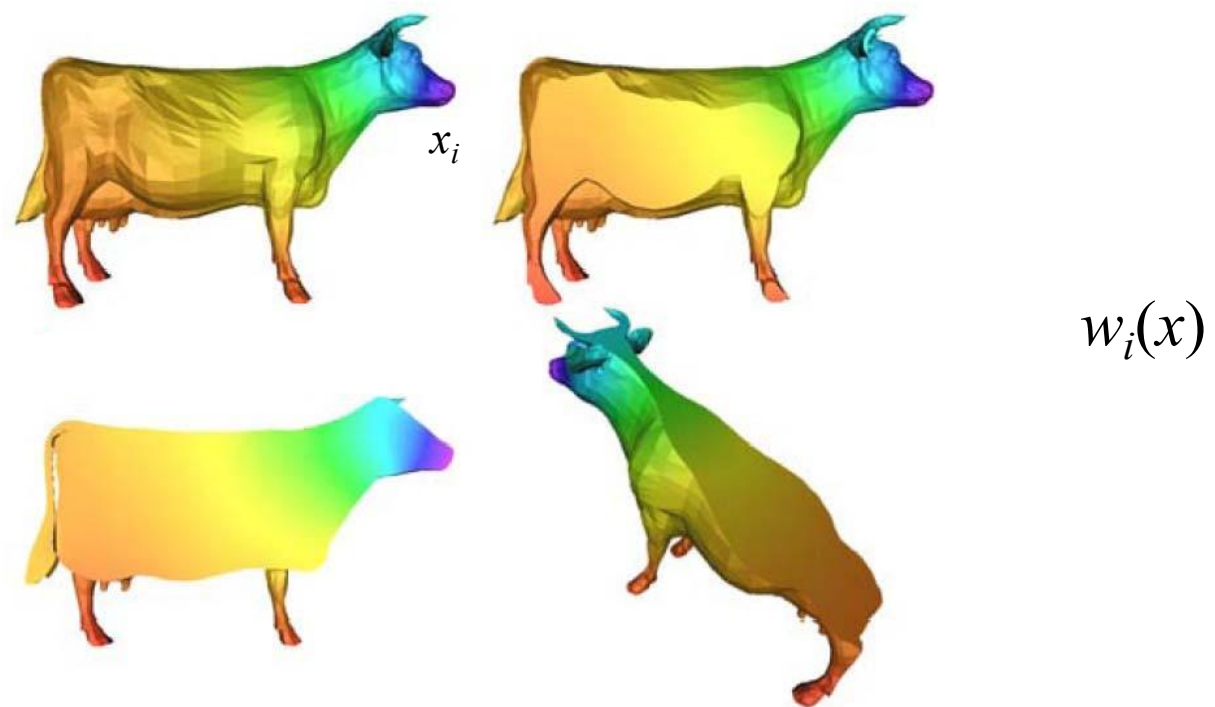
Interior?

# 重心坐标



# 例子

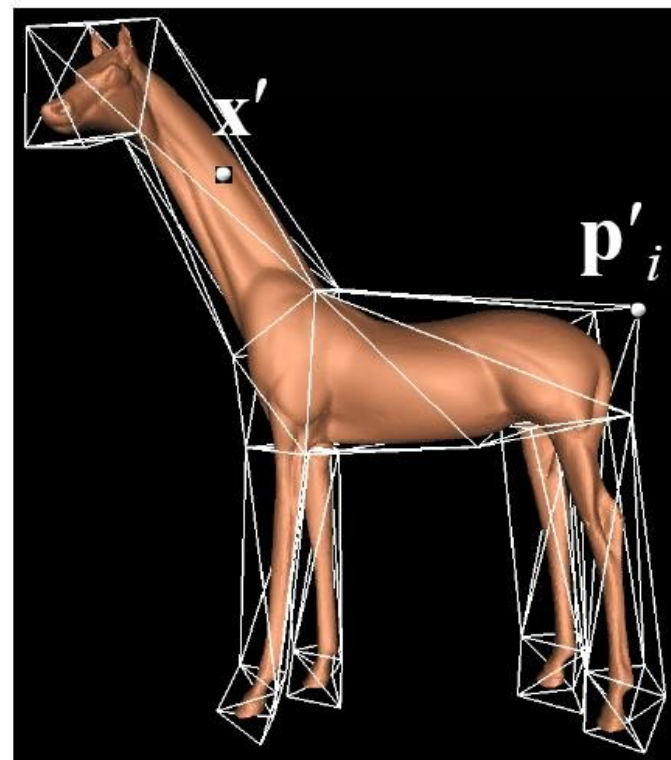
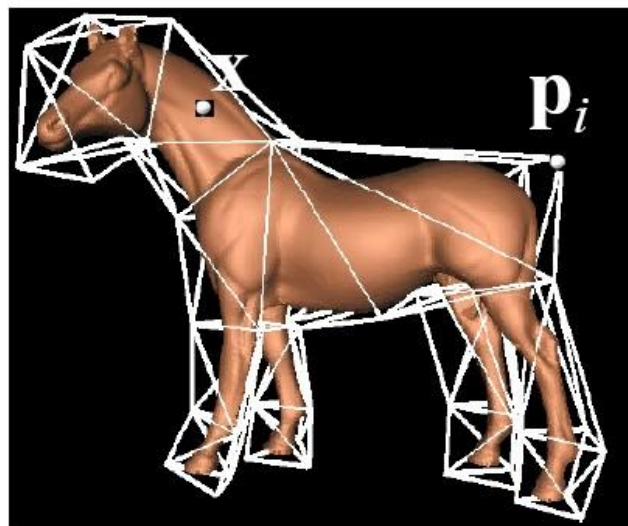
---





# 例子

$$\mathbf{x}' = \sum_{i=1}^k w_i(\mathbf{x}) \mathbf{p}'_i$$



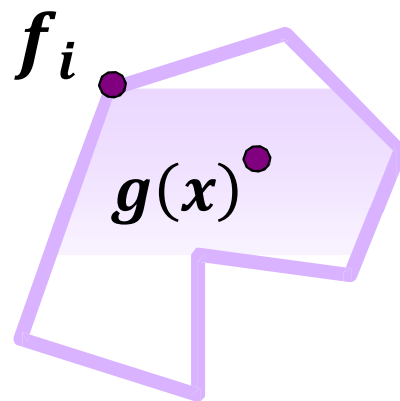
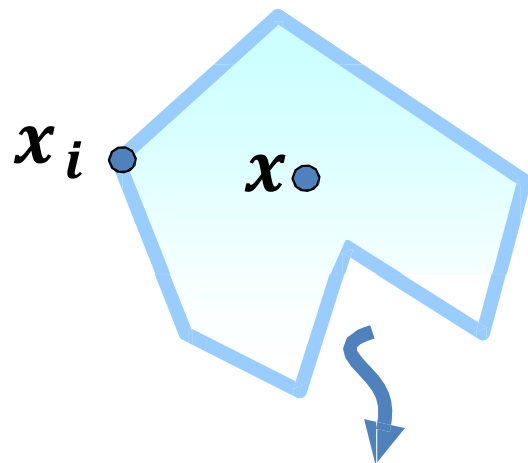
# 重心坐标

求和为一

$$\sum_{i=1}^n w_i(x) = 1$$

单位可重建性

$$\sum_{i=1}^n w_i(x) x_i = x$$



$$g(x) = \sum_{i=1}^n w_i(x) f_i$$

# 重心坐标

---

仿射变换不变性

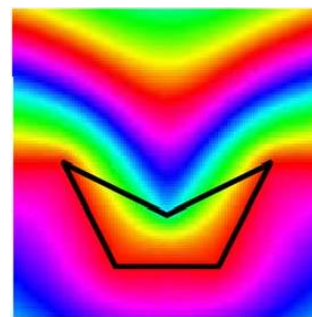
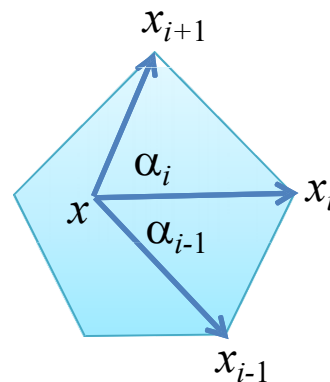
$$\begin{aligned} g_{Ax_i+T}(x) &= \sum_i w_i(x) (Ax_i + T) \\ &= A \underbrace{\sum_i w_i(x) x_i}_x + T \underbrace{\sum_i w_i(x)}_1 \\ &= Ax + T \end{aligned}$$

# 例子

$$k_i(x) = \frac{\tan\left(\frac{\alpha_{i-1}}{2}\right) + \tan\left(\frac{\alpha_i}{2}\right)}{|x_i - x|}$$

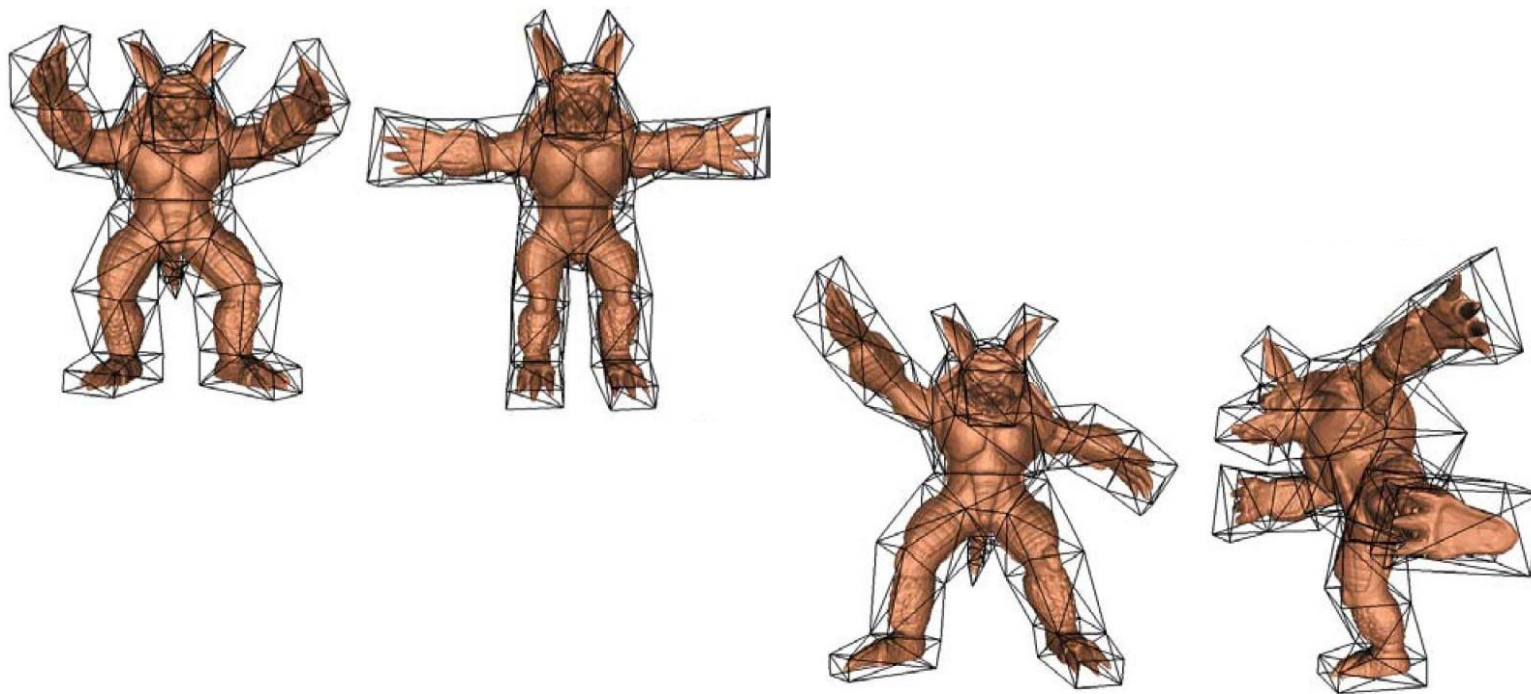
$$w_i(x) = \frac{k_i(x)}{\sum_i k_i(x)}$$

闭式解!



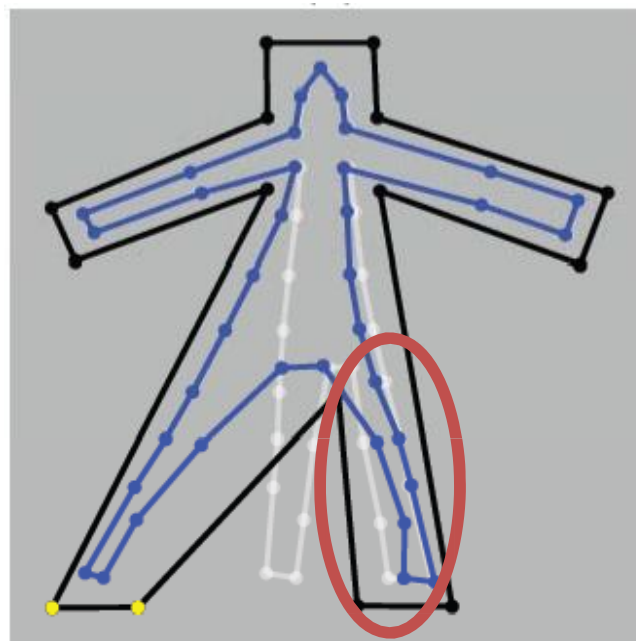
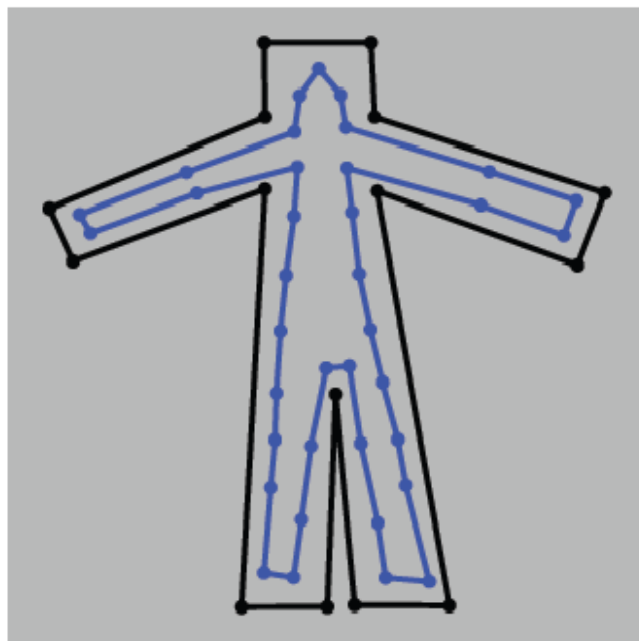
# 例子

---

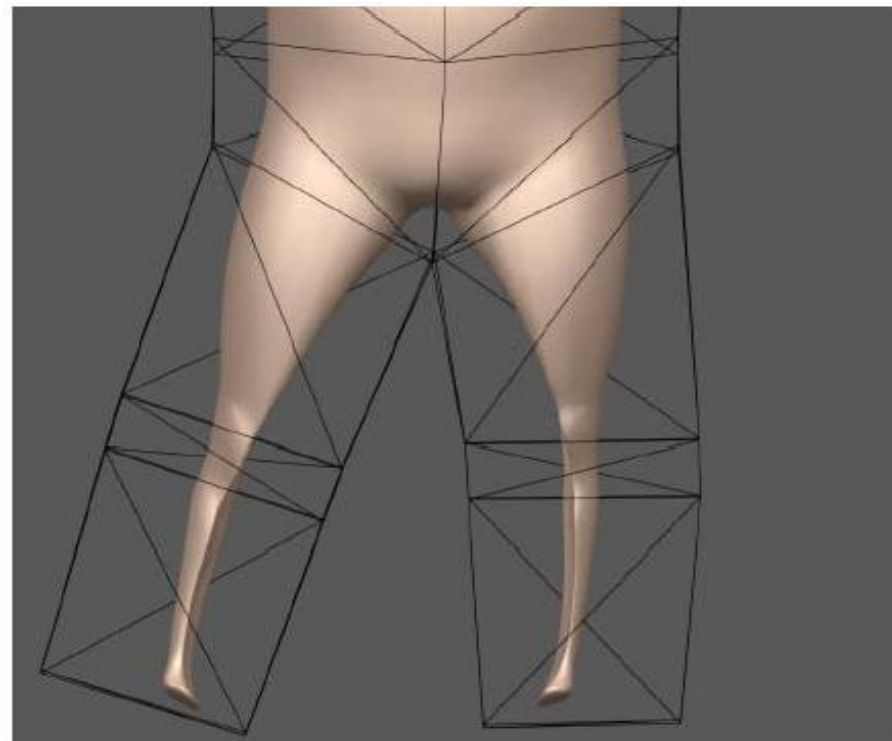
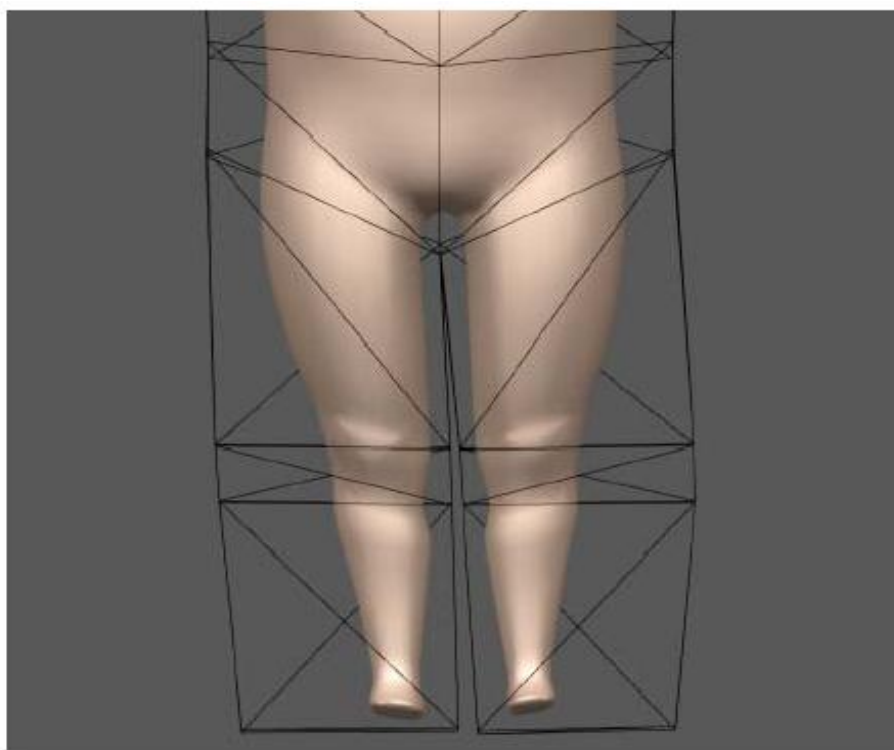


# 重心坐标- 局限性

- 回到pants 问题
- 重心坐标在凹多边形内是负的



# 重心坐标- 局限性



# 重心坐标

---

- 额外要求的属性:

$$w_i(x) \geq 0$$

- Mean value coords 仅在凸多边形内是正的



# Harmonic 坐标

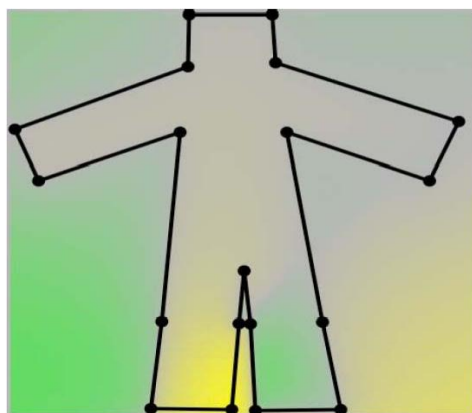
[Joshi et al '07]

求解  $w_i(x)$

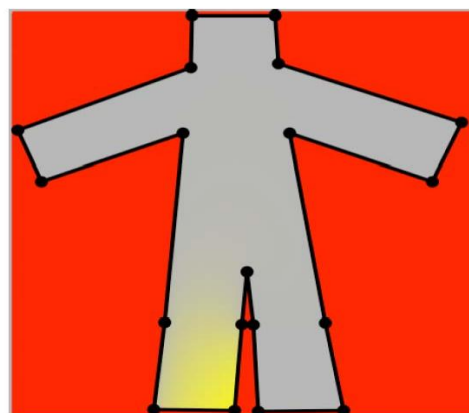
$$\nabla^2 w_i(x) = 0$$

满足:  $w_i$  在边界上是线性的, 并且

$$w_i(x_i) = \delta_{ii}$$

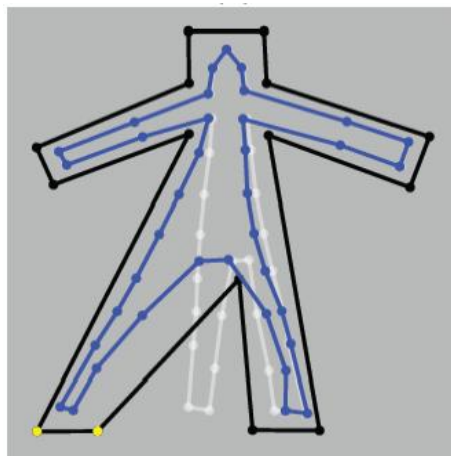
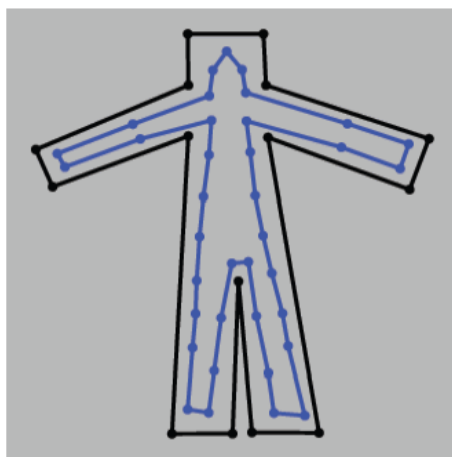


MVC

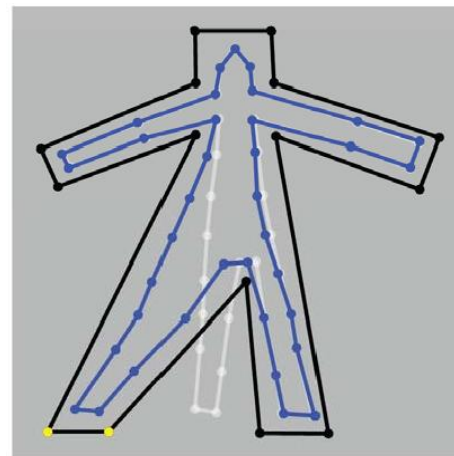


HC

# Harmonic 坐标



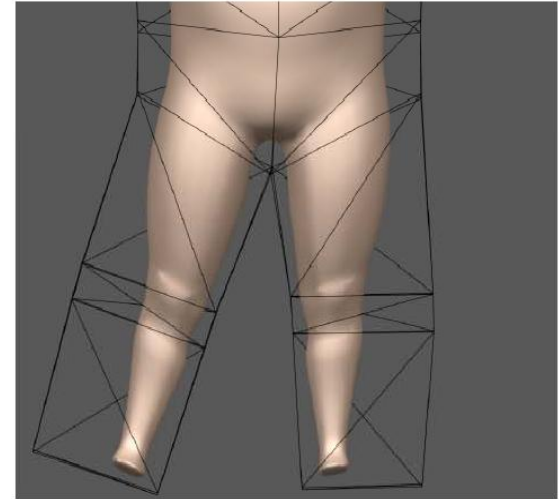
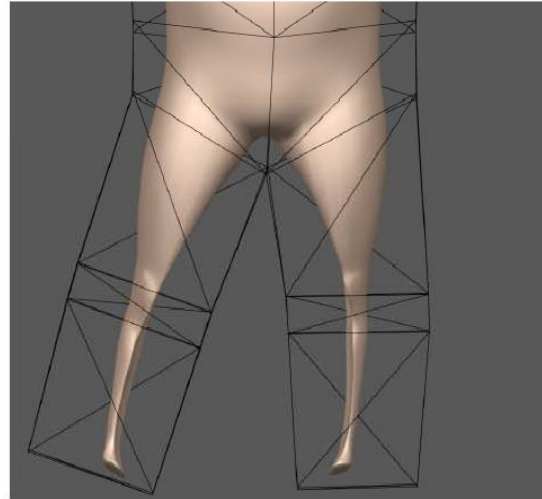
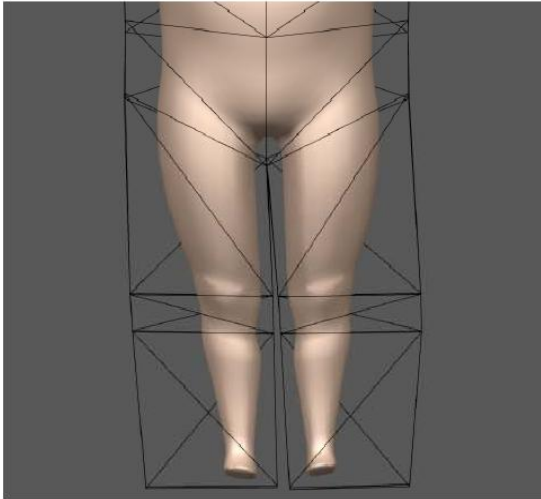
MVC



HC

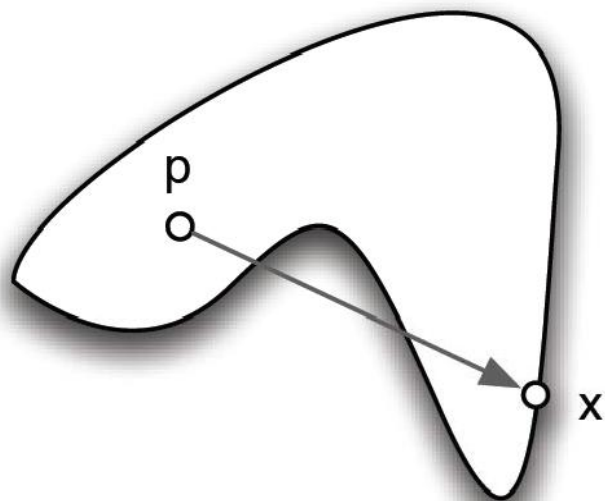
# Harmonic 坐标

---

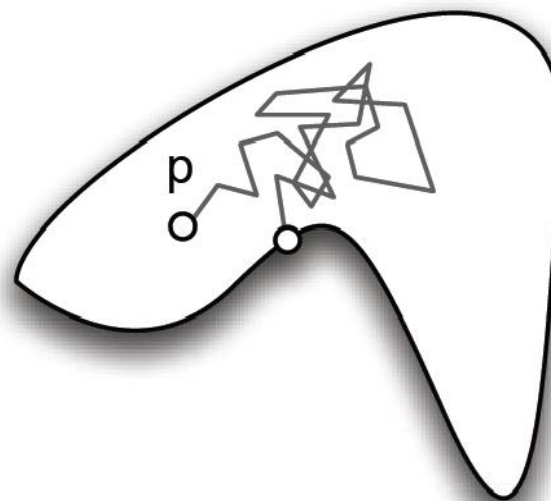


# Harmonic 坐标

## ■ 原理:



MVC 使用欧式距离



HC 电阻距离

# Harmonic 坐标

---

## ■ 属性:

- ◆ 平滑、 旋转平移不变性
- ◆ 处处为正
- ◆ 没有闭式解, 需要求解 PDE

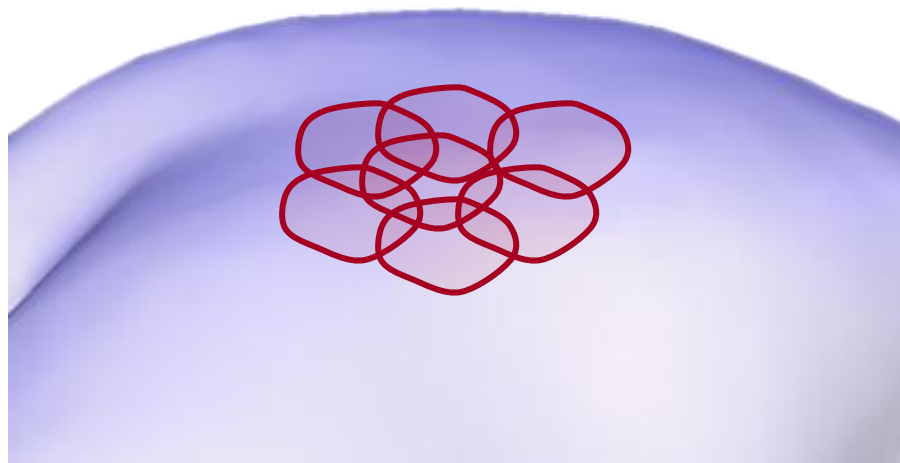
# 参考文献

---

- “On Linear Variational Surface Deformation Methods” [Botsch & Sorkine ‘08]
- Tutorial: “Interactive Shape Modeling and Deformation” [Sorkine & Botsch ‘09]
- “Image deformation using moving least squares” [Schaefer et al ‘06]
- “Mean Value Coordinates for Closed Triangular Meshes” [Ju et al ‘05]
- “Harmonic coordinates for character articulation” [Joshi et al ‘07]
- “Green Coordinates” [Lipman et al ‘08]
- “Complex Barycentric Coordinates with Applications to Planar Shape Deformation” [Weber et al. ‘09]
- Excellent webpage on barycentric coordinates: <http://www.inf.usi.ch/hormann/barycentric/>

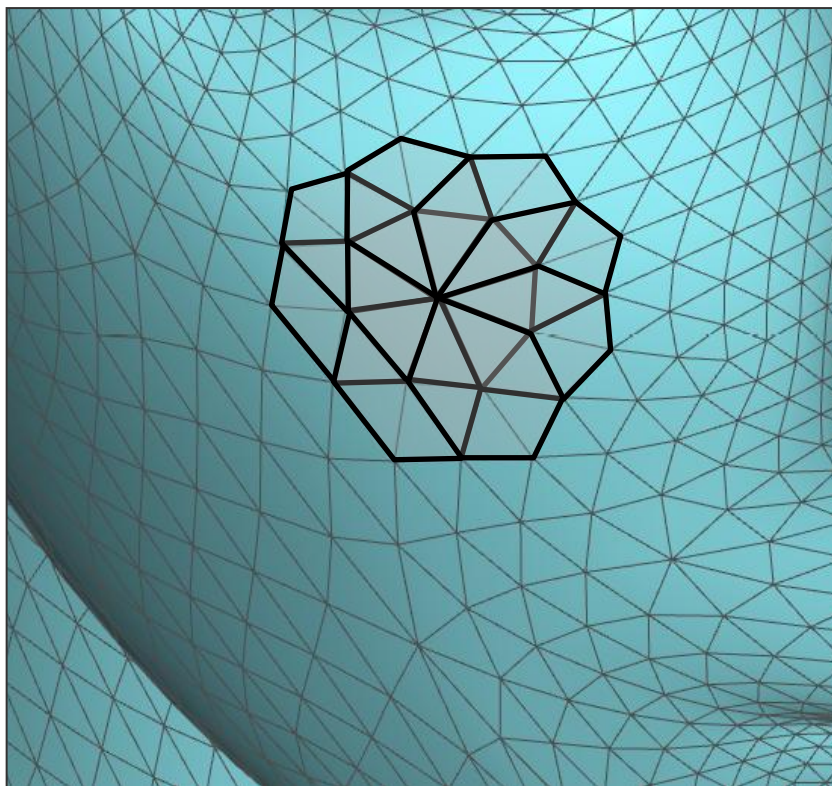
# 直接ARAP建模 (Direct ARAP modeling)

- 保持覆盖在表面的网格单元(cell)形状
  - ◆ 网格单元应给相互覆盖以防止共享边界
  - ◆ 网格单元等大小或者为不同大小提供补偿(Equally-sized cells, or compensate for varying size)



# 直接ARAP建模

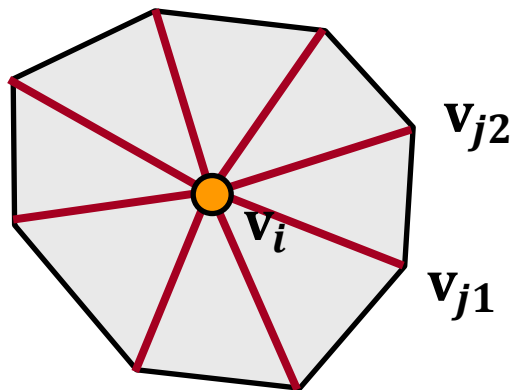
- 来看一个mesh的网格单元





# 网格单元形变能 (Cell deformation energy)

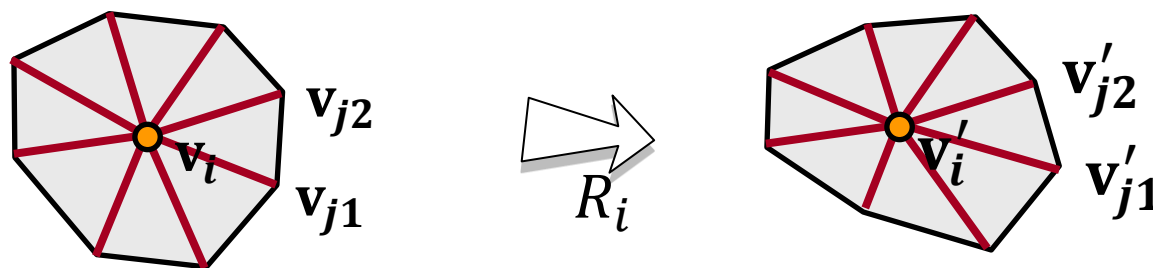
- 让每条边来严格地根据某一旋转矩阵 $R$ 来变形，然后保留单元格的形状。



$$\min \sum_{j \in N(i)} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R(\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$

# 网格单元形变能

- 如果  $\mathbf{v}$ ,  $\mathbf{v}'$  已知, 那么  $R_i$  是唯一定义的。



- 所谓的形状匹配问题

- ◆ 建立协方差矩阵  $S = VV'^T$
- ◆ SVD:  $S = U\Sigma W^T$
- ◆  $R_i = UW^T$  (or use [Horn 87])

# 总形变能

- 全部形变能的公式：

$$\min_{\mathbf{v}'} \sum_{i=1}^n \sum_{j \in N(i)} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R_i(\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$

$$s.t. \mathbf{v}'_j = \mathbf{c}_j, j \in C$$

- 我们分开考虑 $\mathbf{v}'$ 和 $R$ 变量集，来简化优化过程。

# 最小化能量

## ■ 重复迭代

◆ 根据最初猜想的 $\mathbf{v}'_0$  寻找最优的旋转矩阵  $R_i$

■ 对每一个网格单元进行该任务！ 我们已经展示了如何在 $\mathbf{v}$ ,  $\mathbf{v}'$ 已知时定义  $R_i$

$$\min_{\mathbf{v}'} \sum_{i=1}^n \sum_{j \in N(i)} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R_i (\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$

◆ 根据  $R_i$  (固定的), 最小化能量来找到新的  $\mathbf{v}'$

# 最小化能量

## ■ 重复迭代

◆ 根据最初猜想的  $\mathbf{v}'_0$  寻找最优的旋转矩阵  $R_i$

- 对每一个网格单元进行该任务！我们已经展示了如何在  $\mathbf{v}$ ,  $\mathbf{v}'$  已知时定义  $R_i$

$$L\mathbf{v}' = \mathbf{b}$$

Uniform mesh Laplacian

◆ 根据  $R_i$  (固定的), 最小化能量来找到新的  $\mathbf{v}'$

# 优势

---

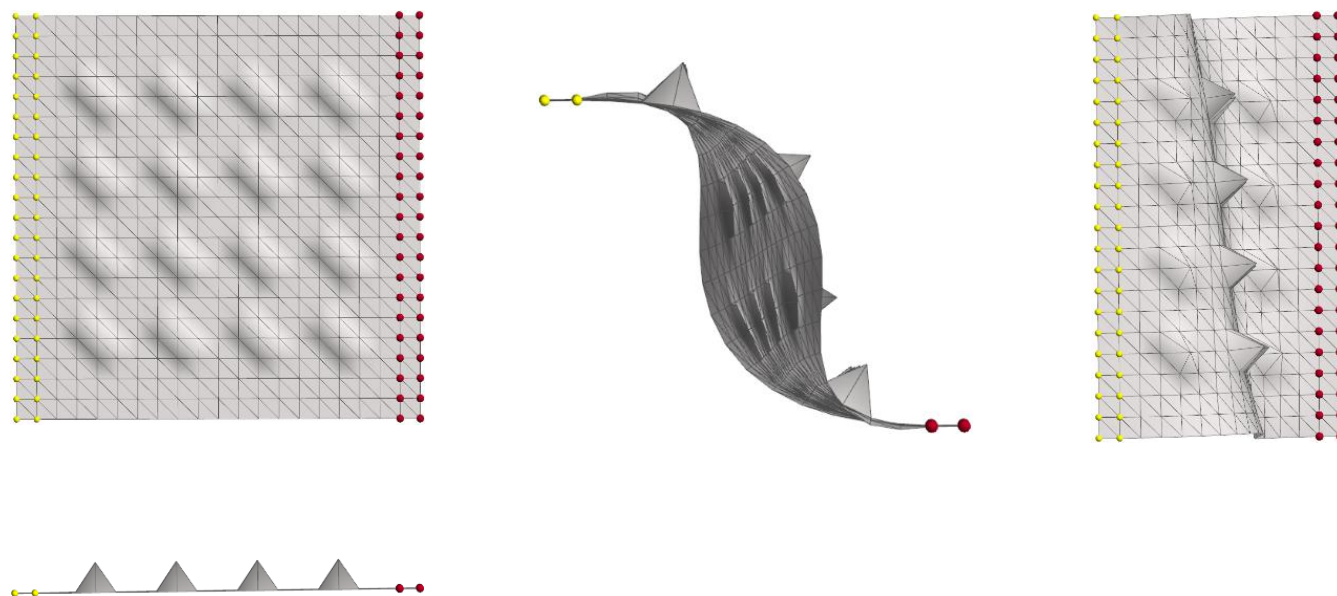
- 每轮迭代都会减小能量（至少不会增大能量）

$$L\mathbf{v}' = \mathbf{b}$$

- $L$  矩阵保持固定
  - ◆ 预处理Cholesky系数
  - ◆ 代回到每次迭代（+SVD分解）

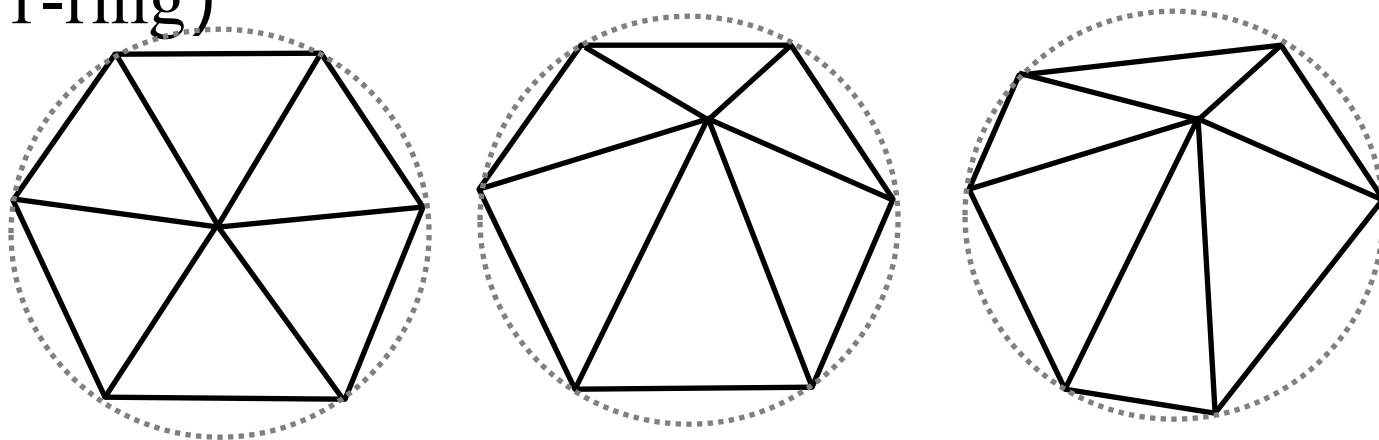
# 第一轮结果

## ■ 非对称结果



# 需要合适的权重

- 问题： 缺少对多种1-环形状的补偿 (lack of compensation for varying shapes of the 1-ring)



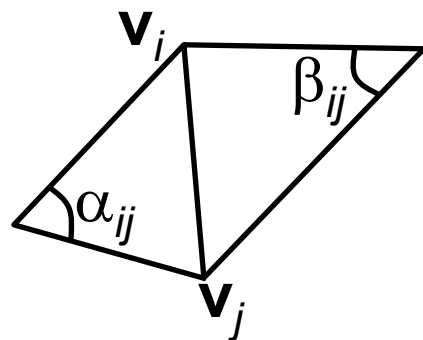
$$E_{cell} = \sum_{j \in N(i)} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R_i(\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$



# 需要合适的权重

- 添加余切权重[Pinkall and Polthier 93]:

$$E_{cell} = \sum_{j \in N(i)} w_{ij} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R_i(\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$



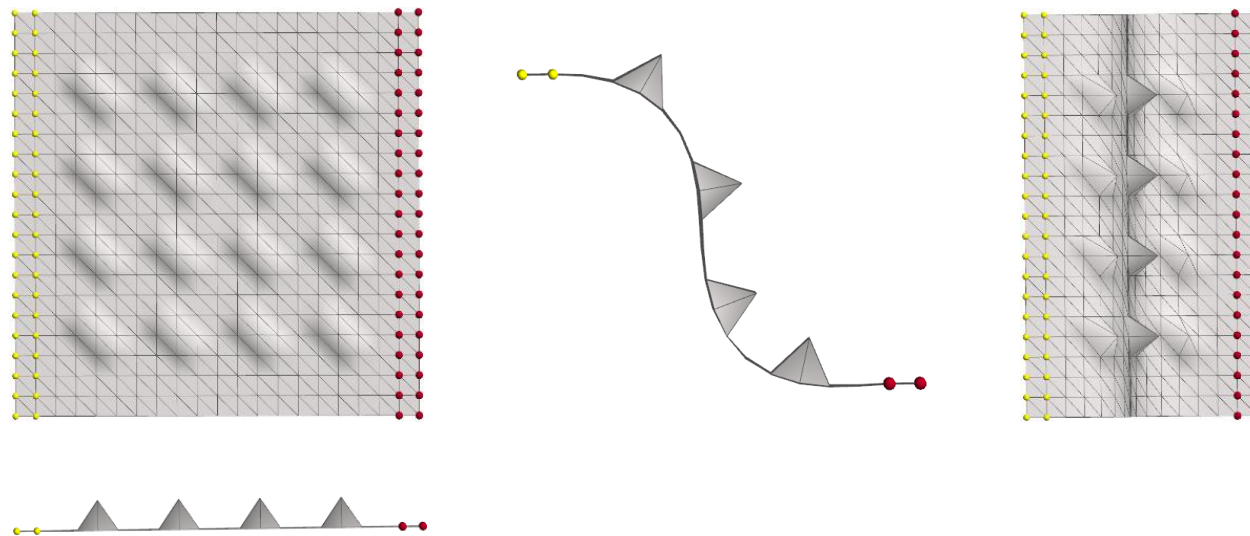
$$w_{ij} = \frac{1}{2} (\cot \alpha_{ij} + \cot \beta_{ij})$$

- 重写含权重的  $R_i$  优化公式 (协方差矩阵权重)

# 带权能量的最小化结果

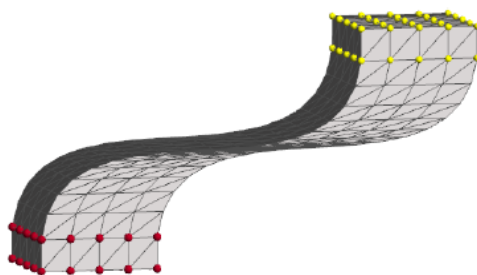
## ■ 对称结果

$$E_{total} = \sum_{i=1}^n \sum_{j \in N(i)} w_{ij} \left\| (\mathbf{v}'_i - \mathbf{v}'_j) - R_i(\mathbf{v}_i - \mathbf{v}_j) \right\|^2$$

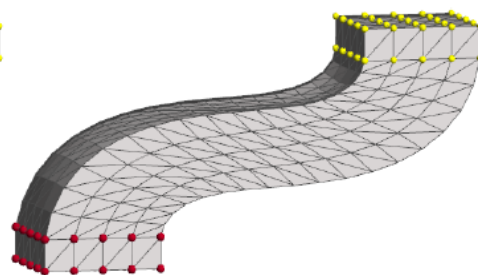


# 初始猜想

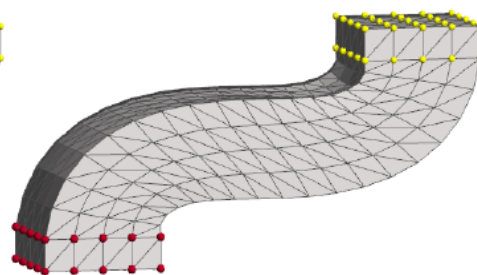
- 可以从朴素拉普拉斯编辑(naïve Laplacian editing)作为最初猜想并迭代



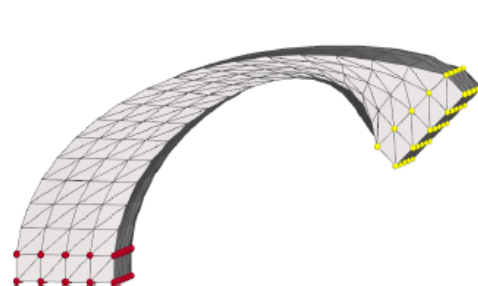
initial guess



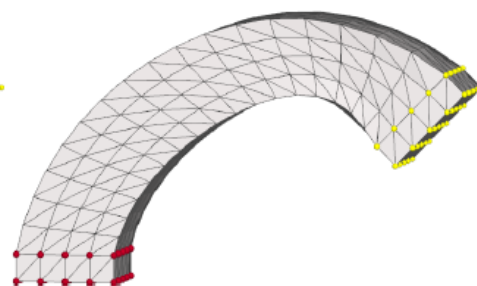
1 iteration



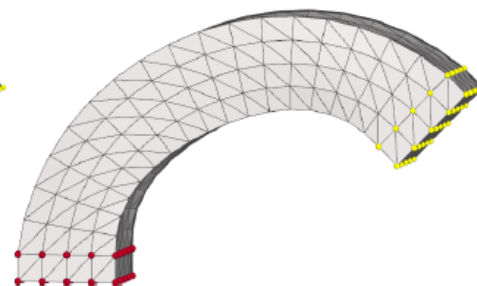
2 iterations



initial guess



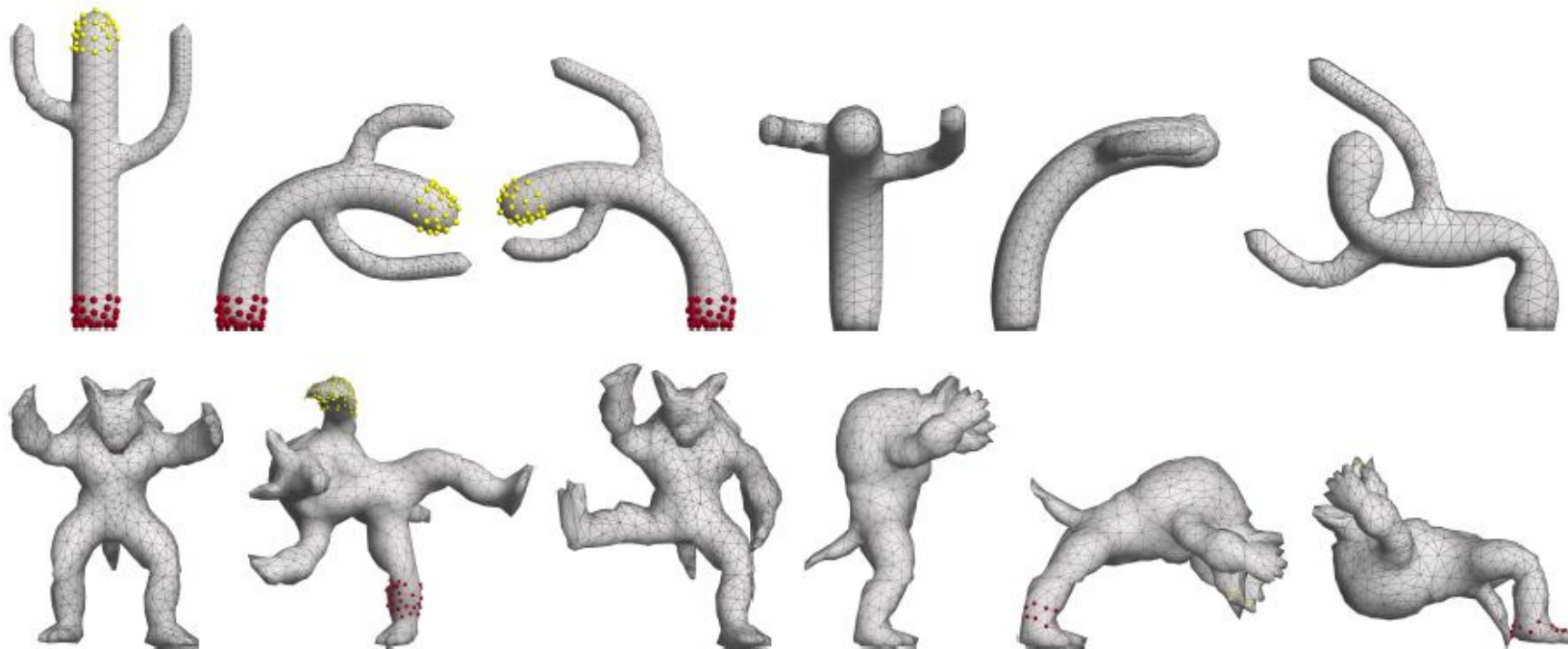
1 iterations



4 iterations

# 初始猜想

- 当我们冲此前的帧开始时，能够更快地收敛 (更适合交互操作)



# Prof. Dr. Olga Sorkine-Hornung

- 苏黎世联邦理工学院计算机科学教授
- Member of the ACM Turing Award Committee.
- EUROGRAPHICS Outstanding Technical Contributions Award, 2017
- 她在图形学很多领域做出了贡献，其中一些方向的问题在几何学上有重大意义，例如为mesh引入了可微线性旋转不变的坐标系、ARAP网格编辑、实时变形、图片编辑和内容感知重新定位中有界双调权重的使用



# 周昆教授

- 浙江大学教授， CAD&CG国家重点实验室主任
- ASIAGRAPHICS Outstanding Technical Contributions Award, 2022
- 他提出了用于3D网格曲面编辑和变形的微分域方法，该方法首先操纵网格曲面的梯度，然后通过求解Poisson方程来重建网格曲面的顶点位置；该方法与特拉维夫大学Sorkine等人提出的Laplace方法一起被公认为微分域几何计算的两个杰出工作
- 他在GPU并行计算方面做出开创性工作，在移动计算环境下的虚拟数字人技术方面做了重要贡献，曾任ACM TOG和IEEE TVCG编委



# 变分方法(variational) vs. 直接方法(direct)

---

- 变分 (数值优化)

$$v' = \arg \min_x E(x)$$

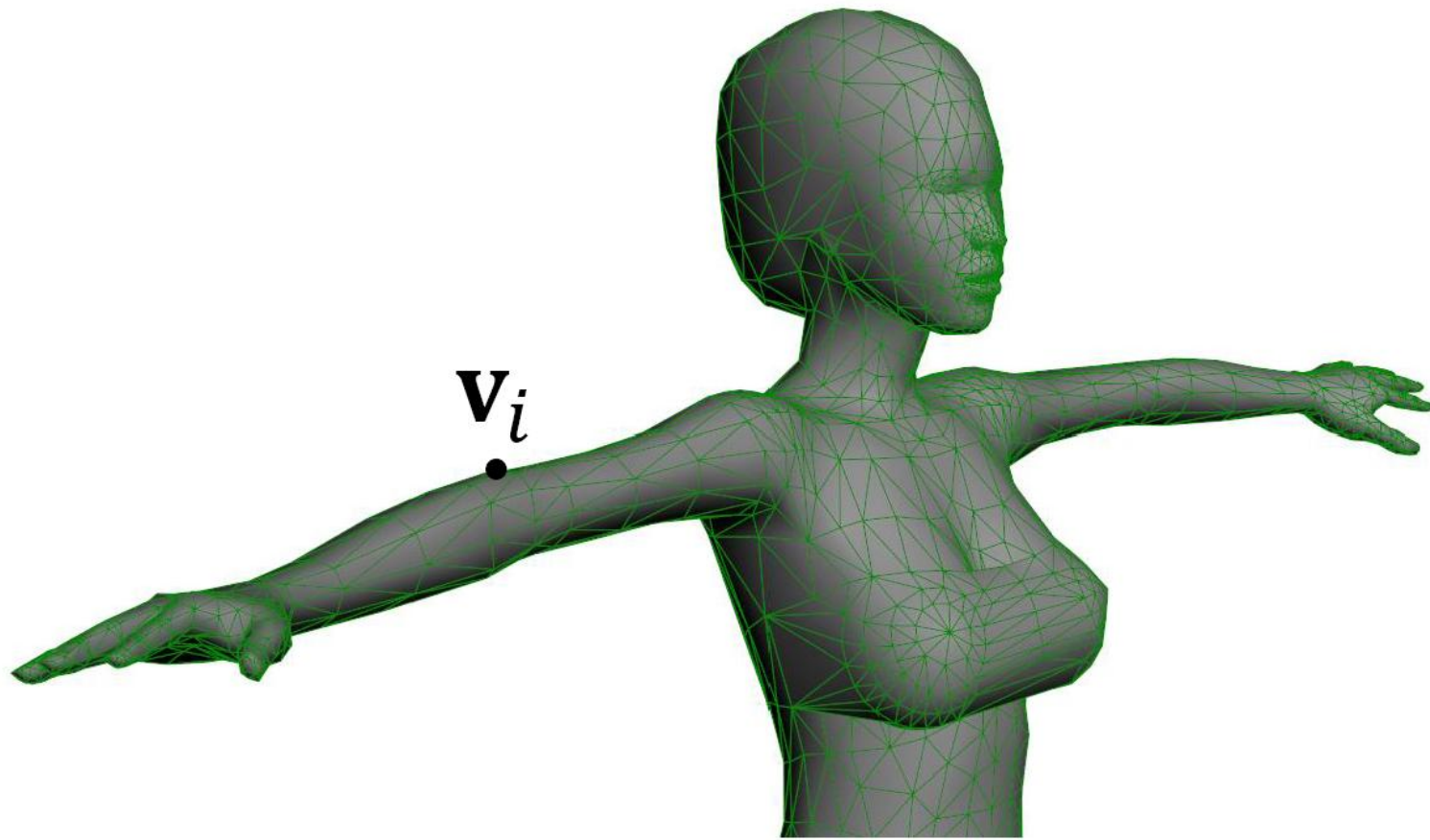
- 直接 (闭式解)

$$v'_i = \sum_{j=1}^m w_{i,j} T_j v_i$$



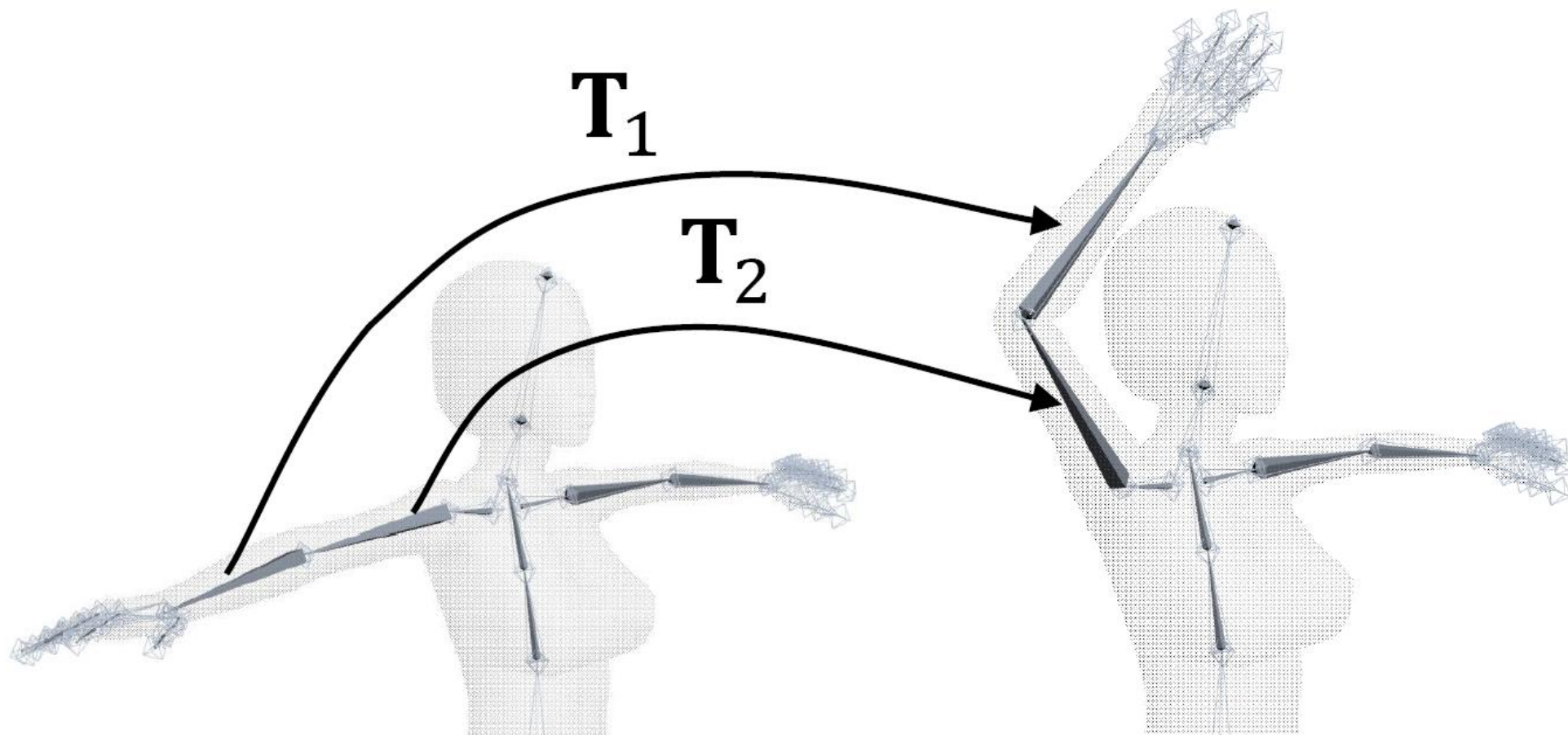
# 其他的姿势

---

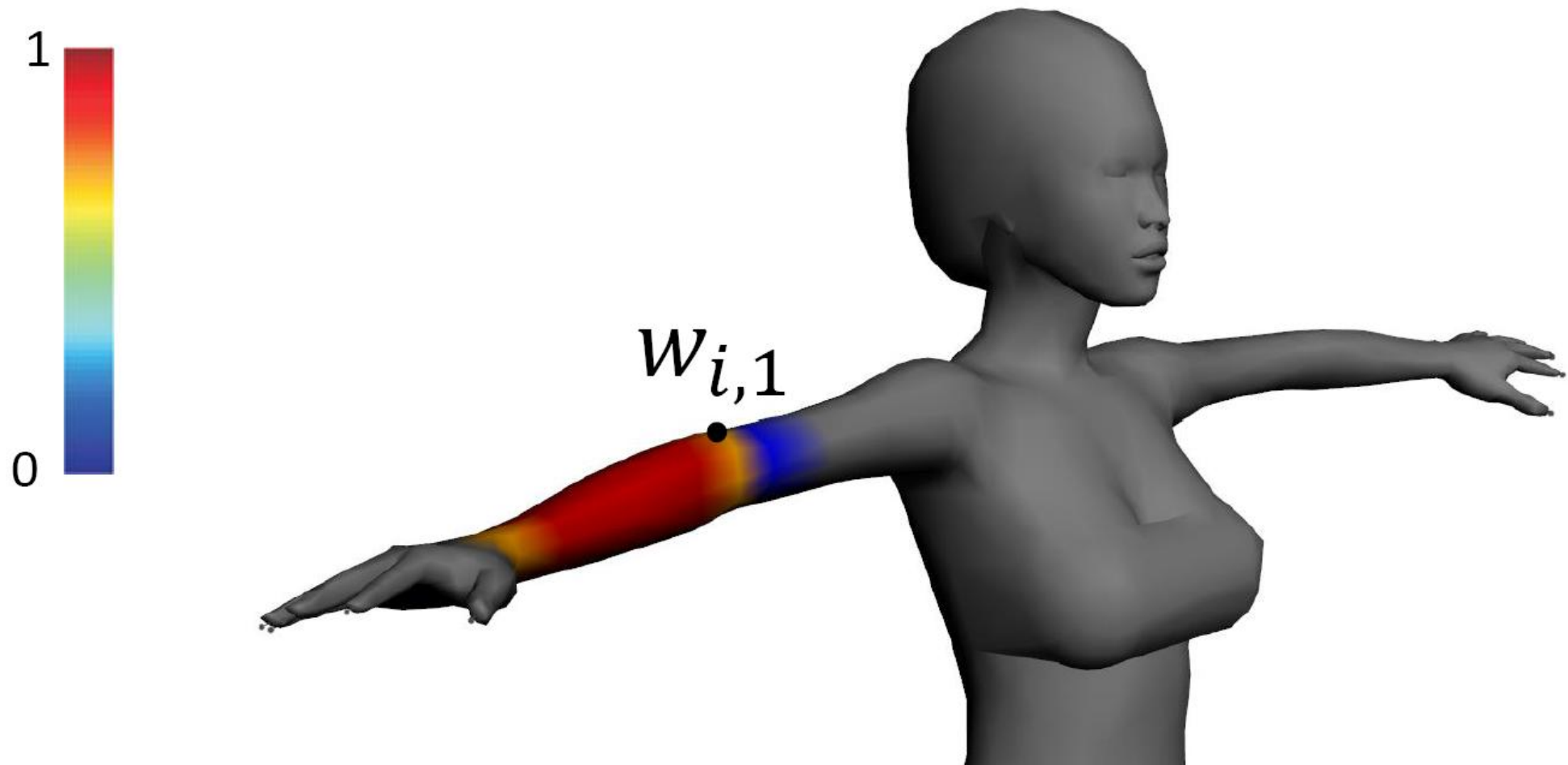




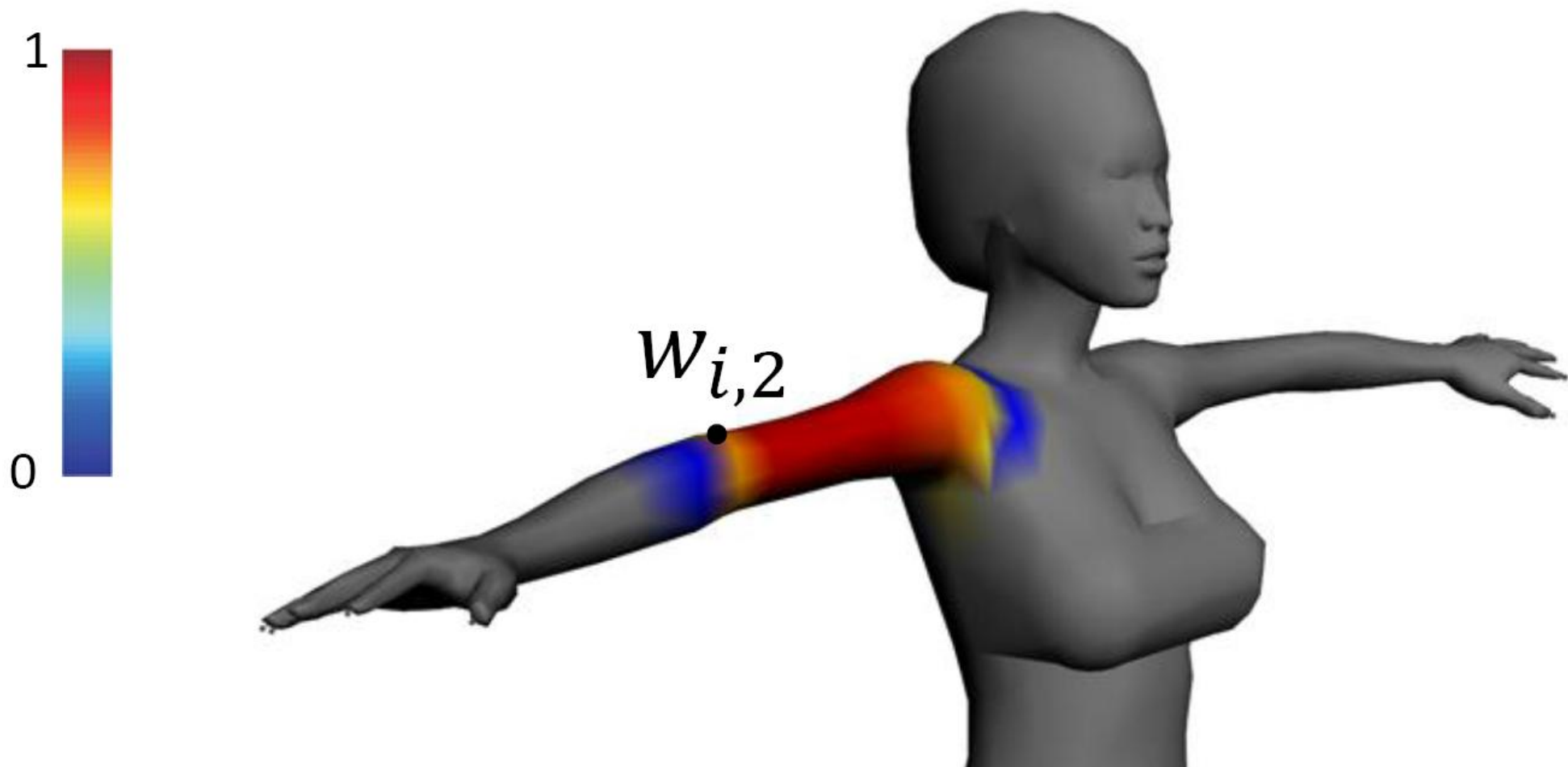
# 蒙皮变换(Skinning transformations)



# 蒙皮权重(Skinning weights)



# 蒙皮权重(Skinning weights)



# 骨骼蒙皮动画算法(Linear Blend Skinning)

---

$$\mathbf{v}'_i = \sum_{j=1}^m w_{i,j} \mathbf{T}_j \mathbf{v}_i = \left( \sum_{j=1}^m w_{i,j} \mathbf{T}_j \right) \mathbf{v}_i$$

# 权重求解方法

- 对于有界双调和权重 $w$ 的计算方法，其数学表达式如下，最小化问题可以转化为求解对应的Euler-Lagrange方程，即双调和方程 $\Delta^2 w_j = 0$ 。

$$\arg \min_{w_j, j=1, \dots, m} \sum_{j=1}^m \frac{1}{2} \int_{\Omega} \|\Delta w_j\|^2 dV$$

*subject to:*

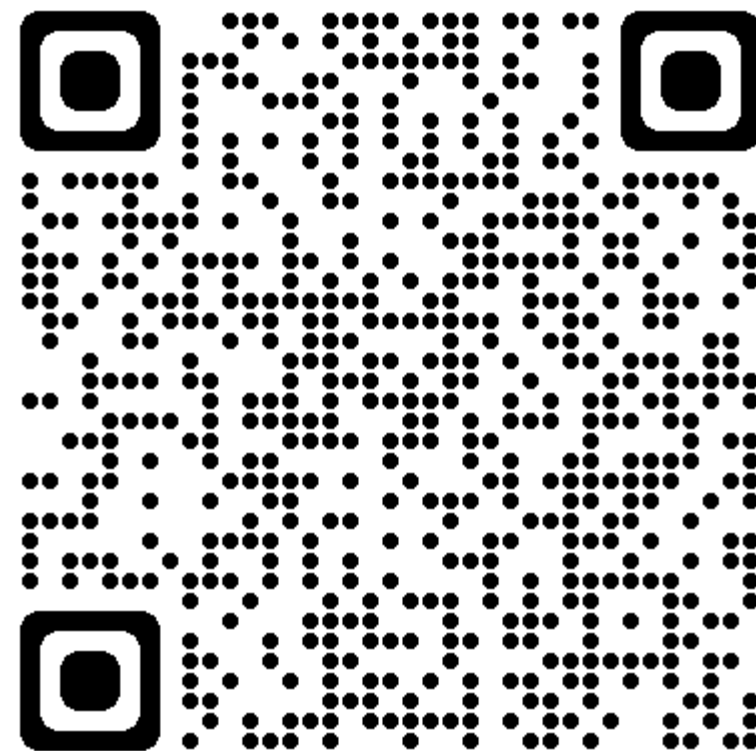
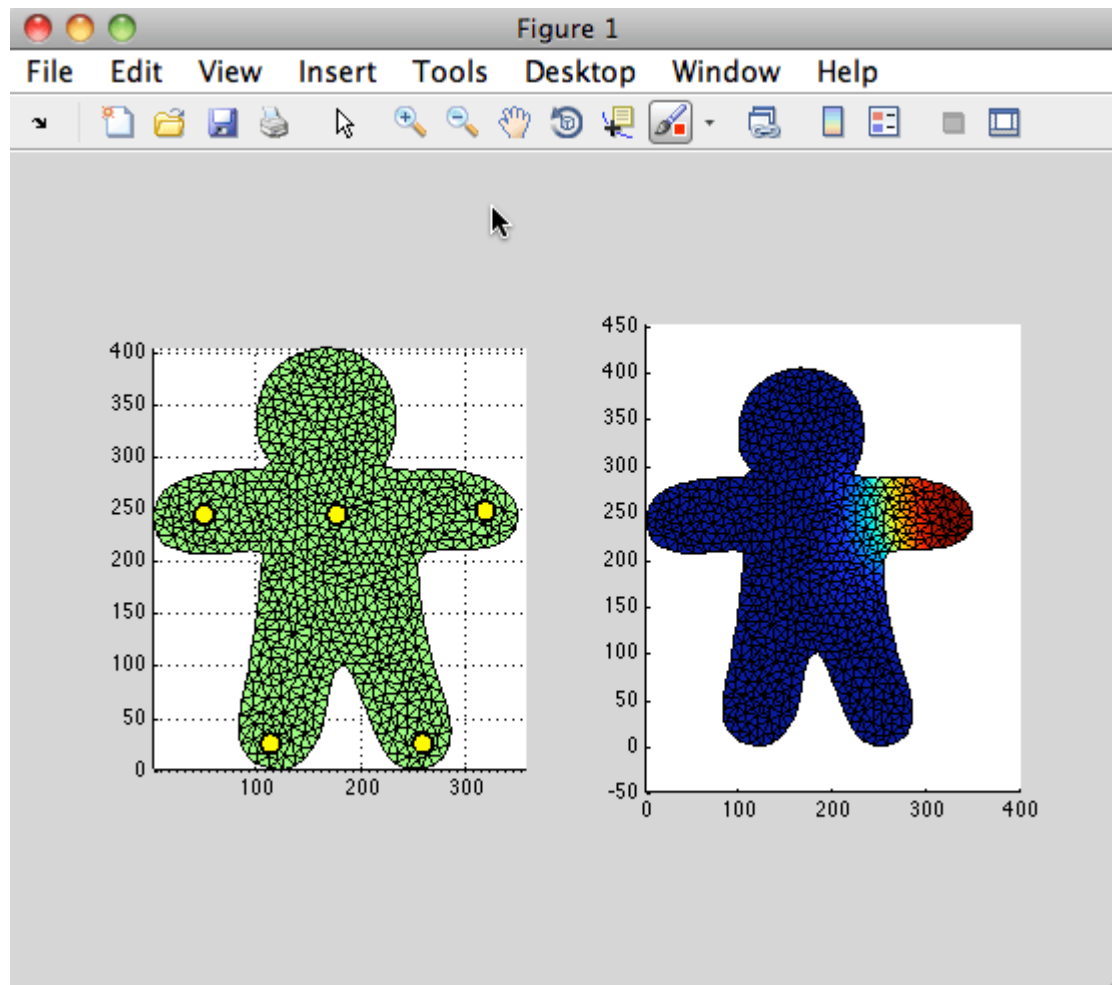
$$w_j|_{H_k} = \delta_{jk}$$

$$w_j|_F \text{ is linear}$$

$$\sum_{j=1}^m w_j(\mathbf{p}) = 1$$

$$0 \leq w_j(\mathbf{p}) \leq 1, j = 1, \dots, m$$

# LBS示例



# Linear Blend Skinning在工业界广泛应用

---



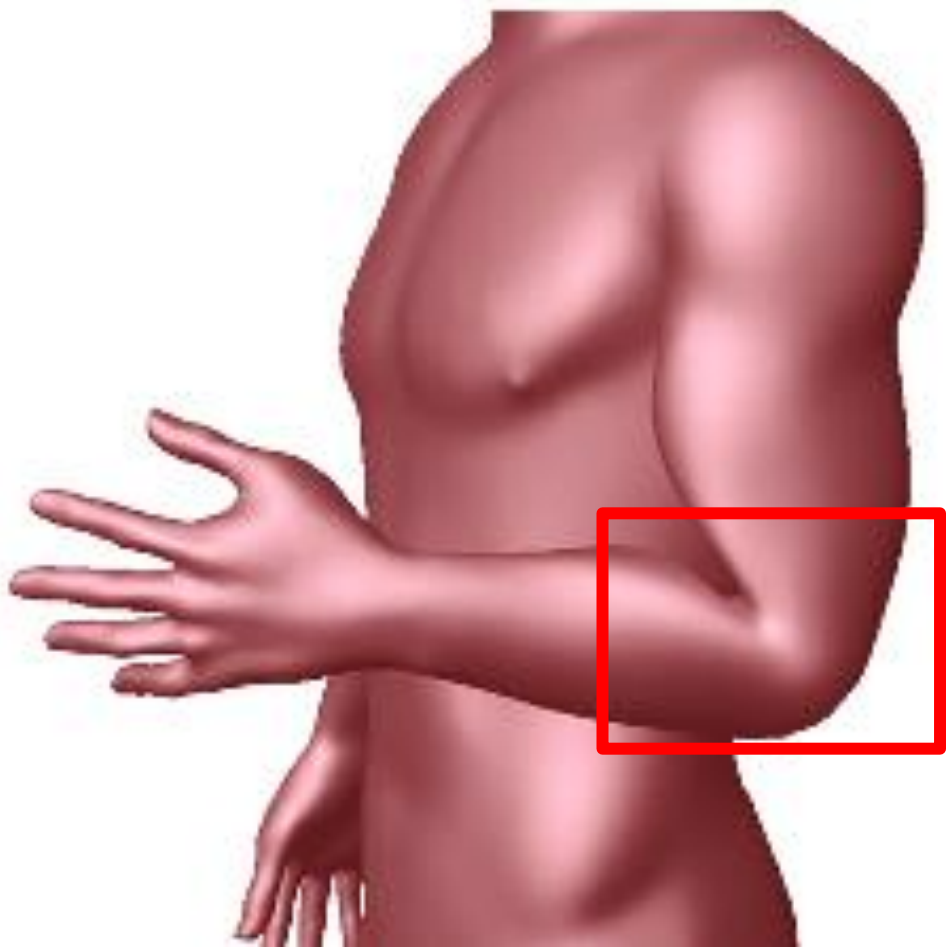
Halo 3



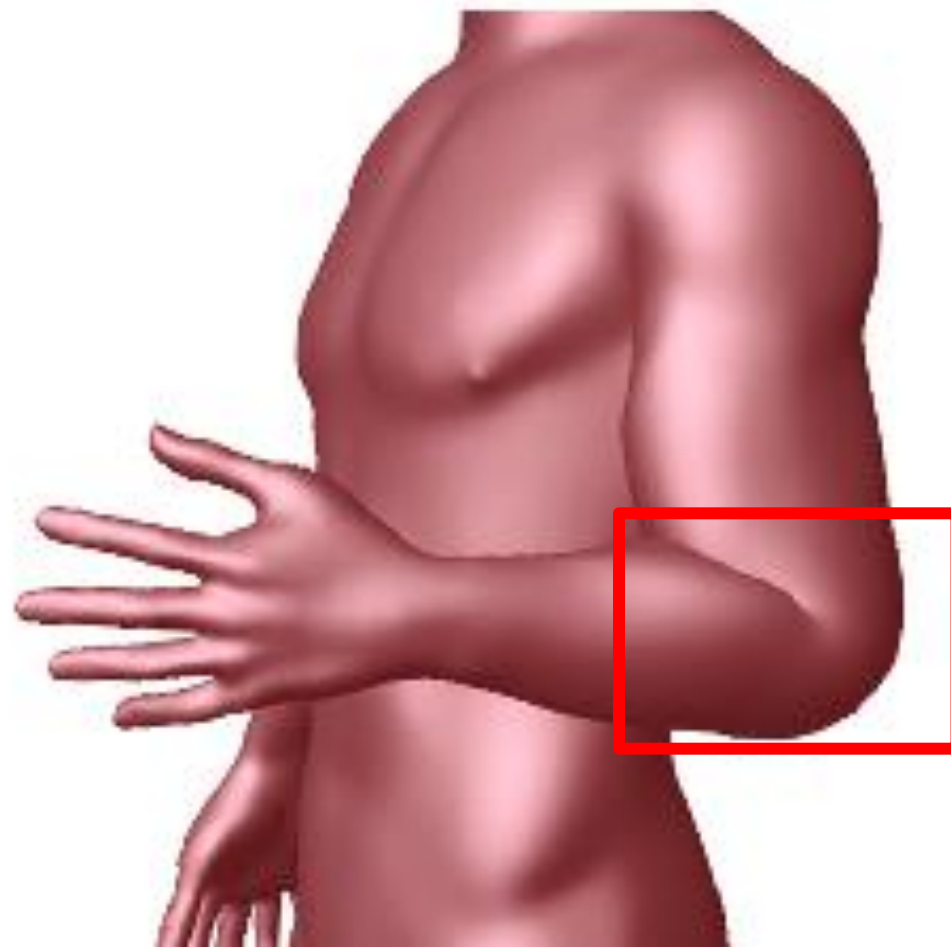
Bolt



# Linear Blend Skinning ‘candy wrapper’瑕疵



Linear blend skinning

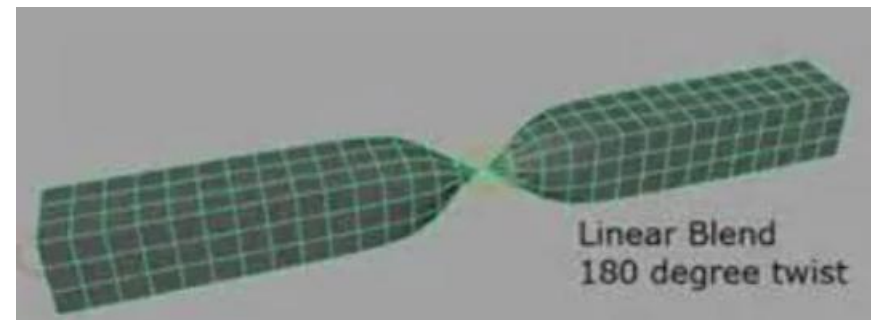
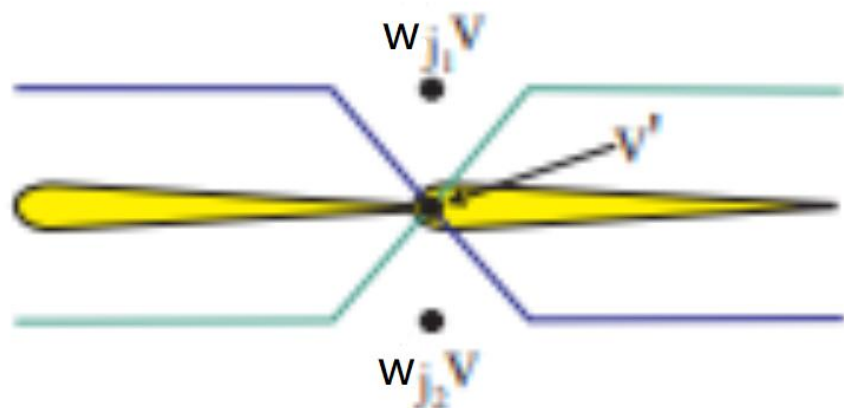


Dual quaternion skinning

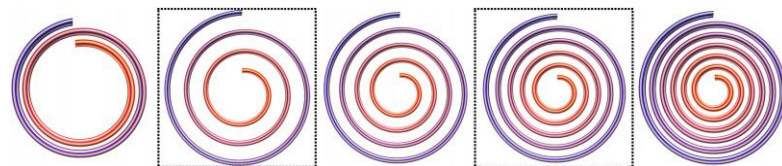


# Linear Blend Skinning ‘candy wrapper’瑕疵

- 因为线性混合蒙皮是线性权重混合的，旋转也是线性混合的。线性混合也就是起始点 $v$ 与多点（对顶点 $v$ 有影响的骨骼 $T_j$ ，按权重为1进行变换后的位置）连线的向量乘以所给权重之和，再加上顶点 $v$ 的初始位置，即为旋转后顶点 $v$ 的位置。
- 当两个（多个）骨骼（控制单元）旋转相差的角度越接近180度（ $\pi$ ）时，“candy-wrapper”就越明显，实际效果图如下所示



# 3D数据（网格）的RIMD 表示



旋转不变网格区别

平移  
不变性



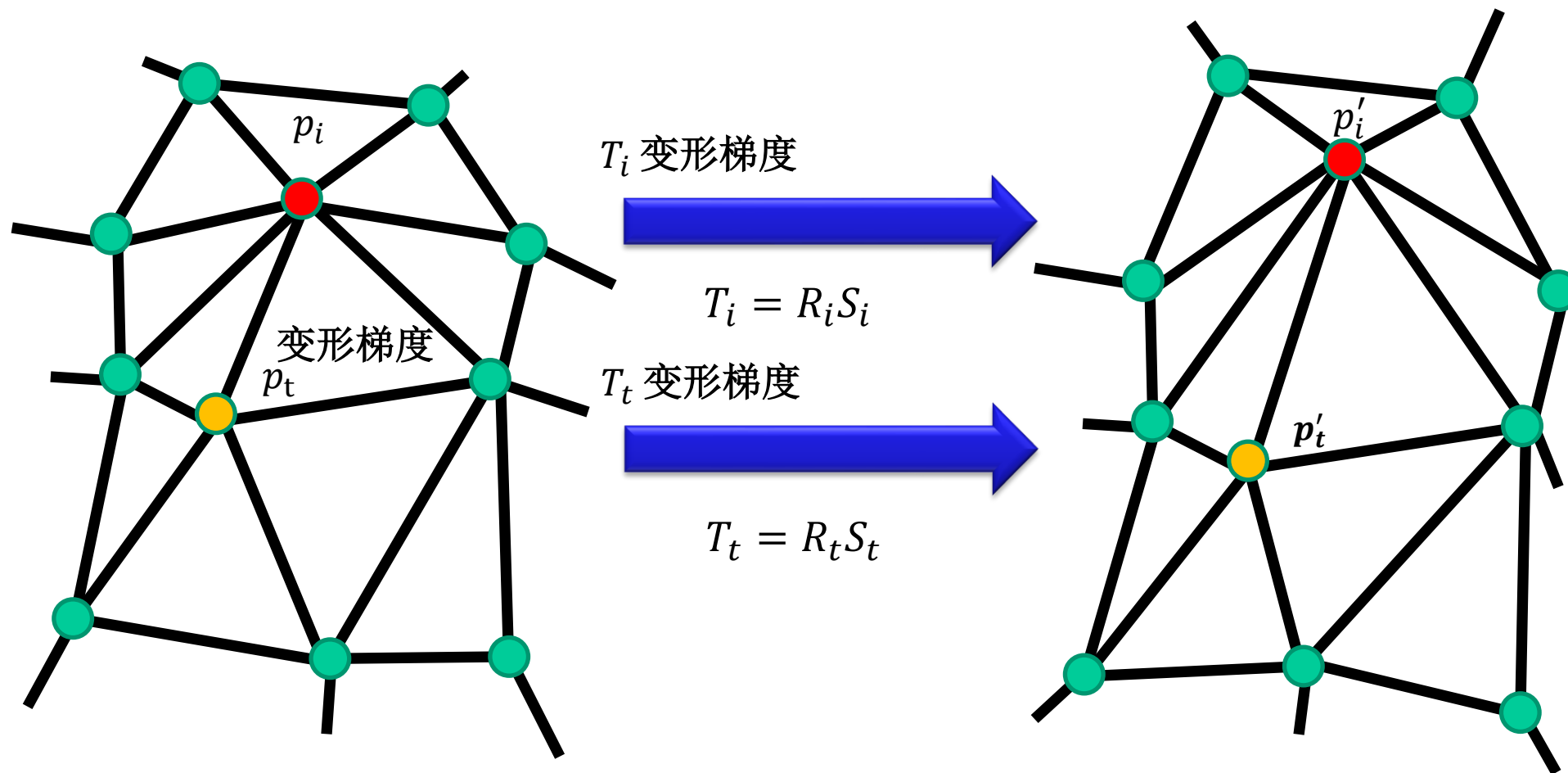
旋转  
不变性



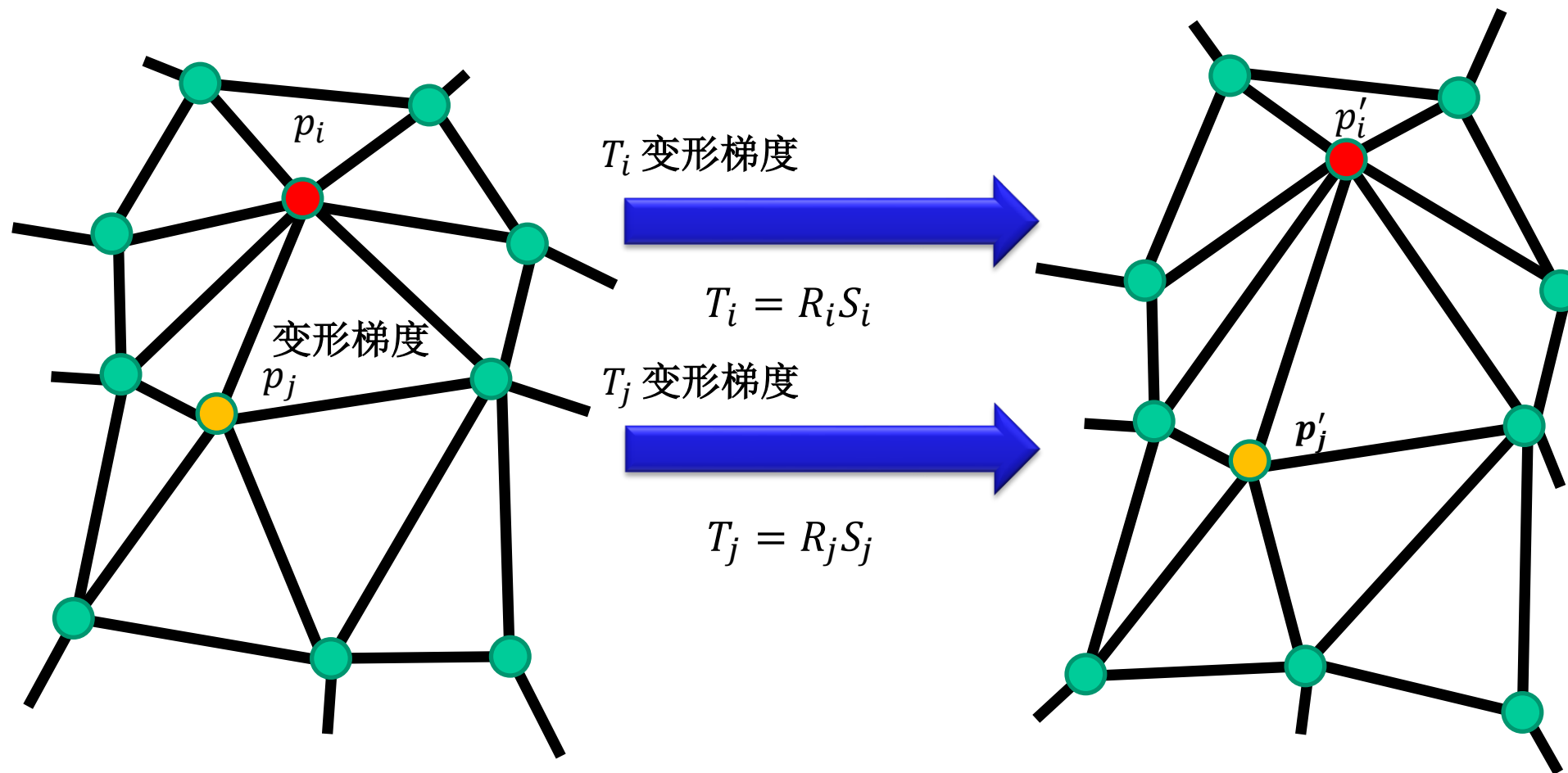
大规模



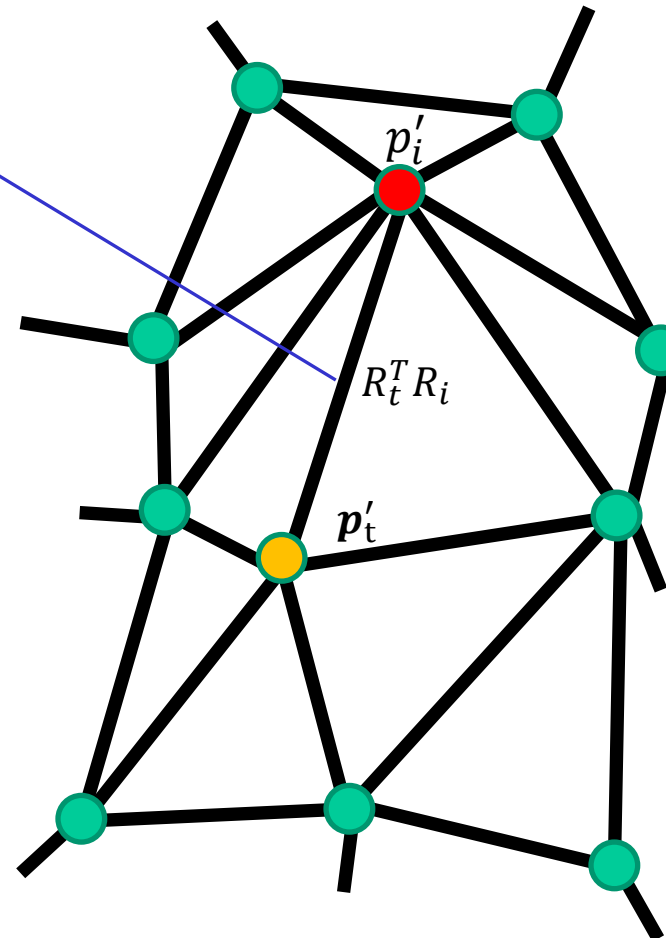
# 表示公式(Representation Formulation)



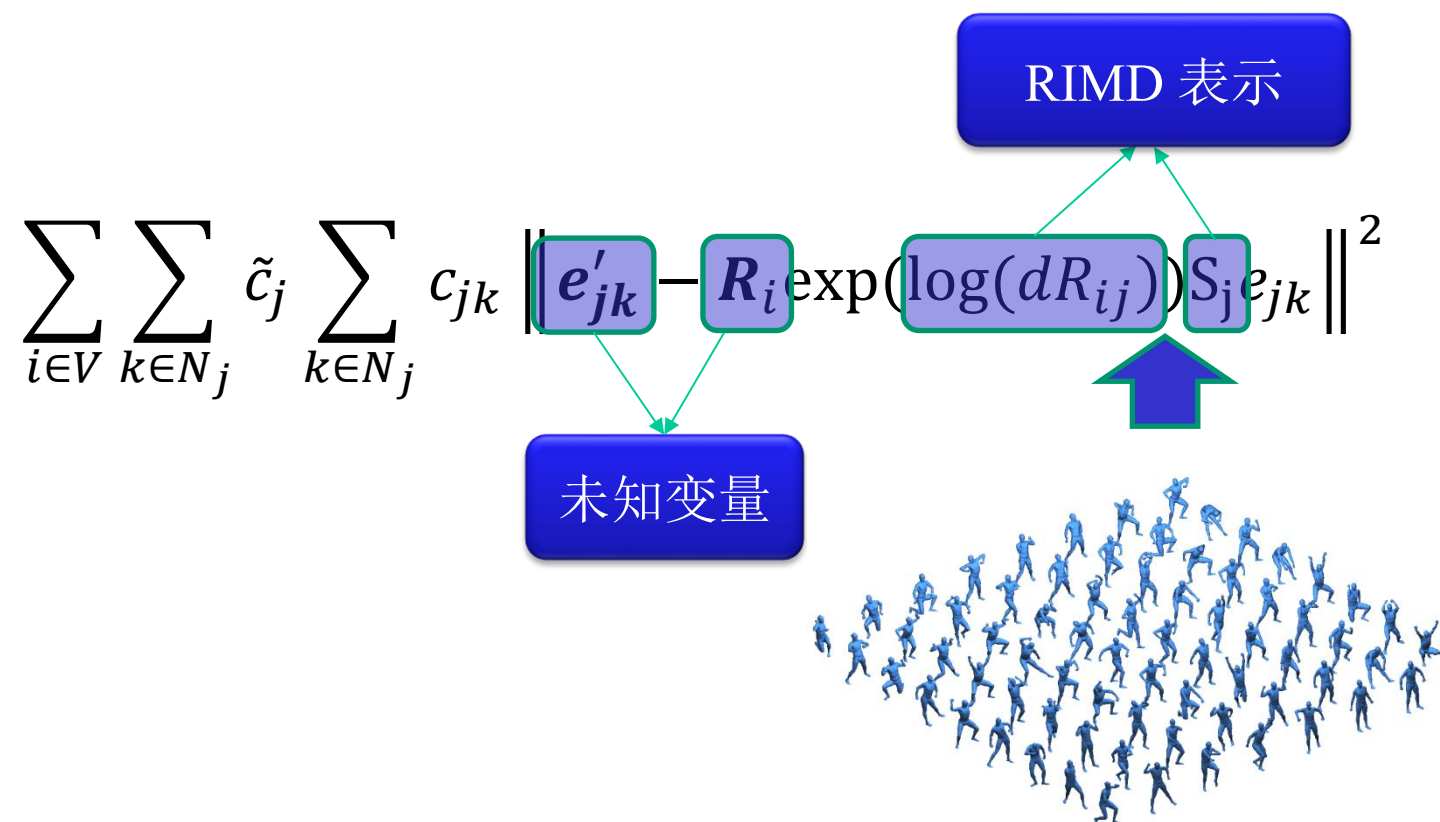
# 表示公式(Representation Formulation)



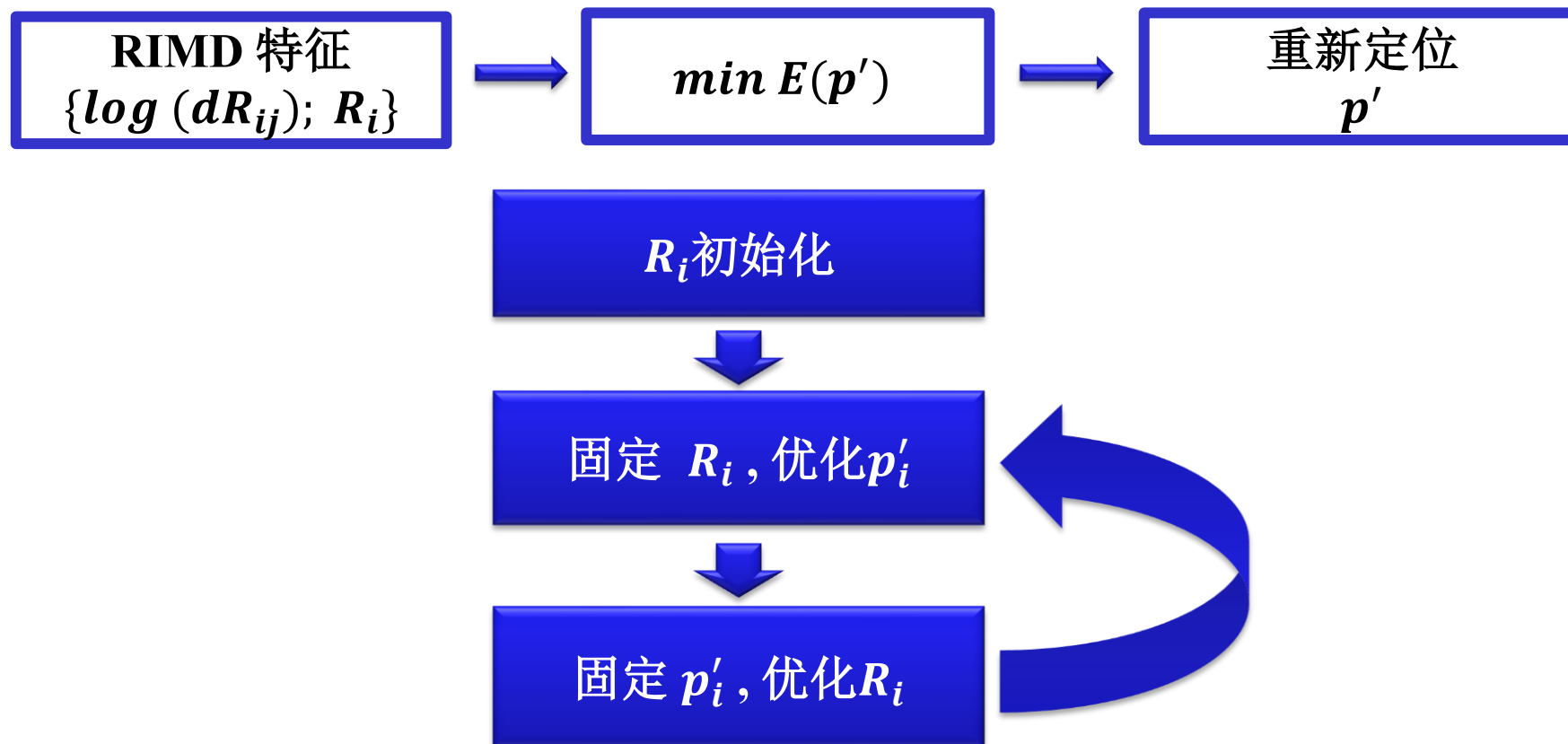
# 表示公式(Representation Formulation)

$$\begin{aligned}
 R_i &= R_t(R_t^T R_i) \quad \forall t \in N_i \\
 E(p') &= \sum_{i \in V} E(T_i) \\
 &= \sum_{i \in V} \sum_{t \in N_i} \tilde{c}_i \sum_{j \in N_i} c_{ij} \|e'_{ij} - T_i e_{ij}\|^2 \\
 &= \sum_{i \in V} \sum_{t \in N_i} \tilde{c}_i \sum_{j \in N_i} c_{ij} \|e'_{ij} - R_i S_i e_{ij}\|^2 \\
 &= \sum_{i \in V} \sum_{t \in N_i} \tilde{c}_i \sum_{j \in N_i} c_{ij} \left\| \underbrace{e'_{ij}}_{\text{UNKNOWN}} - \underbrace{R_t}_{\text{KNOWN}} \underbrace{(R_t^T R_i)}_{\text{KNOWN}} \underbrace{S_i}_{\text{KNOWN}} e_{ij} \right\|^2
 \end{aligned}$$


# 表示公式(Representation Formulation)



# 从 RIMD 表示中重建表面



# 固定 $R_i$ , 优化 $p'_i$

- 最小二乘问题，可以通过线性方程解决：

$$\sum_{k \in N_j} c_{jk} \mathbf{e}'_{jk} = \frac{1}{2} \sum_{k \in N_j} c_{jk} \left( \tilde{c}_k \sum_{s \in N_k} \mathbf{R}_s d \mathbf{R}_{sk} S_k + \sum_{i \in N_j} \mathbf{R}_i d \mathbf{R}_{ij} S_j \right) \mathbf{e}_{jk}$$

$$\sum_{i \in N_j} \mathbf{R}_i d \mathbf{R}_{ij} S_j$$



# 固定 $R_i$ , 优化 $p'_i$

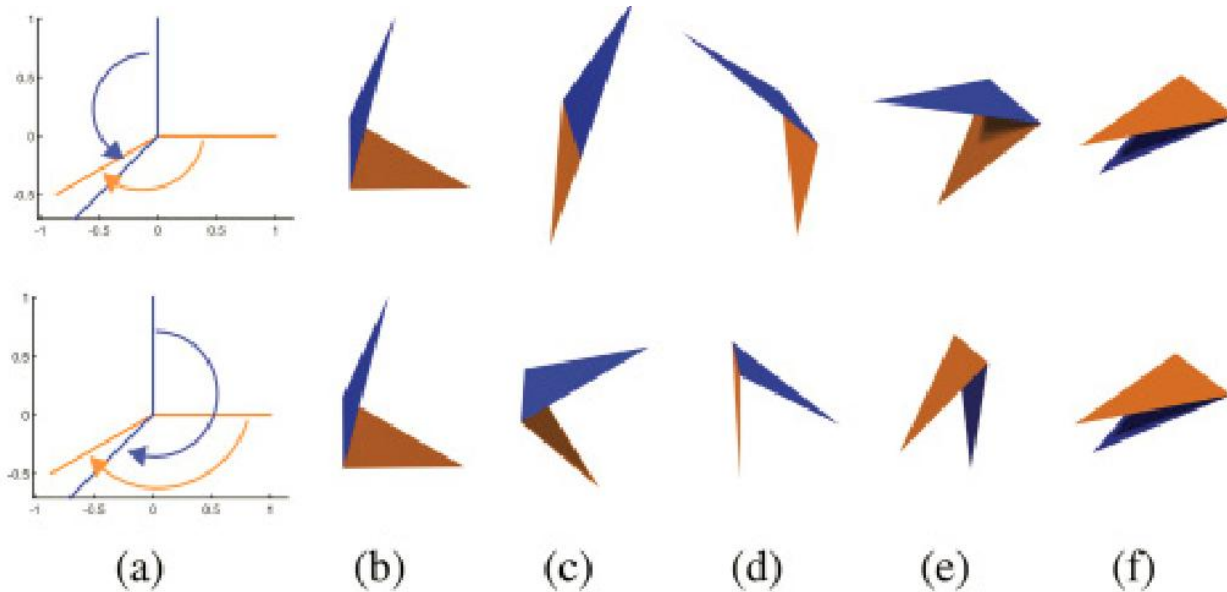
$$\begin{aligned} & \arg \max_{R_i} \sum_{j \in N_i} \tilde{c}_j \sum_{k \in N_j} c_{jk} e_{jk}'^T R_i d R_{ij} S_j e_{jk} \\ & = \text{Tr}(\mathbf{R}_i \sum_{j \in N_i} \tilde{c}_j d R_{ij} S_j \left( \sum_{k \in N_j} c_{jk} e_{jk} e_{jk}'^T \right)) \end{aligned}$$

SVD:  $V_i U_i^T$

SVD:  $U_i \Sigma_i V_i^T$

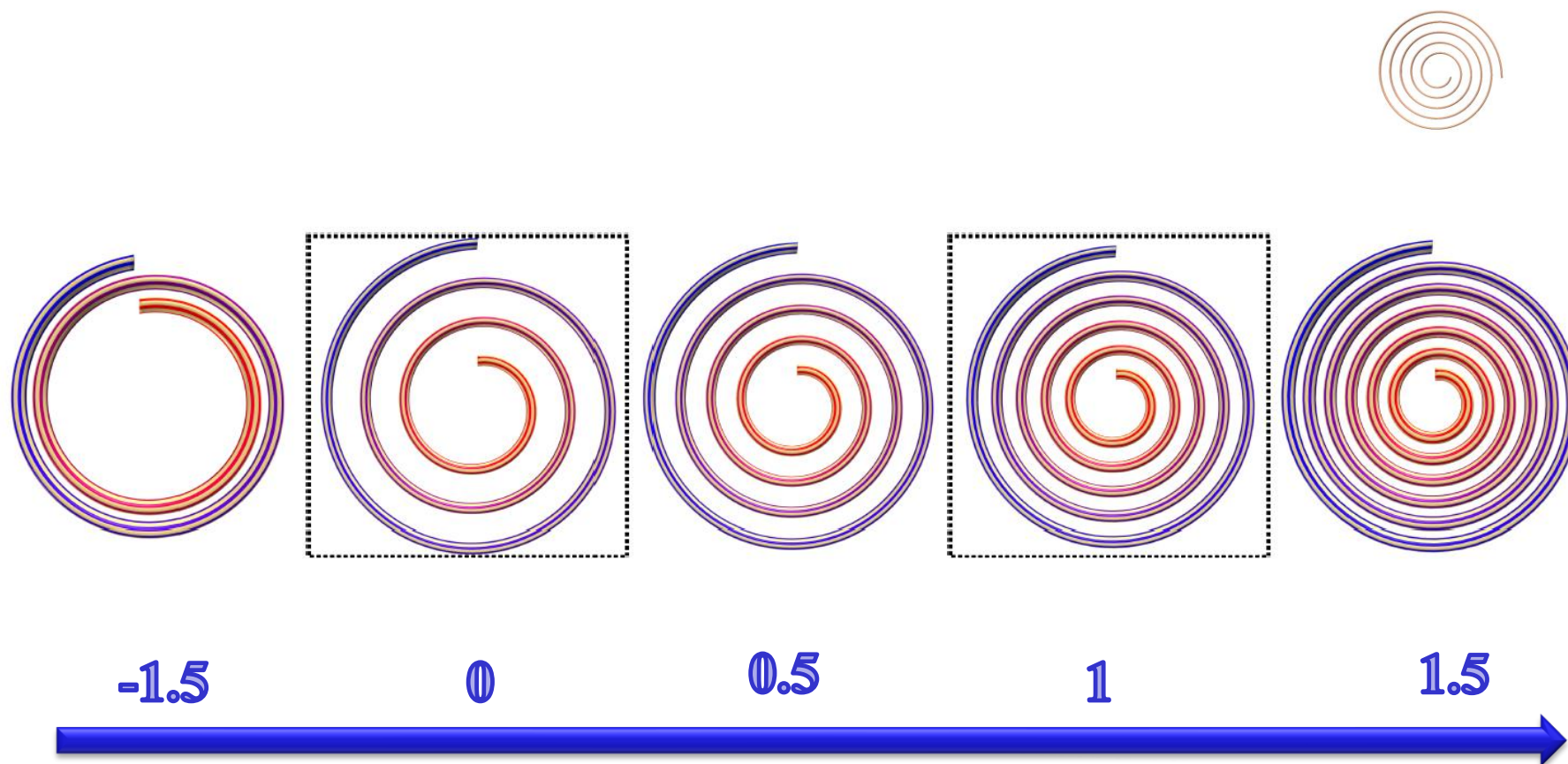
# MeshIK示例

**MeshIK**

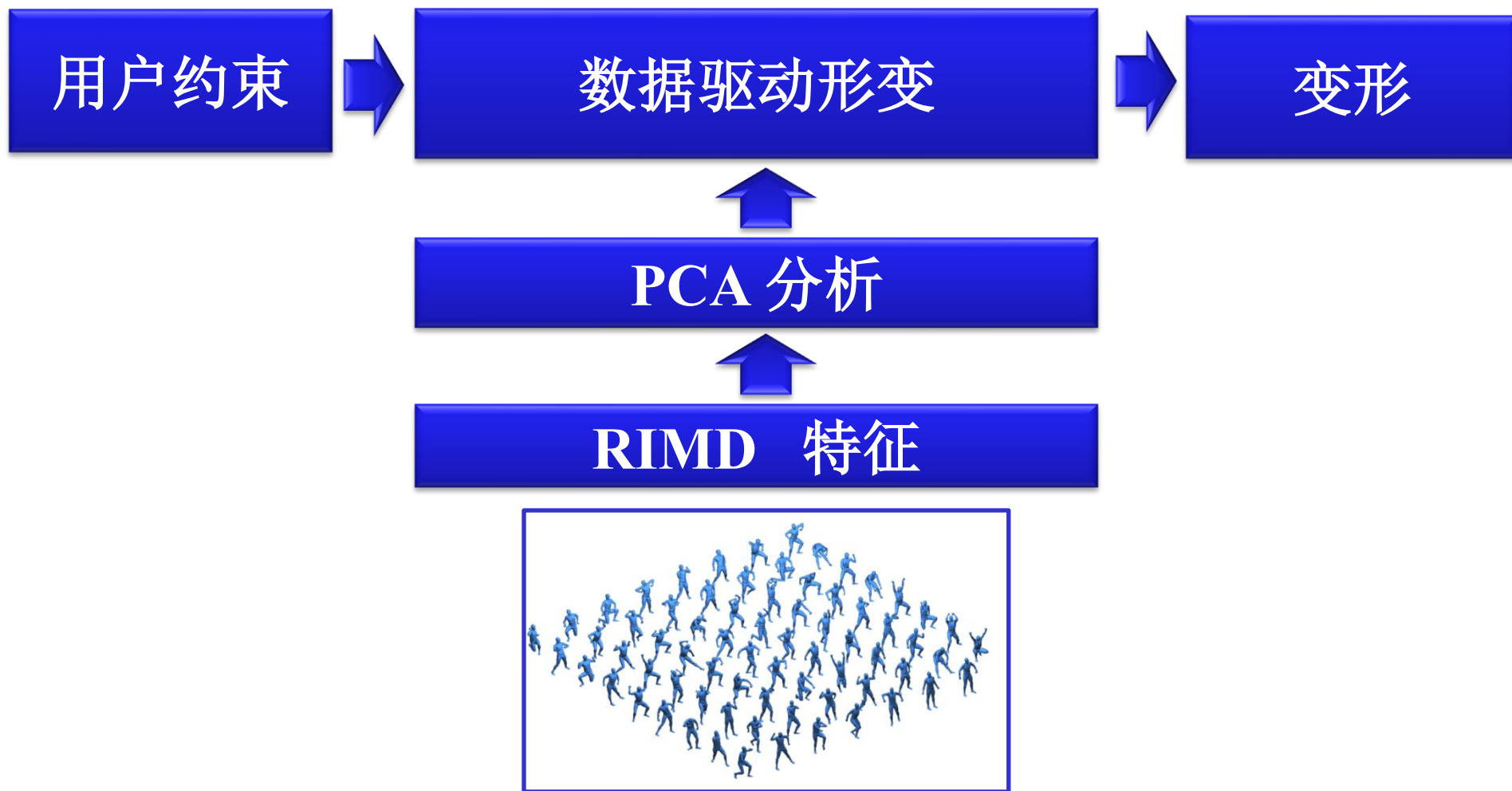


**Our**

# 插值(Interpolation) & 外推(Extrapolation)



# 数据驱动的形变



# 数据驱动的形变

---

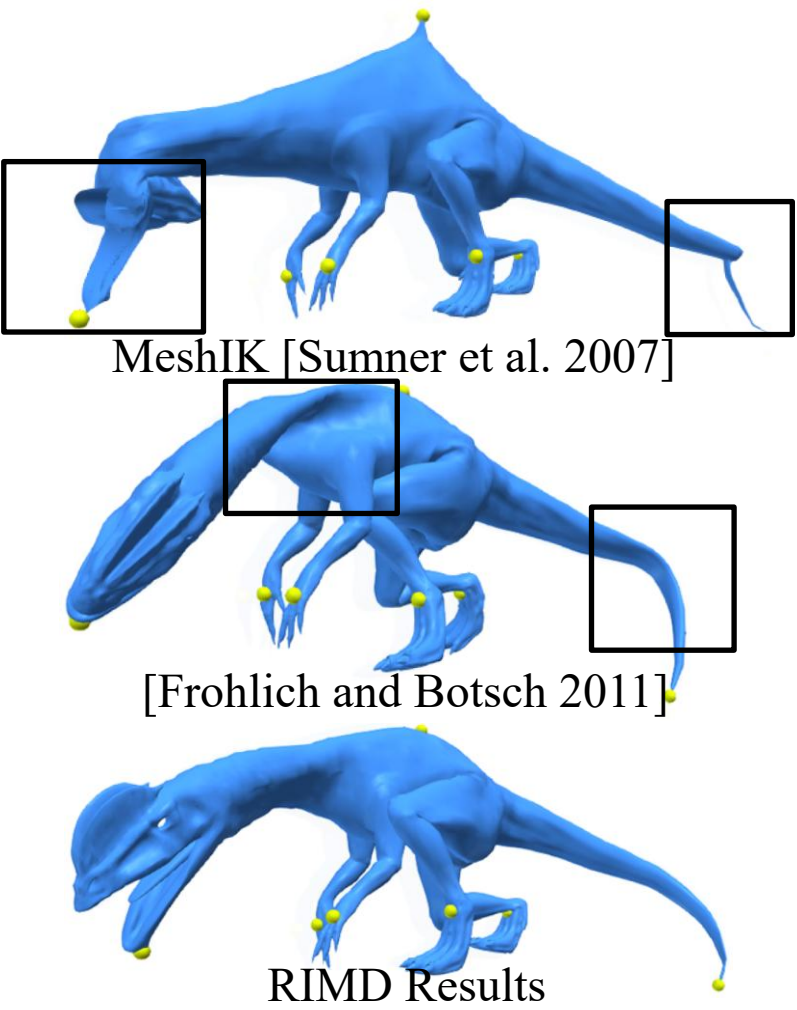
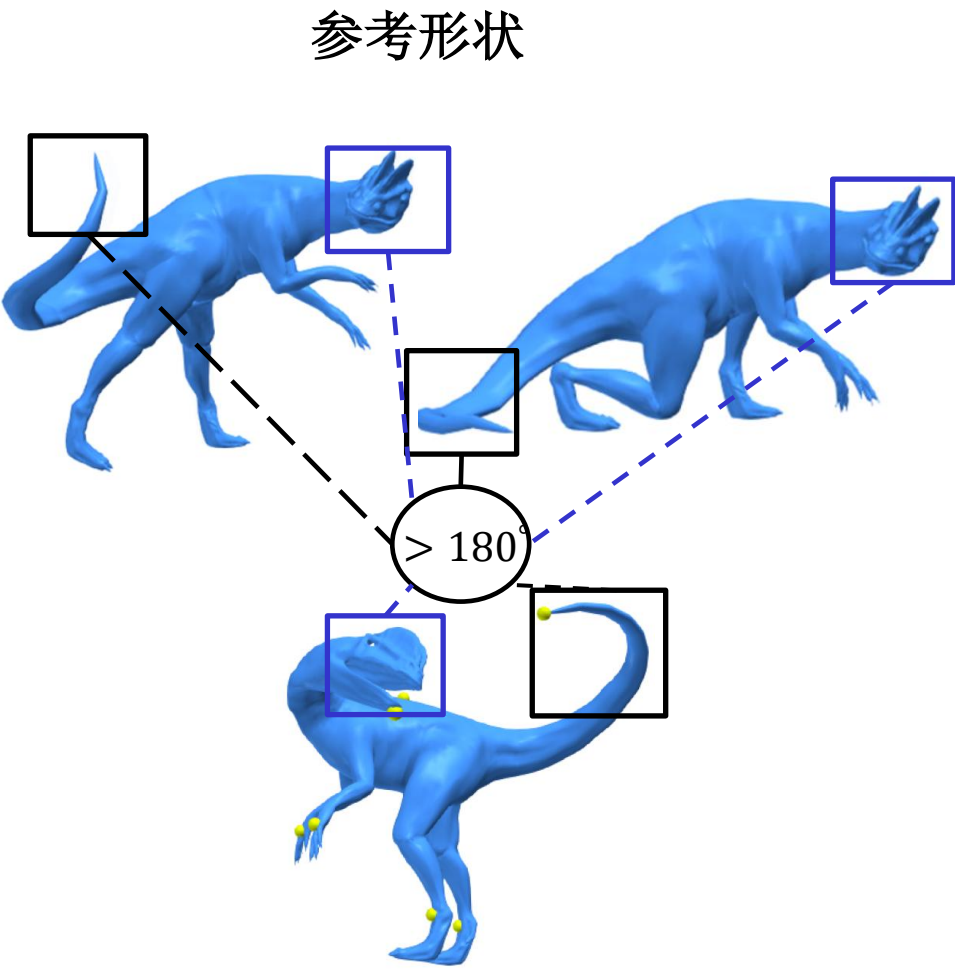
**While**

通过梯度下降(gradient descent)和线索搜(line search)优化权重

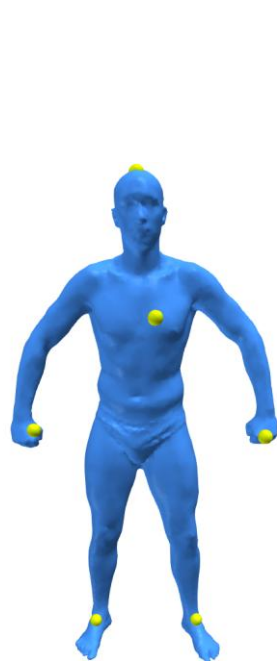
通过重构算法根据权重优化位置 $p$

**Until** 线搜索步幅大小  $r < \tilde{\epsilon}$

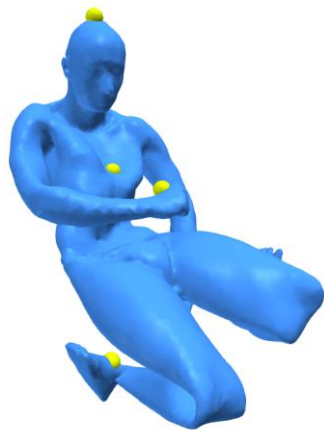
# 数据驱动的形状



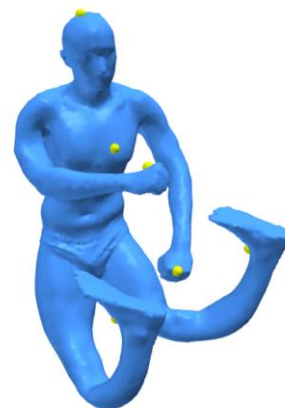
# 形变



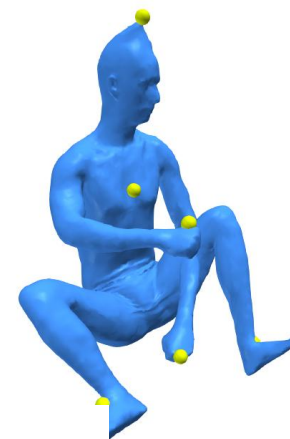
Input



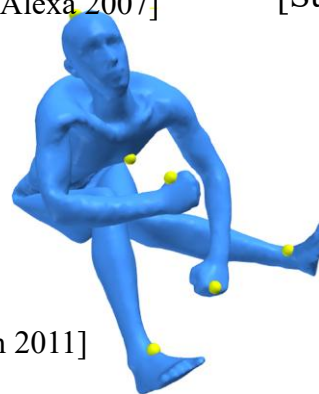
ARAP [Sorkine and Alexa 2007]



[Sumner et al. 2007]



MeshIK [Sumner et al. 2005]



[Frohlich and Botsch 2011]



RIMD

## Data-Driven Deformation

Real Time Screen Recording



# 本次课程内容

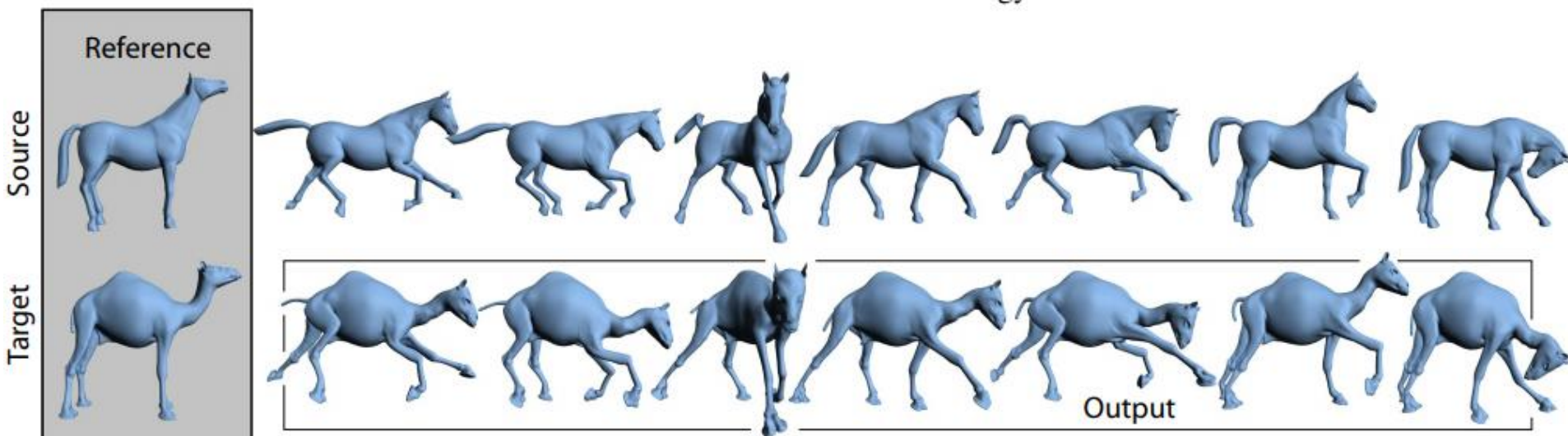
---

- 基于微分方法的网格编辑与变形技术
  - ◆ 基于Laplace坐标的网格编辑
  - ◆ 基于Poisson方程的网格编辑
- 网格变形的迁移

# 网格变形的迁移

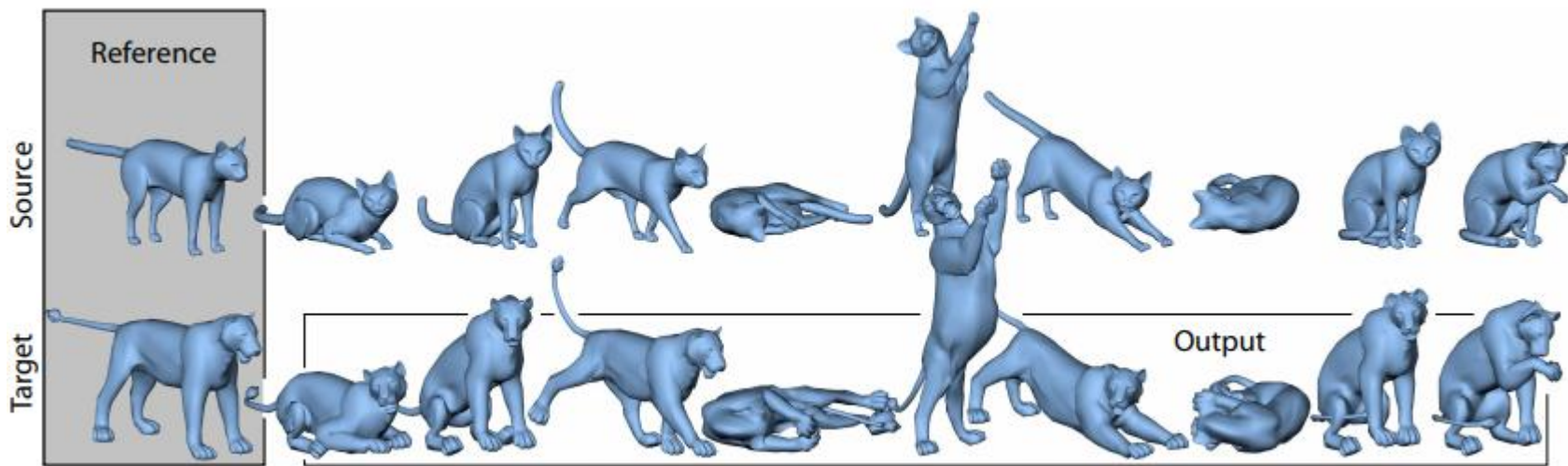
■ 用户对一个网格模型进行的变形编辑，能否自动地迁移到另一个相似的模型上？

- ◆ 用户编辑：站立的马 → 奔跑的马
- ◆ 自动转换：站立骆驼 → 奔跑骆驼？



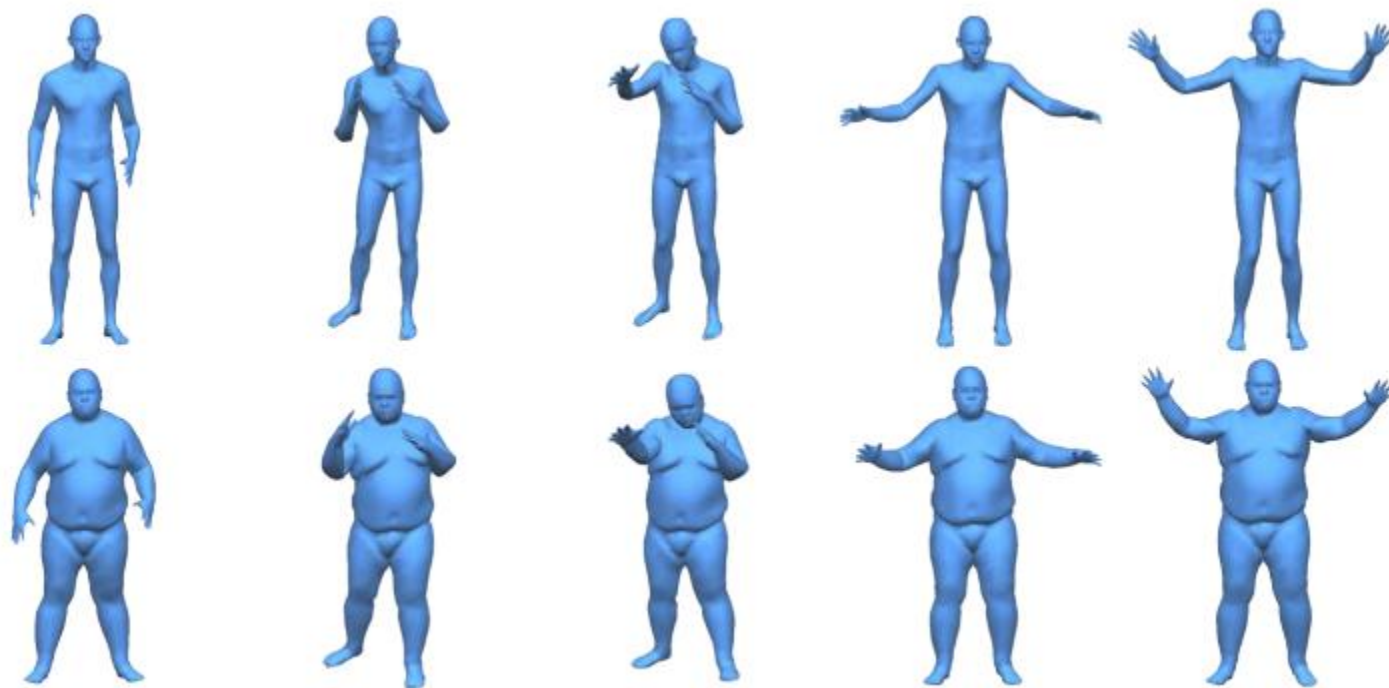
# 网格变形的迁移

- 传统方法：用户指定源网格与目标网格上若干个三角形的对应关系，求解约束优化问题，使对应三角形的变换矩阵差异最小
- Robert W. Sumner and Jovan Popovic'. Deformation Transfer for Triangle Meshes. SIGGRAPH 2004 (1518 cites)



# 网格变形的迁移

- 基于机器学习的变形迁移方法
- Gao et al. Automatic Unpaired Shape Deformation Transfer. ACM SIGGRAPH Asia 2018



# 网格变形的迁移

- 问题描述：给定两组不同种类的网格模型集合 $S$ 和 $T$ ，二者之间没有一一对应关系。现在有一个经过变形的 $S$ 种类的模型 $s$ ，希望得到一个具有类似变形的 $T$ 种类的模型 $t$

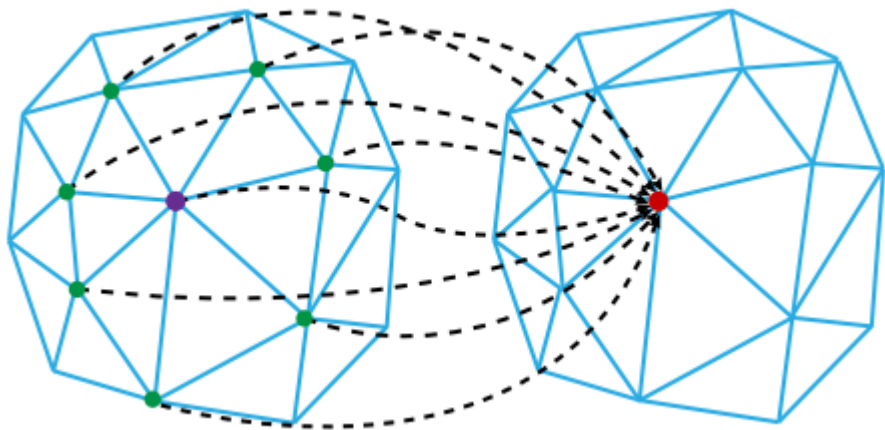
# 网格变形的迁移

## ■ 预备知识：网格上的卷积运算

- ◆ 假设网格上的第 $i$ 个顶点有特征向量 $s_i$
- ◆  $i$ 号顶点有 $D_i$ 个邻居，第 $j$ 个邻居编号为 $n_{ij}$

## ■ 卷积定义为 $i$ 号顶点与其邻居特征向量的加权平均

■ 
$$y_i = W_{point}s_i + W_{neighbor} \frac{1}{D_i} \sum_{j=1}^{D_i} s_{n_{ij}} + b$$



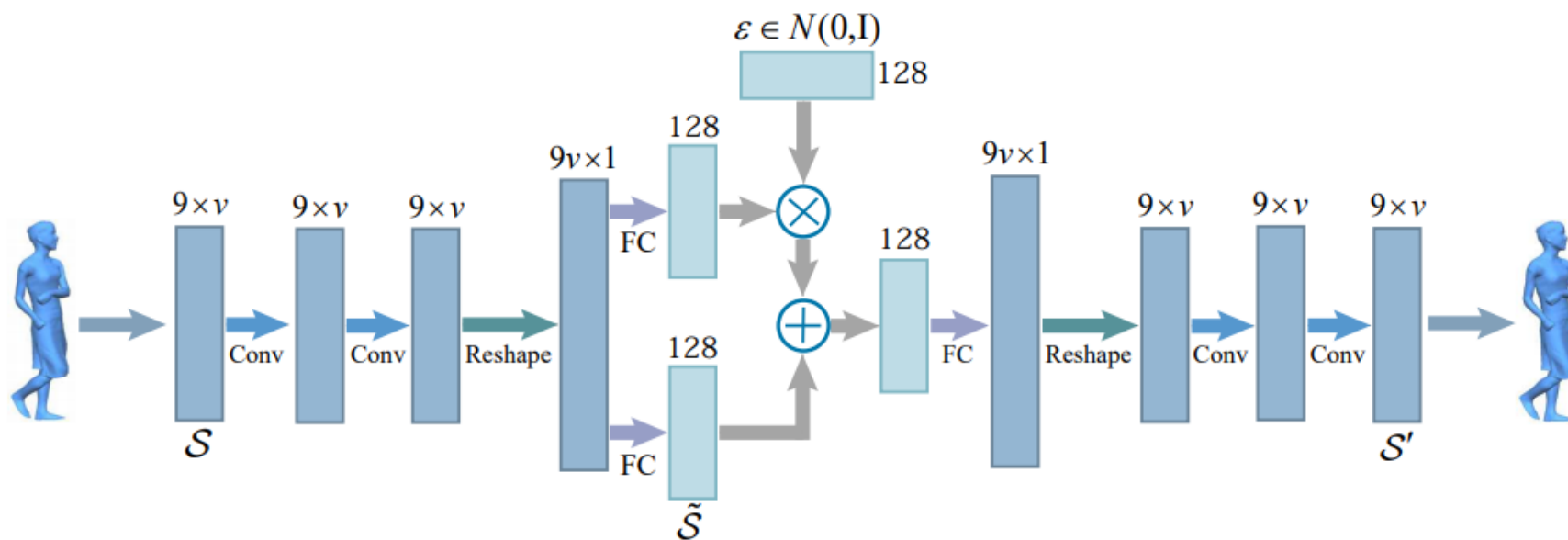
# 网格变形的迁移

---

- 预备知识：变分自编码器
- 自编码器是一种数据压缩的方法，由编码器（encoder）和解码器（decoder）两个神经网络组成，编码器将高维的数据转换成维度较低的隐含层特征，解码器再将隐含层特征还原回原始的数据

# 网格变形的迁移

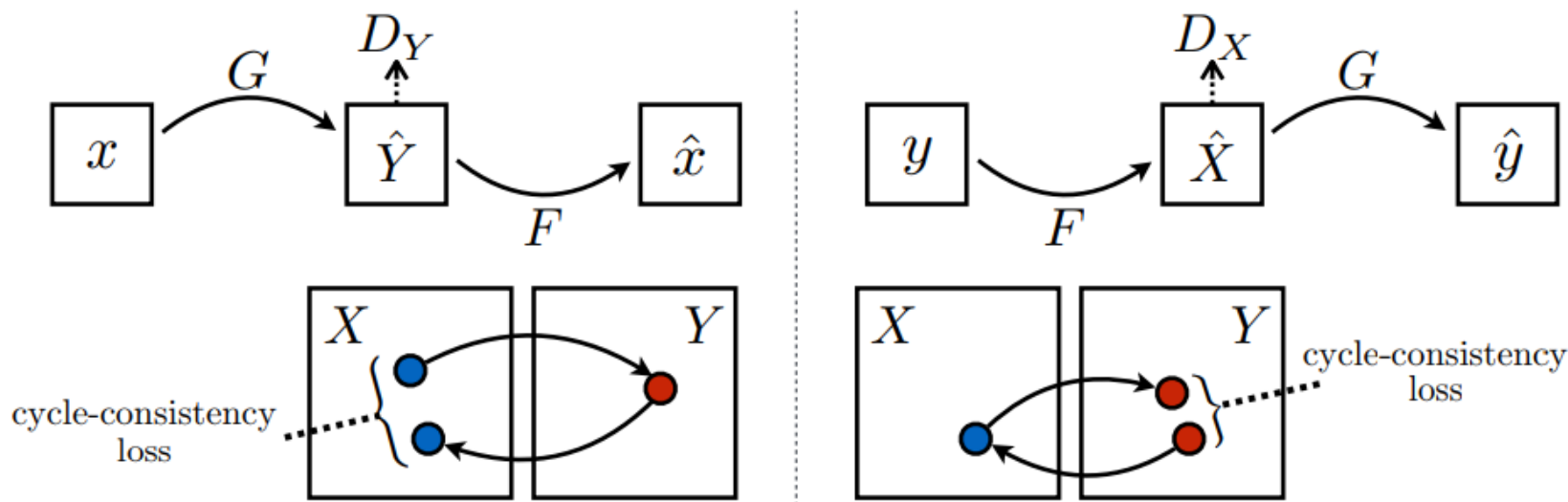
- 预备知识：变分自编码器
- 变分自编码器将隐含层特征表示为一个正态分布模型，编码器预测的是正态分布的均值 $\mu$ 和方差 $\Sigma$ ，解码器的输入 $z$ 是从正态分布 $N(\mu, \Sigma)$ 中采样得到的





# 网格变形的迁移

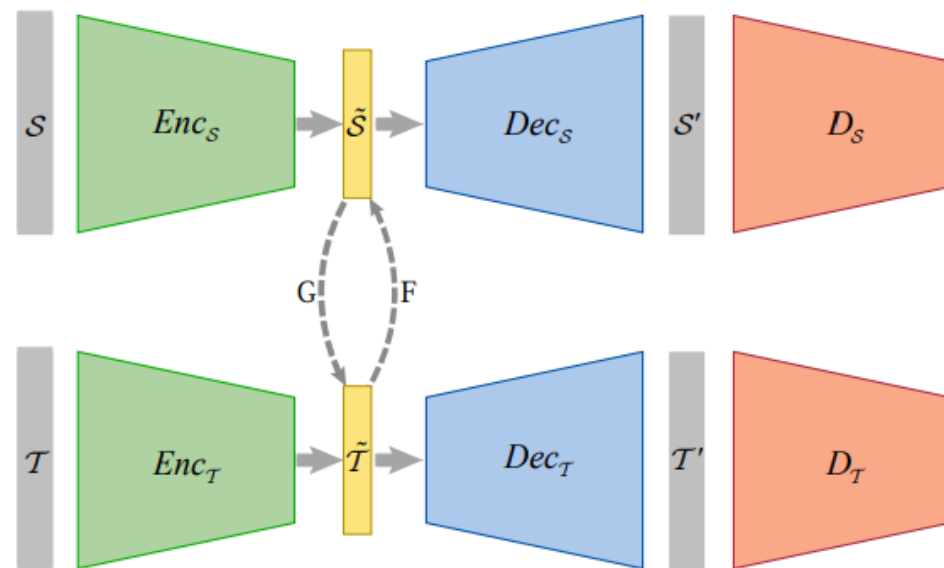
- 预备知识: CycleGAN
- 在没有成对训练数据的风格迁移问题中, 同时训练风格X到风格Y的迁移网络G和风格Y到风格X的迁移网络F, 并添加循环一致性约束:  $F(G(x)) \approx x, G(F(y)) \approx y$



# 网格变形的迁移

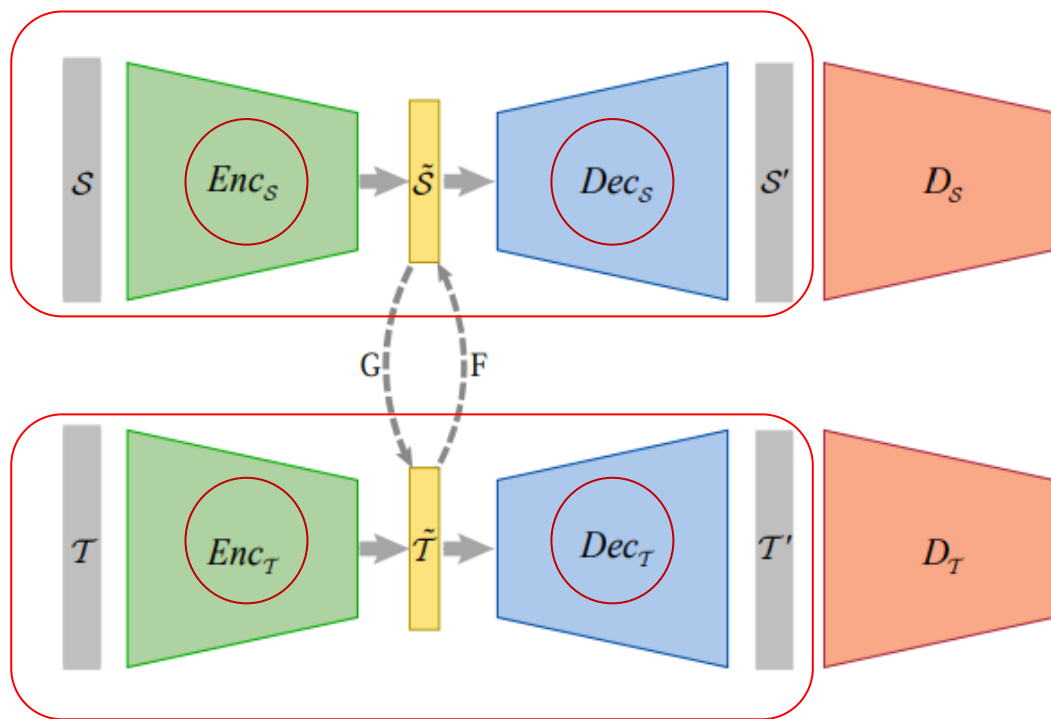
## ■ 算法概要

- ◆ 对网格类型 $S$ 和 $T$ 分别训练变分自编码器 $Enc$ 和 $Dec$ ，将其映射到隐含层向量 $\tilde{S}$ 和 $\tilde{T}$
- ◆ 训练一个网络 $SimNet$ 来估计两个网格模型之间的视觉相似性，即用神经网络近似计算光场距离（light field distance）
- ◆ 训练两个风格转换的网络 $G$ 和 $F$ 及两个风格的判别器 $D$ ，分别实现从类型 $\tilde{S}$ 到 $\tilde{T}$ 以及 $\tilde{T}$ 到 $\tilde{S}$ 的转换



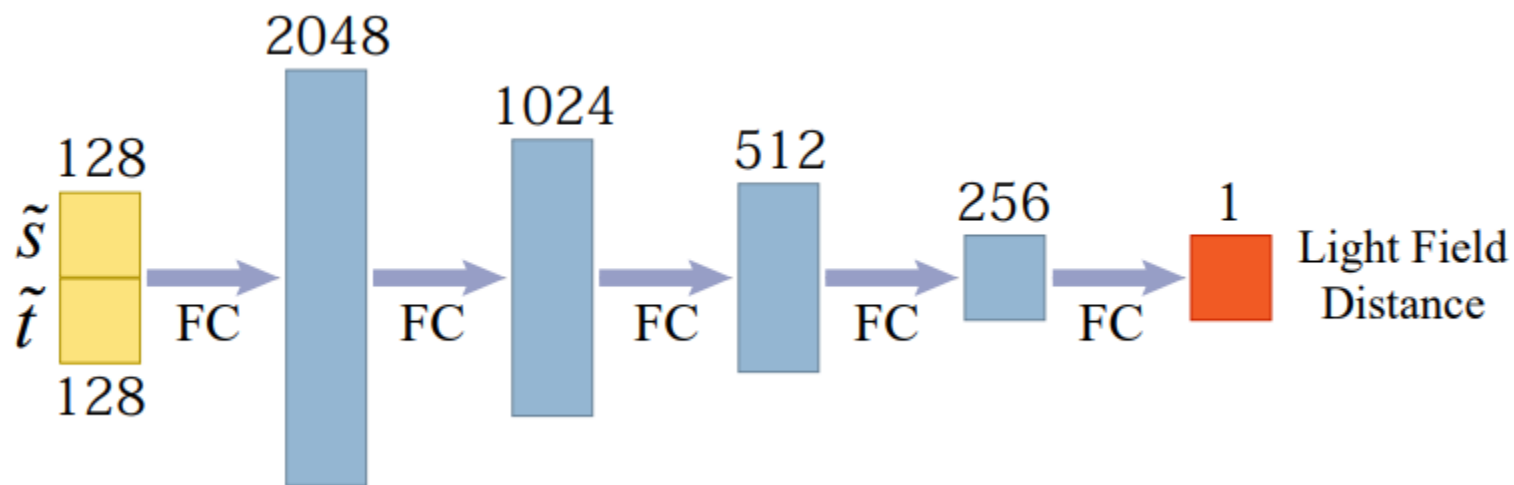
# 网格变形的迁移

- 对网格类型 $S$ 和 $T$ 分别训练变分自编码器 $Enc$ 和 $Dec$ ，将其映射到隐含层向量 $\tilde{S}$ 和 $\tilde{T}$ ，目标是重构输入的网格模型，将训练收敛后得到的 $Enc_S, Dec_S, Enc_T, Dec_T$ 固定



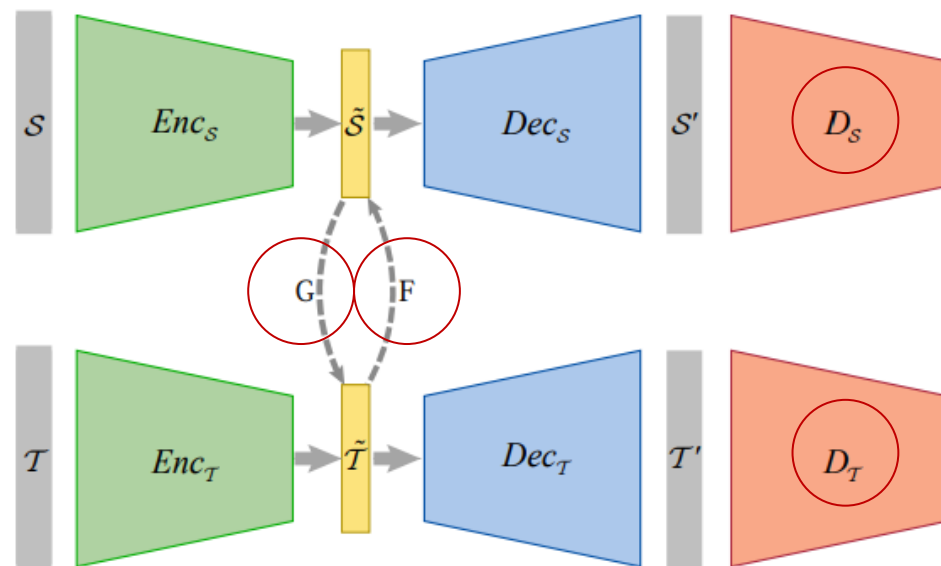
# 网格变形的迁移

- 训练一个网络SimNet来估计两个网格模型之间的视觉相似性，即用神经网络近似计算光场距离（light field distance）
- 因为光场距离的计算不可导，所以要用可导的神经网络进行近似，输入直接采用隐含层向量 $\tilde{s}$ 和 $\tilde{t}$



# 网格变形的迁移

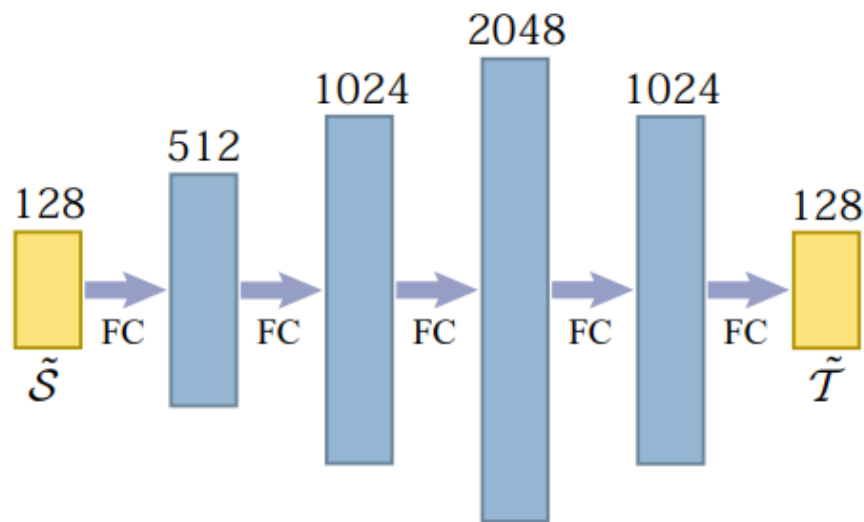
- 训练两个风格转换的网络 $G$ 和 $F$ 及两个风格的判别器 $D$ ，分别实现从类型 $\tilde{S}$ 到 $\tilde{T}$ 以及 $\tilde{T}$ 到 $\tilde{S}$ 的转换
- 采用CycleGAN的训练方法，使用SimNet近似的光场距离，循环一致性误差和判别器误差作为损失函数



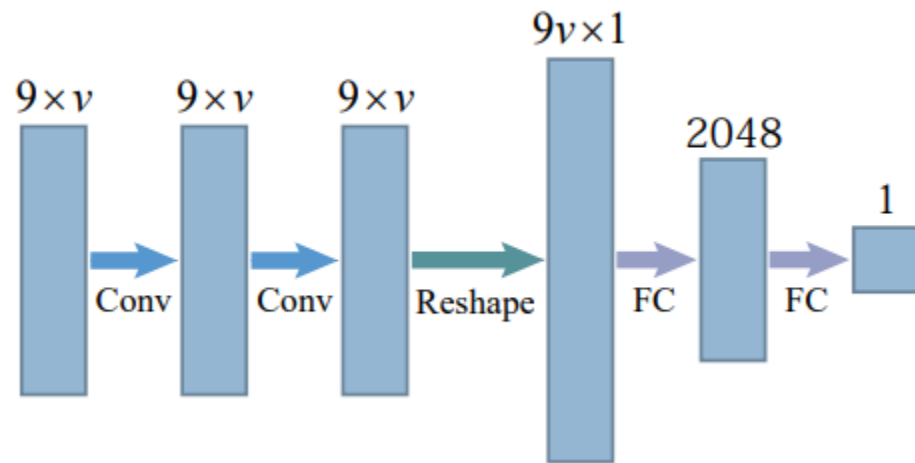
# 网格变形的迁移

- 训练两个风格转换的网络 $G$ 和 $F$ 及两个风格的判别器 $D$ ，分别实现从类型 $\tilde{S}$ 到 $\tilde{T}$ 以及 $\tilde{T}$ 到 $\tilde{S}$ 的转换

$G$ 和 $F$ 的结构

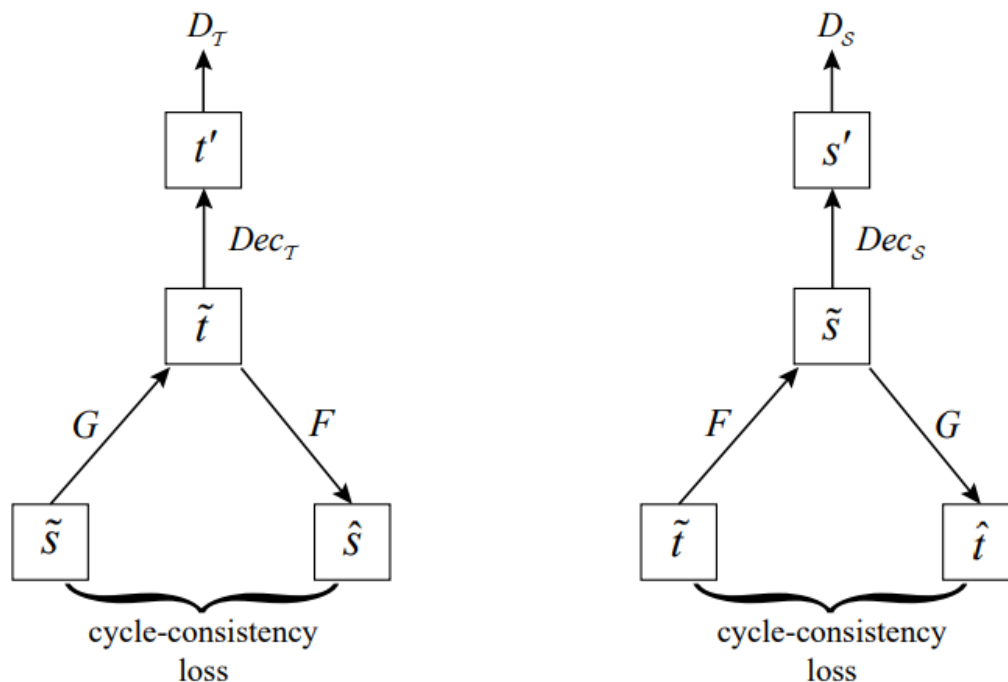


$D$ 的结构



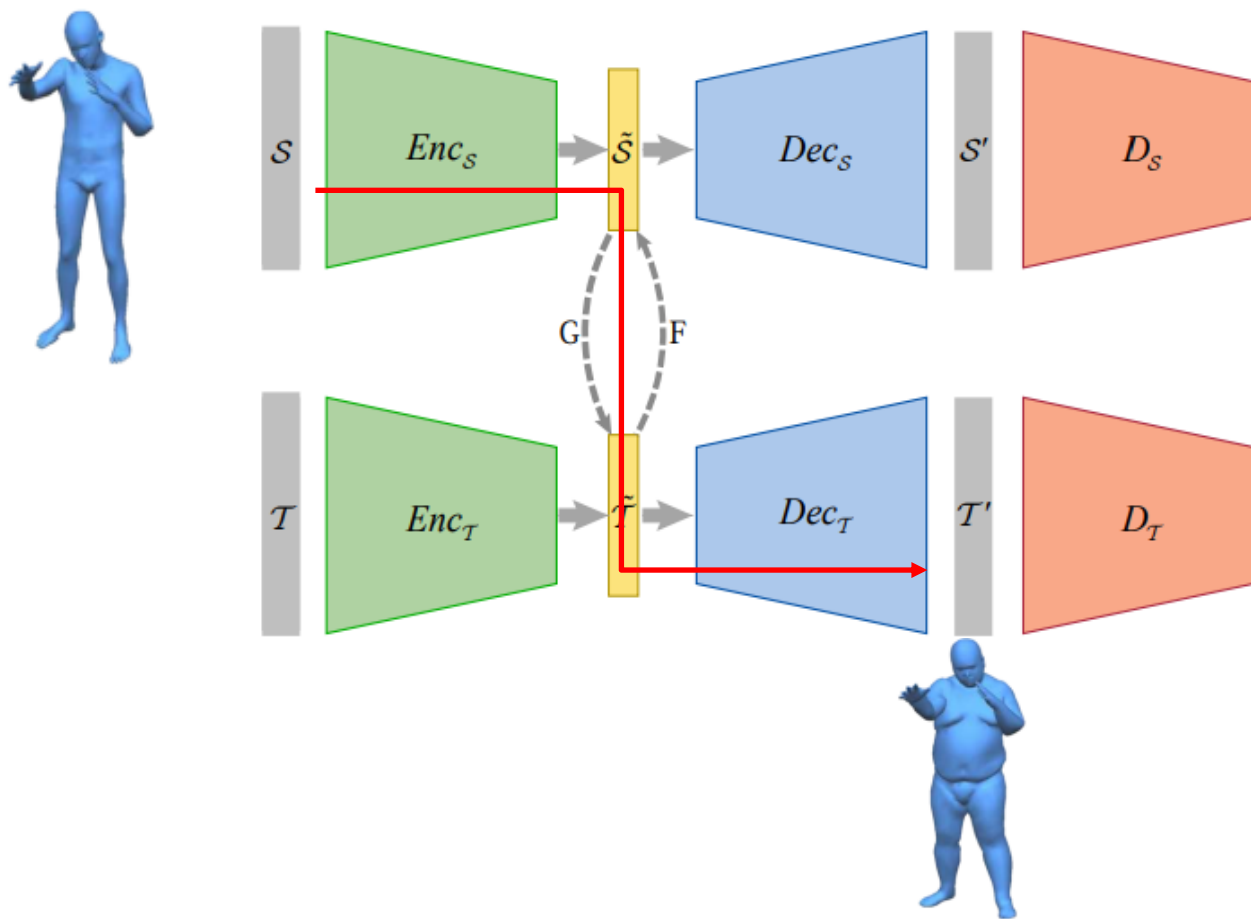
# 网格变形的迁移

- 训练两个风格转换的网络 $G$ 和 $F$ 及两个风格的判别器 $D$ ，分别实现从类型 $\tilde{S}$ 到 $\tilde{T}$ 以及 $\tilde{T}$ 到 $\tilde{S}$ 的转换
- CycleGAN训练时的数据流



# 网格变形的迁移

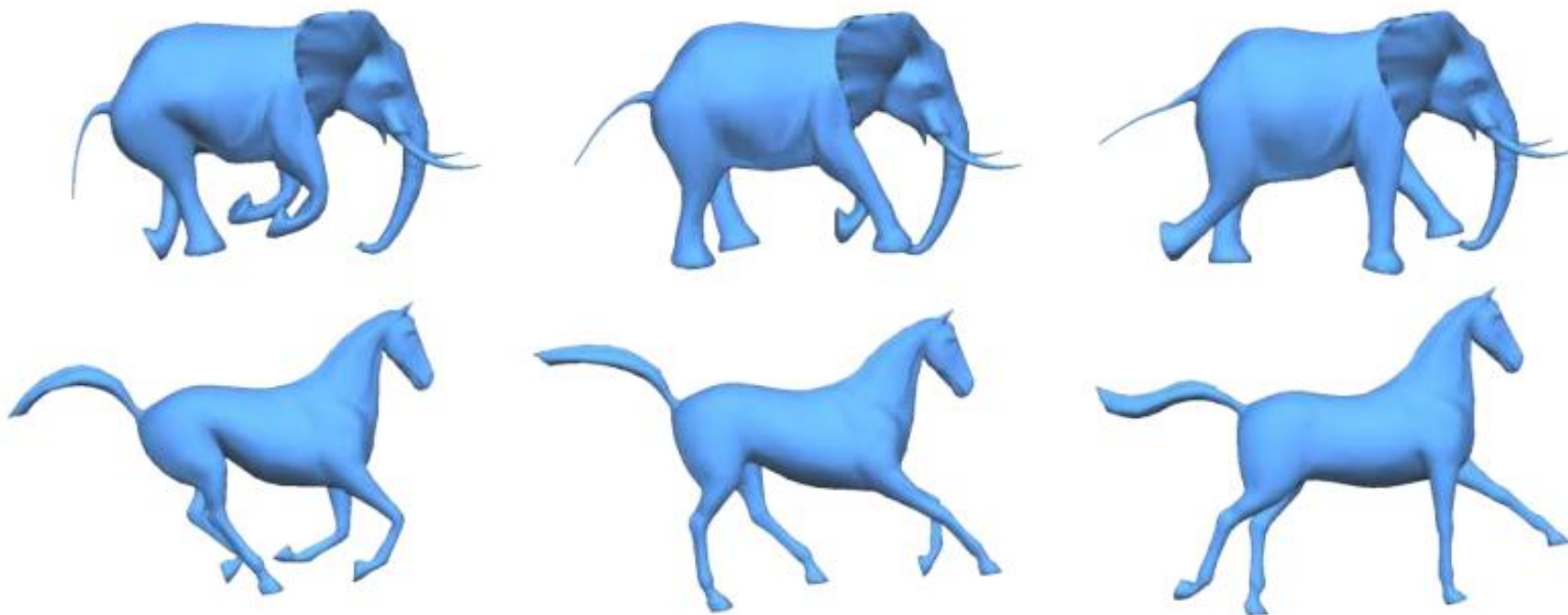
- 网络应用到新的模型 $s$ 上，得到具有相似变形的模型 $t$





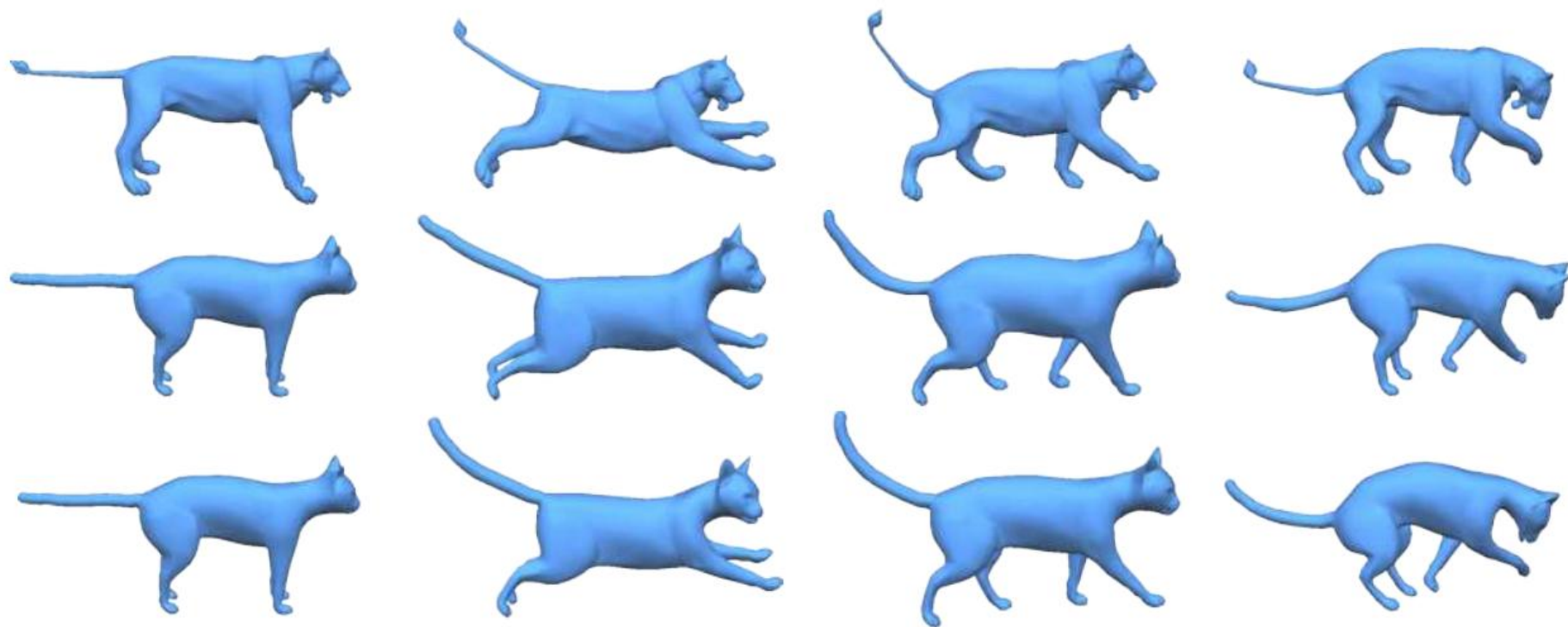
# 网格变形的迁移

## ■ 更多结果



# 网格变形的迁移

- 更多结果：第一行为输入，第二行为Sumner的结果，第三行为该方法的结果



## RESULTS AND EVALUATION

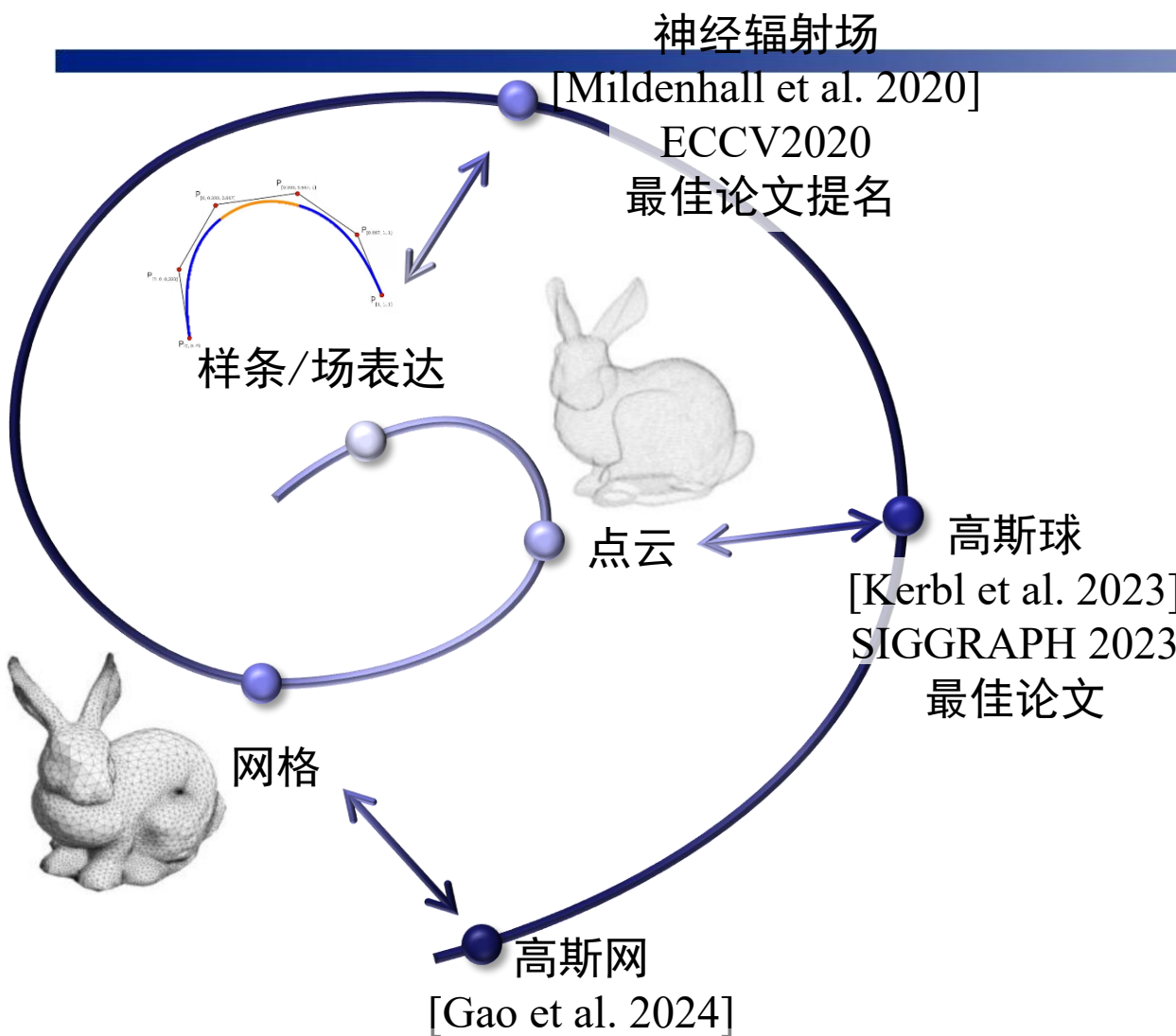


Female



Asimo-Like Robot

# 几何表示进展

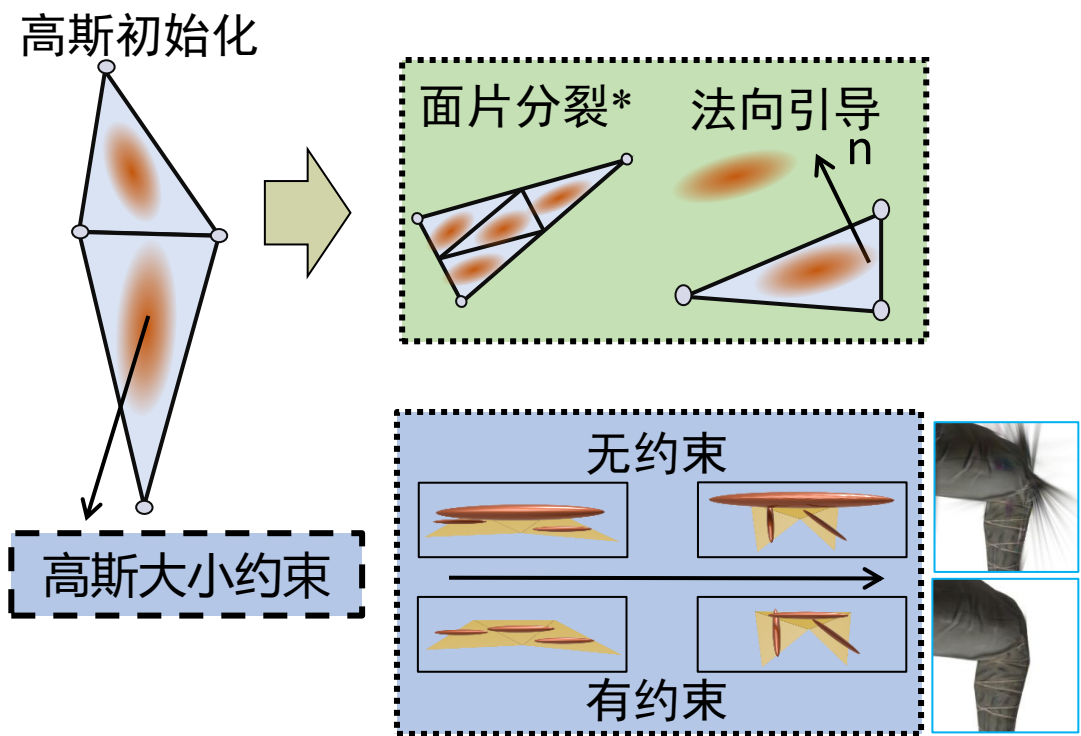


|       | R                        | E               | A                                            | L               |
|-------|--------------------------|-----------------|----------------------------------------------|-----------------|
|       | Photo-Realistic<br>照片级真实 | Efficient<br>效率 | Scalable<br>Large-Scale Deformation<br>大尺度形变 | Low-Cost<br>低成本 |
| 神经辐射场 | ✓                        | ✗               | ✗                                            | ✓               |
| 高斯球   | ✓                        | ✓               | ✗                                            | ✓               |
| 高斯网   | ✓                        | ✓               | ✓                                            | ✓               |



# 基于高斯网的交互式变形方法

- 基于网格的高斯优化：
  - ◆ 面片分裂：高斯的渲染引导网格面分割以进行自适应细化，并且网格面分割指导高斯球的分裂
  - ◆ 法向引导：沿面片法向优化高斯位置



$$\mu = (w_a \mathbf{v}_a + w_b \mathbf{v}_b + w_c \mathbf{v}_c) + \tau R \mathbf{n},$$

重心坐标:  $[w_a, w_b, w_c]$   
沿法向位移:  $\tau R$ ,  $R$ 是当前面片外接圆半径,  $\mathbf{n}$ 是面片单位法向

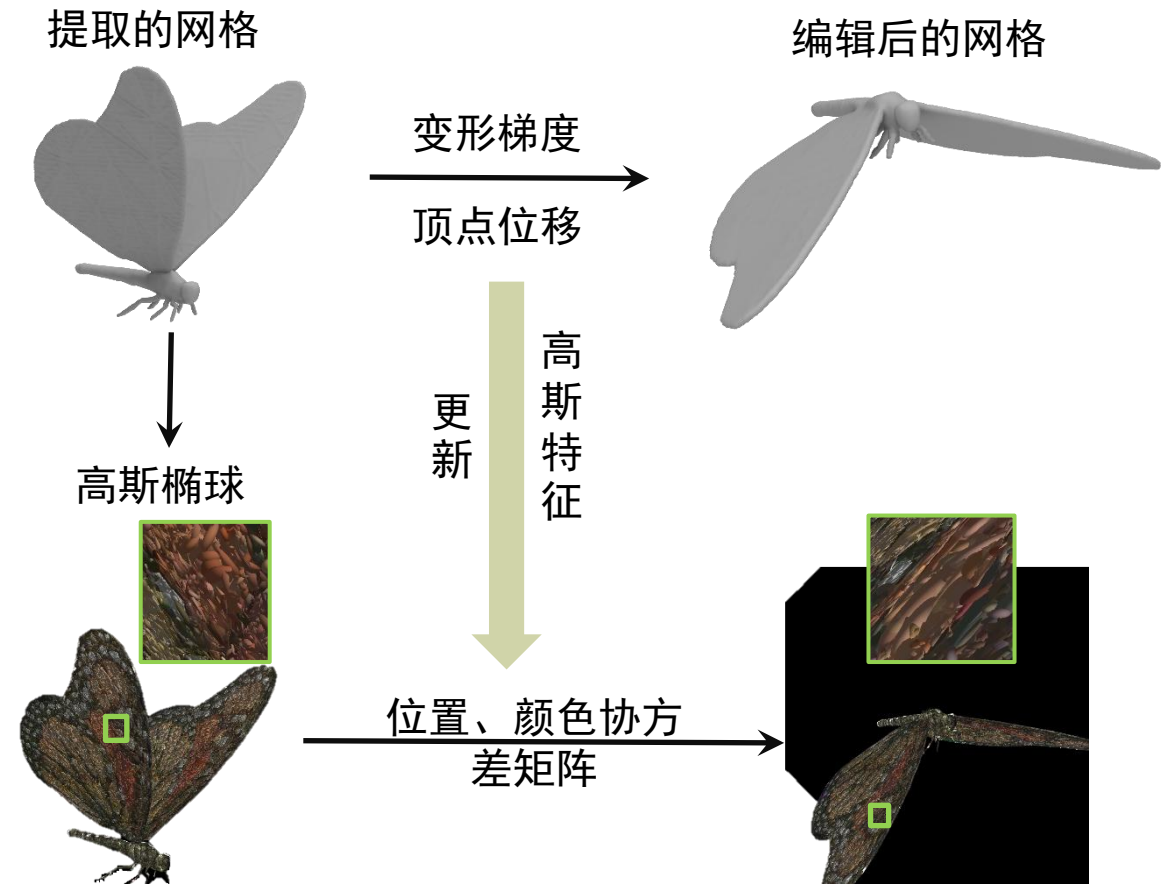
$$L_r = \sum_{g \in G} \max(\max(s_i) - \gamma R_i, 0)$$

高斯大小约束:  
 $\gamma$ : 超参  
 $R_i$ : 面片外接圆半径  
 $s_i$ : 高斯核的scale

网格约束高斯的高斯网表示

# 基于高斯网的交互式变形方法

- 变形传递：利用网格变形方法中的顶点位置和变形梯度引导高斯变化



变形: 得到变形梯度  $T_i = R_i S_i$

$$E(\mathcal{M}') = \sum_{i=1}^N \sum_{j \in N(i)} w_{ij} \left\| (v'_i - v'_j) - T_i (v_i - v_j) \right\|^2,$$

更新高斯核参数

$$\begin{aligned} \Delta P &= w_a(v'_a - v_a) + w_b(v'_b - v_b) + w_c(v'_c + v_c), \\ \bar{R}_i &= w_a \log(\bar{R}_{v'_a}) + w_b \log(\bar{R}_{v'_b}) + w_c \log(\bar{R}_{v'_c}), \\ \bar{S}_i &= w_a \bar{S}_{v'_a} + w_b \bar{S}_{v'_b} + w_c \bar{S}_{v'_c}, T_i = \exp(\bar{R}_i) \bar{S}_i. \end{aligned}$$

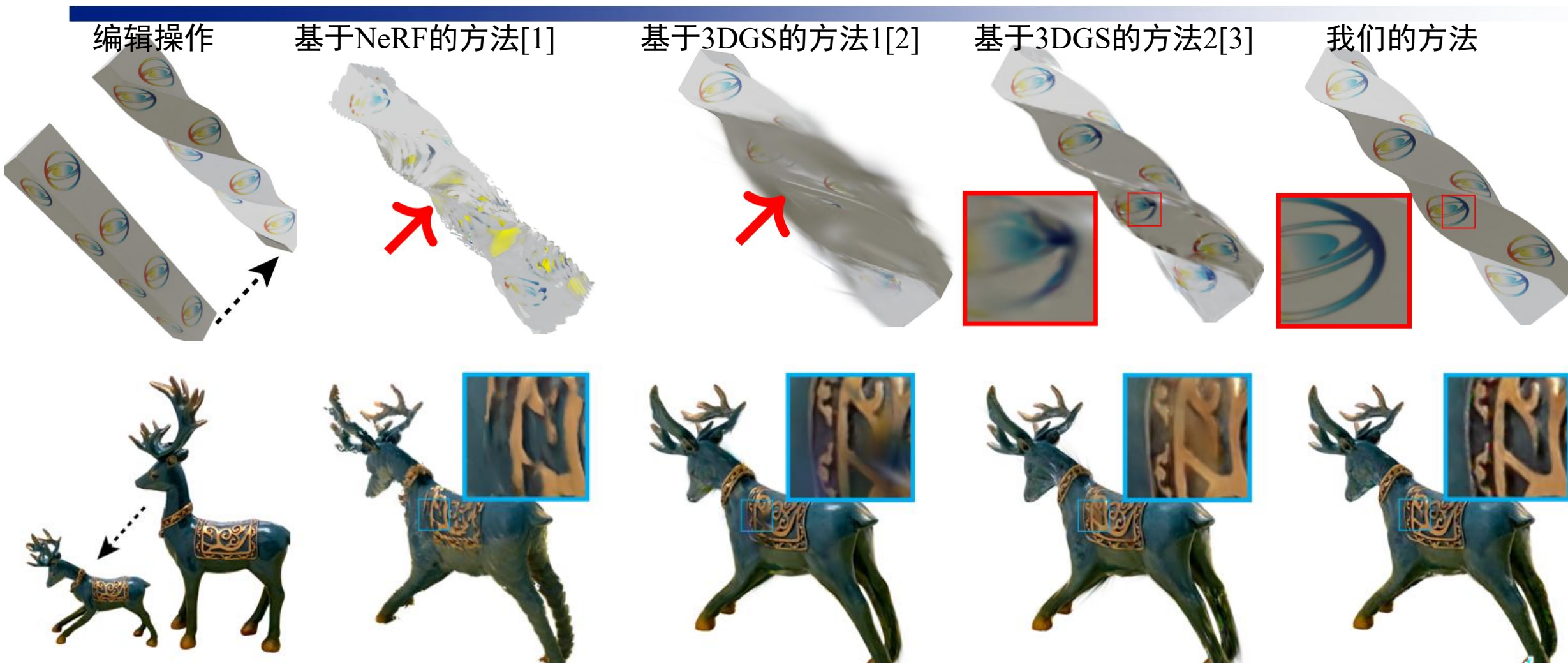
变形后的高斯核

$$g'(x) = e^{-\frac{1}{2}(x - (\mu + \Delta P))^T (T_i \Sigma T_i^T)^{-1} (x - (\mu + \Delta P))}.$$

变形传递



# 基于网格的高斯交互式编辑方法



[1] NeRF-Editing: Geometry Editing of Neural Radiance Fields.

[2] 3d gaussian splatting for real-time radiance field rendering.

[3] SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering.



# 基于网格的高斯交互式编辑方法

## ■ 实时高质量交互式高斯变形工具



Human  
Avg. FPS: 32  
# Gaussian: 615975



Cylinder  
Avg. FPS: 70  
# Gaussians: 165636



Butterfly  
Avg. FPS: 59  
# Gaussians: 285797



# 基于网格的高斯交互式编辑方法



**Lin Gao\***, Jie Yang, Bo-Tao Zhang, Jia-Mu Sun, Yu-Jie Yuan, Hongbo Fu, Yu-Kun Lai. Mesh-based Gaussian Splatting for Real-time Large-scale Deformation, **ACM Transactions on Graphics (SIGGRAPH ASIA 2024)**



# 基于网格的高斯交互式编辑方法



**Lin Gao\***, Jie Yang, Bo-Tao Zhang, Jia-Mu Sun, Yu-Jie Yuan, Hongbo Fu, Yu-Kun Lai. Mesh-based Gaussian Splatting for Real-time Large-scale Deformation, **ACM Transactions on Graphics (SIGGRAPH ASIA 2024)**



谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn